

# Diplomarbeit: Lastenroboter

Höhere Technische Bundeslehranstalt Graz Gösting  
Schuljahr 2024/25



**Diplomanden:**

Daniel Schauer 5AHEL  
Simon Spari 5AHEL  
Felix Hochegger 5AHEL

**Betreuer:**

Prof. DI. Gernot Mörtl

---

# Eidesstattliche Erklärung

Wir erklären an Eides statt, dass wir die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht haben.

---

Ort, am TT.MM.JJJJ

---

Daniel Schauer

---

Simon Spari

---

Felix Hochegger

---

# Danksagung

An dieser Stelle möchten wir unseren aufrichtigen Dank aussprechen.

Ein besonderer Dank gilt Herrn Prof. DI Gernot Mörtl für seine wertvolle Unterstützung, seine fachliche Begleitung und seine konstruktiven Anregungen während der gesamten Arbeit. Seine Expertise und sein Engagement haben maßgeblich zum Gelingen dieser Diplomarbeit beigetragen.

Ebenso danken wir unseren Freunden, insbesondere Michael Johannes Anderhuber, für seine Unterstützung beim Schweißen des Gehäuses. Sein handwerkliches Geschick und seine Hilfe waren für die Umsetzung unseres Projekts von großem Wert.

Unser großer Dank gilt zudem unserem großzügigen Sponsor, "Vogl Baumarkt Rosental", für das Sponsoring des Metalls für das Gehäuse. Durch diese Unterstützung konnten wir unser Projekt in dieser Form verwirklichen.

# Inhaltsverzeichnis

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                 | <b>7</b>  |
| 1.1      | Kurzzusammenfassung . . . . .                     | 7         |
| 1.2      | Abstract . . . . .                                | 8         |
| <b>2</b> | <b>Projektmanagement</b>                          | <b>9</b>  |
| 2.1      | Projektteam . . . . .                             | 9         |
| 2.2      | Projektstrukturplan . . . . .                     | 10        |
| 2.3      | Meilensteine . . . . .                            | 11        |
| 2.4      | Kostenaufstellung . . . . .                       | 12        |
| <b>3</b> | <b>Antrieb</b>                                    | <b>13</b> |
| 3.1      | Motoren . . . . .                                 | 13        |
| 3.1.1    | Übersicht . . . . .                               | 13        |
| 3.1.2    | Funktionsweise . . . . .                          | 13        |
| 3.1.3    | Technische Daten . . . . .                        | 15        |
| 3.2      | Motorentreiber . . . . .                          | 15        |
| 3.2.1    | Überblick . . . . .                               | 15        |
| 3.2.2    | Aufbau und Funktionen . . . . .                   | 15        |
| 3.3      | Schaltungsaufbau mit einem Motor . . . . .        | 19        |
| 3.3.1    | Komponenten . . . . .                             | 19        |
| 3.3.2    | Schaltungsaufbau und Verbindungen . . . . .       | 19        |
| 3.4      | Schaltungsaufbau mit vier Motoren . . . . .       | 21        |
| 3.4.1    | Komponenten . . . . .                             | 21        |
| 3.4.2    | Schaltungserweiterung für vier Motoren: . . . . . | 22        |
| 3.5      | Code . . . . .                                    | 23        |
| 3.5.1    | Initialisierung . . . . .                         | 23        |
| 3.5.2    | Setup-Code . . . . .                              | 24        |
| 3.5.3    | Handling der Steuerbefehle . . . . .              | 25        |
| 3.5.4    | Ansteuerung der Treiberplatinen: . . . . .        | 26        |
| <b>4</b> | <b>Webserver</b>                                  | <b>30</b> |
| 4.1      | Grundlegende Ziele . . . . .                      | 30        |
| 4.2      | Webserver . . . . .                               | 31        |
| 4.2.1    | Webserver Setup . . . . .                         | 31        |
| 4.2.2    | SPIFFS Setup . . . . .                            | 32        |
| 4.3      | WebSocket Kommunikation . . . . .                 | 34        |
| 4.3.1    | Kommunikation Setup . . . . .                     | 34        |



---

|          |                                                             |            |
|----------|-------------------------------------------------------------|------------|
| 4.3.2    | Message Handling . . . . .                                  | 35         |
| 4.4      | Website . . . . .                                           | 39         |
| 4.4.1    | Implementierung der Steuerung . . . . .                     | 39         |
| 4.4.2    | Anzeige von Sensordaten und Verbindungsstatus . . . . .     | 50         |
| 4.5      | Herausforderungen und Optimierungen . . . . .               | 57         |
| 4.5.1    | Latenz- und Performance Optimierungen . . . . .             | 57         |
| <b>5</b> | <b>Gehäuse</b>                                              | <b>58</b>  |
| 5.1      | Planung und Design . . . . .                                | 58         |
| 5.2      | Realisierung . . . . .                                      | 64         |
| 5.3      | Vor- und Nachteile von Elektrodenschweißen . . . . .        | 66         |
| 5.4      | Materialliste . . . . .                                     | 71         |
| 5.4.1    | Stahlliste . . . . .                                        | 71         |
| 5.4.2    | Komponentenliste . . . . .                                  | 71         |
| <b>6</b> | <b>Hardwaredesign</b>                                       | <b>72</b>  |
| 6.1      | Zielsetzung . . . . .                                       | 72         |
| 6.2      | Grundsaltungen . . . . .                                    | 73         |
| 6.2.1    | Schaltplan Steuerplatine . . . . .                          | 74         |
| 6.2.2    | 8. Expander_Anschlüsse (GPIO-Erweiterung über MCP23017) . . | 81         |
| 6.3      | Steuerplatine PCB . . . . .                                 | 86         |
| 6.4      | Leiterplatten herstellen mit JLCPCB . . . . .               | 87         |
| 6.5      | Bestückte Platine . . . . .                                 | 87         |
| <b>7</b> | <b>Kamera</b>                                               | <b>88</b>  |
| 7.1      | Kamera im Überblick . . . . .                               | 88         |
| 7.1.1    | Technische Daten . . . . .                                  | 89         |
| 7.1.2    | Nutzung . . . . .                                           | 89         |
| 7.2      | Code . . . . .                                              | 90         |
| 7.2.1    | Kamera Setup . . . . .                                      | 90         |
| 7.2.2    | Videoübertragung . . . . .                                  | 92         |
| 7.3      | Schwenkmechanismus der Kamera . . . . .                     | 95         |
| 7.3.1    | Servomotor . . . . .                                        | 95         |
| 7.3.2    | Nutzung . . . . .                                           | 96         |
| 7.3.3    | Verbindung und Steuerung des Servomotors . . . . .          | 96         |
| 7.3.4    | Gehäuse der Kameranachwenkung . . . . .                     | 99         |
| <b>8</b> | <b>Sensoren und Aktoren</b>                                 | <b>101</b> |
| 8.1      | Abstandsensor . . . . .                                     | 101        |

---

|           |                                            |            |
|-----------|--------------------------------------------|------------|
| 8.1.1     | Grundprinzip . . . . .                     | 101        |
| 8.1.2     | Schaltungsaufbau . . . . .                 | 103        |
| 8.1.3     | Nutzung . . . . .                          | 104        |
| 8.1.4     | Code . . . . .                             | 105        |
| 8.2       | Gewichtsmessung . . . . .                  | 106        |
| 8.2.1     | Grundprinzip . . . . .                     | 106        |
| 8.2.2     | Schaltungsaufbau . . . . .                 | 109        |
| 8.2.3     | Nutzung . . . . .                          | 110        |
| 8.2.4     | Code . . . . .                             | 111        |
| 8.3       | Spannungsmessung per ESP32 . . . . .       | 113        |
| 8.3.1     | Code . . . . .                             | 113        |
| 8.3.2     | Verbesserungen und Optimierungen . . . . . | 116        |
| 8.4       | Lichtsteuerung per Transistor . . . . .    | 116        |
| 8.4.1     | Code . . . . .                             | 116        |
| <b>9</b>  | <b>Entwicklungstools</b>                   | <b>119</b> |
| 9.1       | Autodesk Fusion . . . . .                  | 119        |
| 9.2       | Autodesk Eagle . . . . .                   | 119        |
| 9.3       | VS-Code . . . . .                          | 119        |
| 9.3.1     | Setup . . . . .                            | 120        |
| 9.3.2     | Bibliotheken . . . . .                     | 120        |
| 9.3.3     | Verwendete Bibliotheken . . . . .          | 120        |
| 9.4       | LaTeX . . . . .                            | 122        |
| 9.5       | GitHub . . . . .                           | 122        |
| <b>10</b> | <b>Abbildungsverzeichnis</b>               | <b>123</b> |
| <b>11</b> | <b>Literaturverzeichnis</b>                | <b>128</b> |

---

# 1 Einleitung

## 1.1 Kurzzusammenfassung

In dieser Diplomarbeit wird ein Lastenroboter entwickelt, der bis zu 25 Kilogramm transportieren kann. Der Roboter wird über eine Website gesteuert, die als Steuerungsplattform dient. Zusätzlich ist eine Kamera eingebaut, die zur visuellen Überwachung des Transportbereichs dient, sowie eine Waage, die das Gewicht der transportierten Last misst.

Ein Schwerpunkt der Arbeit liegt auf der mechanischen Konstruktion des Roboters, bei der ein stabiles Gehäuse aus Stahl gebaut wird, um Sicherheit und Stabilität zu gewährleisten. Außerdem wird eine eigene Platine entwickelt, die die verschiedenen Hardware-Komponenten, wie die Sensoren und die Motoren miteinander verbindet.

Die Steuerung des Roboters erfolgt über eine Website, welche die Bedienung sowie die Anzeige relevanter Daten wie Akkustand und Gewicht ermöglicht. Ein besonderer Fokus liegt dabei auf der Übertragung des Kamerabildes auf die Web-Oberfläche sowie der Integration einer schwenkbaren Kamera, um eine flexible Sicht auf den Transportbereich zu gewährleisten. Der ESP32-Mikrocontroller sorgt dafür, dass die Befehle des Benutzers an den Roboter übermittelt werden.

Zusätzlich wird eine OnBoard-Software entwickelt, die es ermöglicht, die Sensoren auszulesen und sowohl die Motoren als auch die Kamera anzusteuern.

---

## 1.2 Abstract

This thesis is about the development of a load robot that can carry up to 25 kilograms. The robot is controlled via a website, which serves as the control platform. Additionally, a camera is integrated to display the transport area, as well as a scale to measure the weight of the carried load.

A key focus of the work is on the mechanical design of the robot, where a sturdy steel housing is built to ensure safety and stability. Furthermore, a custom circuit board is developed to control and connect the various hardware components, such as sensors and motors.

The robot is controlled via a website, which allows the user to operate the robot and access important data such as battery level and weight. A particular focus is also placed on transferring the camera feed to the web interface and integrating a swivel camera to ensure flexible viewing of the transport area. The ESP32-microcontroller ensures that the user's commands are transmitted to the robot.

Additionally, onboard software is developed to read the sensors and control the motors and camera.

---

# 2 Projektmanagement

## 2.1 Projektteam

**Betreuer: Prof. DI. Gernot Mörtl**



Abbildung 1:  
Porträt  
Daniel Schauer

### **Daniel Schauer: OnBoard-Software**

- Projektleiter
- Verbindung von Software und Hardware (ESP32 zu Sensoren)
- Kamera-Übertragung zur Web-Oberfläche
- Umsetzung einer schwenkbaren Kamera
- Dokumentation



Abbildung 2:  
Porträt  
Simon Spari

### **Simon Spari: Software-App**

- Benutzeroberfläche (Web-Oberfläche) für Steuerung
- Übertragung der Steuerung/Befehle von Web-Oberfläche zu ESP32
- Kamera Visualisierung auf der Website
- Dokumentation



Abbildung 3:  
Porträt Felix  
Hohegger

### **Felix Hohegger: Hardware-Design und Mechanik**

- Bau des Roboter Gehäuses
- Ansteuerung und Verbindung von Hardware (Ansteuerung und Berechnung der Motoren)
- Dokumentation

## 2.2 Projektstrukturplan

Unsere Diplomarbeit beschäftigt sich mit der Entwicklung eines Lastenroboters. Der Lastenroboter soll etwa ein Gewicht von 25 Kilogramm tragen können. Die Steuerung erfolgt über eine Website, und zusätzlich soll eine Kamera eingebaut werden. Im Lastenroboter wird auch eine Waage eingebaut, damit man sehen kann, wie schwer die transportierte Last ist.

Das Ziel dieser Diplomarbeit ist also, einen betriebsbereiten Prototyp eines Lastenroboters mit Kamerasystem und Waage zu entwickeln und zu realisieren.

### Geplantes Ergebnis der individuellen Themenstellungen:

**Felix Hohegger:** Bau des Roboter-Gehäuses und Integration der Komponenten, Verbindung der Hardware im besonderen Ansteuerung der Motoren

**Simon Spari:** Entwicklung einer Benutzeroberfläche (Web-Oberfläche) zur Steuerung des Roboters, Übertragung der Steuerbefehle in Echtzeit an die Hardware

**Daniel Schauer:** Verbindung von Software und Hardware zur Umsetzung der Steuerbefehle, Kamera-Übertragung in Echtzeit zur GUI, Umsetzung schwenkbare Kamera

### Projektstrukturplan:

Um unser Projekt besser zu strukturieren, haben wir am Anfang einen Grobplan für den Lastenroboter erstellt. Er hilft uns, die einzelnen Aufgaben und deren Verknüpfungen besser zu definieren. So können wir sicherstellen, dass Mechanik, Hardware und Software ohne Probleme zusammenarbeiten. Durch diese Planung behalten wir den Überblick über unser Projekt.

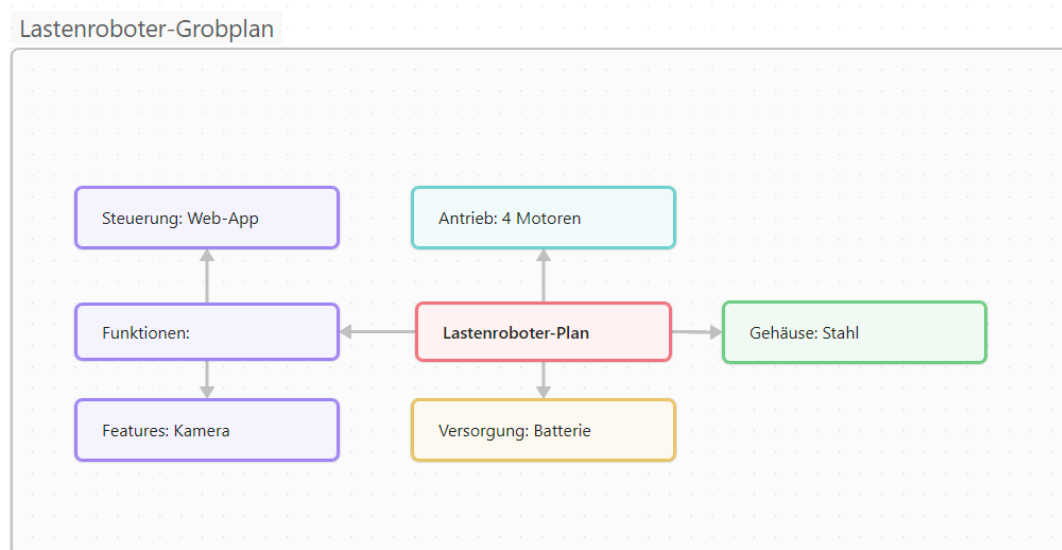


Abbildung 4: Lastenroboter Projekt-Grobplan

Quelle: eigene Abbildung erstellt mit Obsidian

Anschließend haben wir, basierend auf dem Grobplan, einen genaueren, detaillierten Plan erstellt. In diesem Feinkonzept wurden die einzelnen Arbeitsschritte konkreter ausgearbeitet, Zuständigkeiten verteilt und ein zeitlicher Ablauf festgelegt. So konnten wir gezielt vorgehen und mögliche Probleme frühzeitig erkennen und vermeiden.

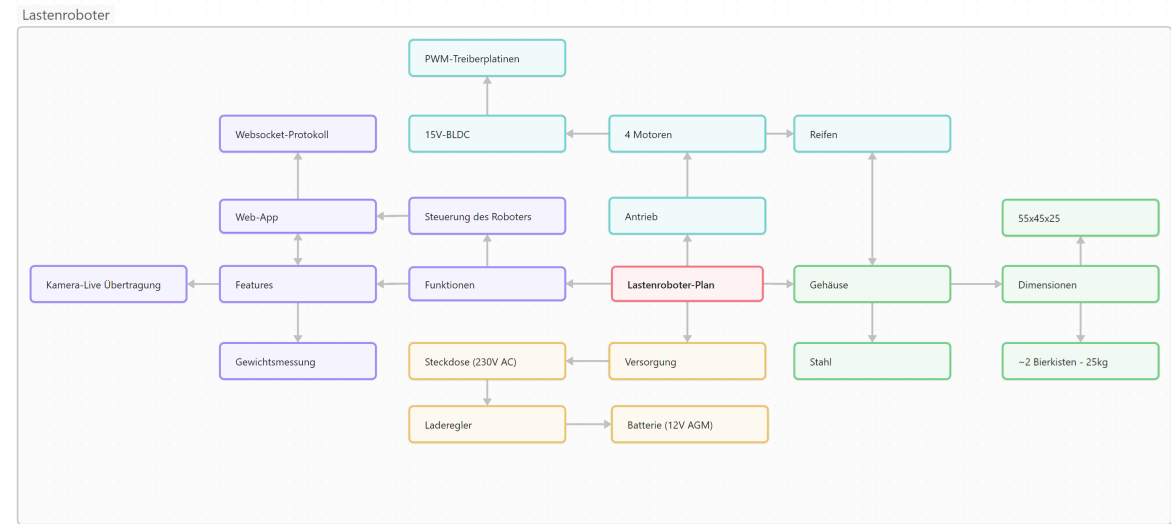


Abbildung 5: Lastenroboter Projekt-Plan  
Quelle: eigene Abbildung erstellt mit Obsidian

### 2.3 Meilensteine

Um unseren Fortschritt und unsere Zeiteinteilung besser im Überblick zu behalten, wurden bestimmte Meilensteine für das Projekt definiert. Diese sind sehr hilfreich, um das Projekt strukturiert umzusetzen, angefangen von der Projektplanung bis hin zum fertigen Prototyp.

Die folgenden Meilensteine wurden bei der Projektplanung definiert:

| Meilenstein                        | Datum      |
|------------------------------------|------------|
| Grundlegendes Gehäuse              | 07.11.2024 |
| Funktionsfähige Website            | 19.12.2024 |
| Funktionsfähige steuerbare Motoren | 16.01.2025 |
| Funktionsfähiger Prototyp          | 06.03.2025 |

---

## 2.4 Kostenaufstellung

In der Kostenaufstellung wurden alle Ausgaben erfasst, die im Laufe des Projekts angefallen sind. Dazu gehören unter anderem Bauteile und sonstiges Zubehör. So behalten wir den Überblick über unser Budget und können dieses besser einschätzen.

| Artikel                                      | Einzelpreis | Stückzahl | Gesamt          |
|----------------------------------------------|-------------|-----------|-----------------|
| Aufblasbares Rad 10" 260x85                  | 16.70 €     | 4         | 66.80 €         |
| MCP23017 GPIO Expander-Board                 | 4.54 €      | 2         | 9.08 €          |
| PICAA LED Arbeitsscheinwerfer                | 6.55 €      | 2         | 13.10 €         |
| 2 Stück PWM Motor Steuerung Treiber Platinen | 35.78 €     | 2         | 71.56 €         |
| Micro Servo Motor SG90                       | 6.04 €      | 1         | 6.04 €          |
| Dino KRAFTPAKET 4A/12V Batterieladegerät     | 17.99 €     | 1         | 17.99 €         |
| Platinen                                     | 37.14 €     | 1         | 37.14 €         |
| ESP32-CAM                                    | 13.70 €     | 1         | 13.70 €         |
| 12 Stück Halbbrücken Wägezelle               | 11.09 €     | 1         | 11.09 €         |
| ESP32                                        | 11.09 €     | 1         | 11.09 €         |
| Dunkermotoren                                | 65.00 €     | 2         | 130.00 €        |
| AGM 12V Batterie                             | 24.80 €     | 1         | 24.80 €         |
| Diverse Kleinteile                           | 30.00 €     | 1         | 30.00 €         |
| <b>Summe</b>                                 |             |           | <b>442.39 €</b> |



---

## 3 Antrieb

### 3.1 Motoren

#### 3.1.1 Übersicht

Für den Antrieb des Lastenroboters werden BLDC-Motoren eingesetzt. Verwendet werden die BLDC-Motoren von Dunkermotoren (Typ BG 40X25). Ein BLDC-Motor, auch bürstenloser Gleichstrommotor genannt, wird über drei Phasen betrieben. Der BLDC-Motor wird oft in der Industrie eingesetzt, da er eine hohe Leistung bietet.



Abbildung 6: Motoren

Quelle: eigene Abbildung

#### 3.1.2 Funktionsweise

Die Funktionsweise eines bürstenlosen Gleichstrommotors basiert auf der Wechselwirkung zwischen dem Magnetfeld des Stators und dem Magnetfeld des Rotors.

**Stator:** Der Stator besteht aus mehreren Spulen, die üblicherweise drei Phasen umfasst. Die drei Phasen werden mit Wechselstrom angesteuert die dann ein rotierendes Magnetfeld erzeugen.

**Rotor:** Der Rotor besteht aus einem Permanentmagneten. Dadurch, dass die Spulen ein rotierendes Magnetfeld erzeugen, dreht sich der Rotor synchron mit diesem Magnetfeld mit.

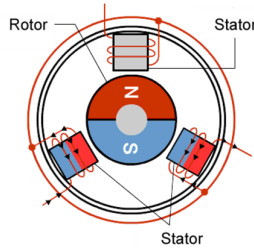


Abbildung 7: BLDC-Motor  
Quelle: [www.elektrikrehberiniz.com](http://www.elektrikrehberiniz.com)

Damit die Phasen des Motors korrekt geschaltet werden, muss die genaue Position des Rotors im Inneren des Motors erfasst werden. Die genaue Position des Rotors wird durch Hallsensoren erfasst. Die drei Hallsensoren geben diese Informationen an die Steuerelektronik weiter. Mit diesen Daten kann die optimale Beschaltung der Phasen berechnet werden.

Zur Steuerung der Drehzahl wird ein Pulsweitenmodulations-Signal (PWM-Signal) verwendet. Durch die PWM wird die Stromzufuhr zu den Phasen des Motors gesteuert. Wenn das Pulsweitenmodulations-Signal (PWM) erhöht wird, bedeutet das, dass die Phasen in kürzeren Abständen angesteuert werden. Dadurch erhält der Motor häufiger Energieimpulse, was zu einer höheren Drehzahl des Rotors führt. Das PWM-Signal regelt die Versorgungsspannung, indem es sie in schneller Folge ein- und ausschaltet. Je höher das Tastverhältnis des PWM-Signals, desto länger bleibt die Spannung während eines Zyklus eingeschaltet, wodurch der Motor mehr Energie erhält und sich schneller dreht.

Auf diese Weise kann der Motor effizient mit variabler Geschwindigkeit betrieben werden.

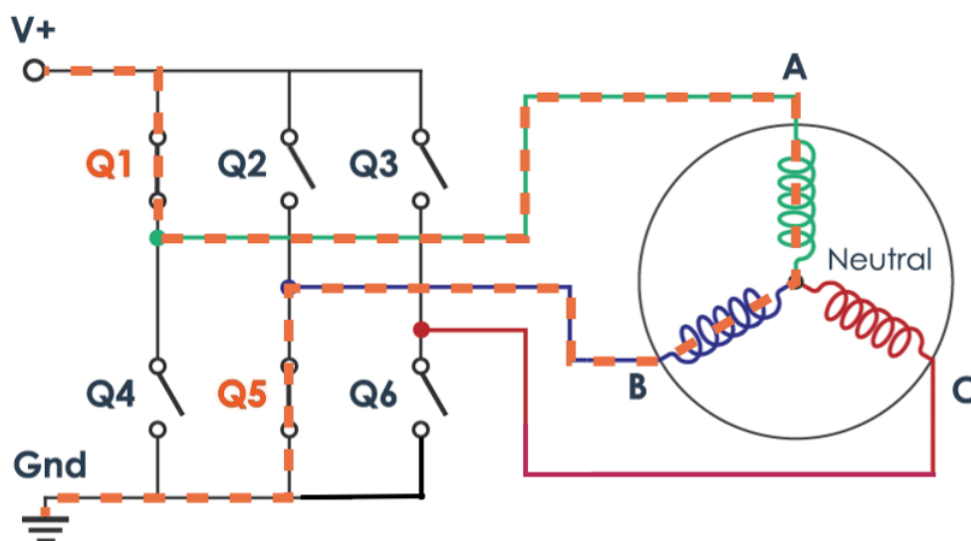


Abbildung 8: Phasensteuerung  
Quelle: [davincii.de/arduino-projekte/brushless-motor-ansteuerung](http://davincii.de/arduino-projekte/brushless-motor-ansteuerung)

### 3.1.3 Technische Daten

| Eigenschaft                                         | Wert     |
|-----------------------------------------------------|----------|
| Versorgungsspannung                                 | 15V      |
| Maximale Rotationsgeschwindigkeit der Antriebswelle | 3260 rpm |
| Max. Strom                                          | 2A       |

## 3.2 Motorentreiber

### 3.2.1 Überblick

Für die Ansteuerung der BLDC-Motoren wird der Motortreiber ZX-X11H verwendet. Dieser Motortreiber übernimmt die elektronische Kommutierung, das heißt dieser Treiber schaltet die Phasen des Motors damit sich der Rotor dreht. Die Rotorposition wird über Hallensoren erfasst. Mit diesen Daten werden die Phasen richtig angesteuert. Dies ermöglichten einen gleichmäßigen Betrieb sowie eine exakte Regelung von Drehzahl und Drehmoment. Durch die integrierte PWM-Steuerung kann die Geschwindigkeit präzise eingestellt werden. Zusätzlich verfügt der ZS-X11H über eine Richtungssteuerung und einer Brems-Funktion.

### 3.2.2 Aufbau und Funktionen

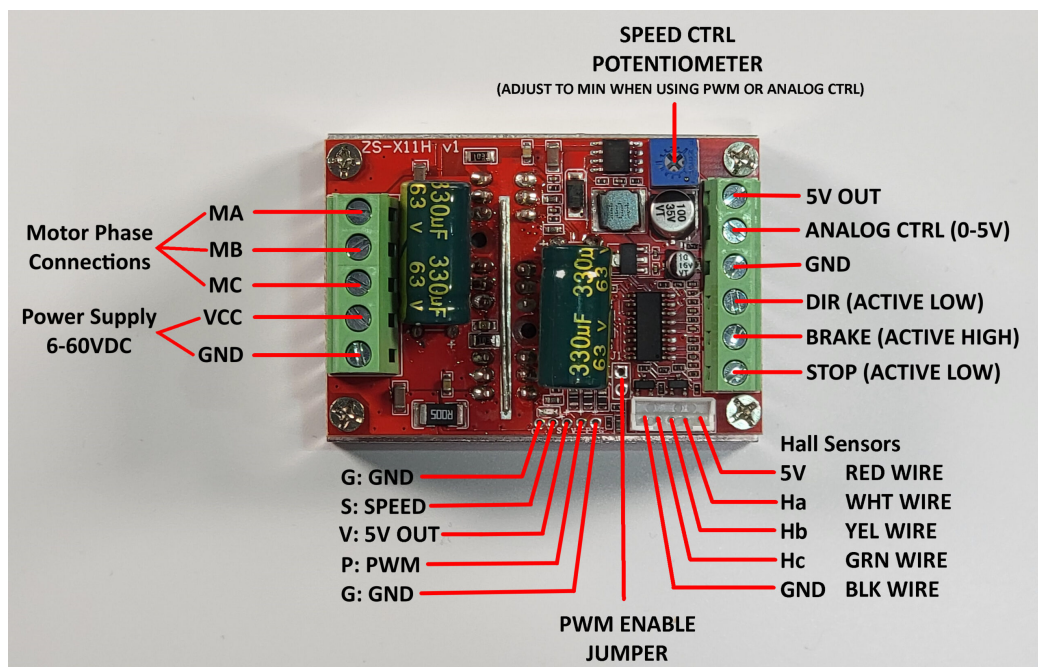


Abbildung 9: Motortreiber-ZS-X11H

Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

---

## Steuerung der Phasenströme:

Der Treiber steuert die drei Motorphasen (MA, MB, MC) mithilfe eines integrierten MOSFET-Brückenschaltkreises. Mit den Daten der Hallsensoren schaltet der Treiber die Phasen damit sich der Rotor mit der gewünschten Geschwindigkeit dreht.



Abbildung 10: Motorphasen Verbindung

Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

## PWM-Steuerung

- Die Pulsweitenmodulation (PWM) steuert die Geschwindigkeit des Motors, indem sie die Einschaltdauer der Spannung variiert, wodurch die durchschnittliche Leistung, die den Motorwicklungen zugeführt wird, angepasst wird.
- Die Amplitude des PWM-Signals muss zwischen 2,5-5 V liegen.
- Die PWM-Frequenz muss zwischen 50-20 kHz liegen.
- Die Platine ist ausgestattet mit einer externen und internen PWM-Steuerung.
- Wenn die Brücke auf der Platine verbunden wird, kommt die externe PWM zum Einsatz.

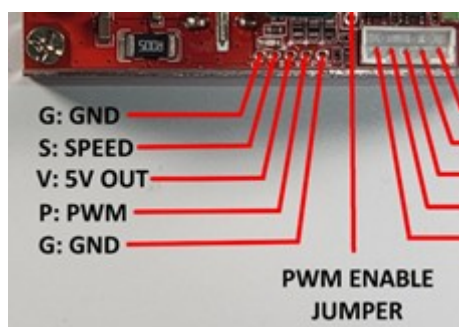


Abbildung 11: Pins für die PWM

Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

---

## Richtungswechsel und Bremse

- Die Platine verfügt über zwei Steuereingänge, einen für Richtungswechsel (DIR) und eine Bremsfunktion (BRAKE).
- Der Richtungswechsel erfolgt, indem das Signal bei dem Eingang von LOW auf HIGH oder umgekehrt wechselt. Das Umschalten der Richtung wechselt die Reihenfolge der Phasenströme, wodurch sich dann der Motor in die andere Richtung dreht.
- Die Bremse reagiert auf ein HIGH-Signal. Das heißt wenn am Eingang ein HIGH-Signal angelegt wird, stoppt der Motor, indem die Wicklungen kurzgeschlossen werden.

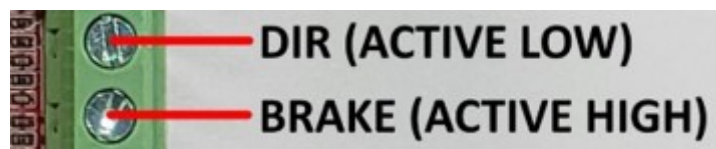


Abbildung 12: Pin für die Bremse und den Richtungswechsel  
Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

## Spannungs- und Stromversorgung

- Der Treiber kann mit einer Versorgungsspannung von 12V-60V betrieben werden, wodurch er für eine Vielzahl von BLDC-Motoren geeignet ist.
- Die maximale Stromaufnahme des Treibers liegt bei zirka 15A.
- Der Treiber ist für Motoren bis zu 500 Watt geeignet und kann damit leistungsstarke Motoren betreiben

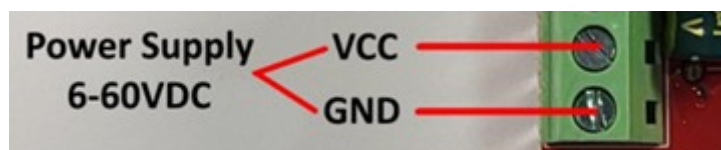


Abbildung 13: Versorgungs-Pins  
Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

---

## Hall-Sensoren

- Der Treiber verwendet drei Hall-Sensor-Eingänge. (Ha, Hb, Hc)
- Mit diesen Hall-Sensoren wird die Rotorposition ermittelt damit die Phasen des Motors richtig geschaltet werden.
- Die Sensoren ermöglichen eine effiziente und stabile Steuerung.



Abbildung 14: Hall-sensoren Eingänge

Quelle: [mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g](http://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/g)

## Sicherheit und Schutzfunktionen

- Der Treiber verfügt über einen Überstrom- und Überspannungsschutz damit der Motor und die Elektronik geschützt ist.
- Ein thermischer Schutz verhindert Schäden durch Überhitzung.

### 3.3 Schaltungsaufbau mit einem Motor

#### 3.3.1 Komponenten

- BLDC-Motor
- Motortreiber (ZS-X11H)
- Mikrocontroller (ESP32)
- Expander Modul (MCP23017)

#### 3.3.2 Schaltungsaufbau und Verbindungen

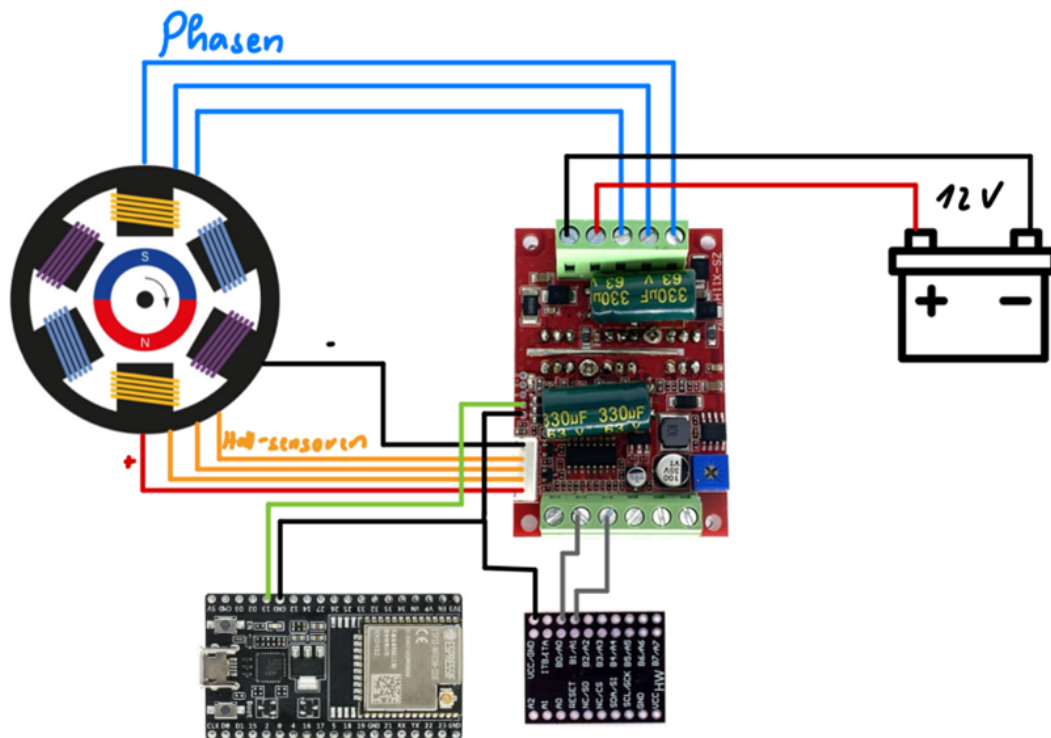


Abbildung 15: Schaltungsaufbau mit einem Motor

Quelle: eigene Abbildung

### Phasenanschlüsse des Motors (blau):

Die Phasen des Motors werden mit der Treiberplatine verbunden. Diese werden abwechselnd mit Spannung versorgt, um eine Drehbewegung zu erzeugen. Die weiße Phase des Motors wird mit MA verbunden. Die blaue Phase des Motors wird mit MB verbunden. Die orange Phase des Motors wird mit MC verbunden.

|   |   |        |    |
|---|---|--------|----|
| 5 | B | AW G22 | BU |
| 6 | C | AW G22 | OR |
| 7 | A | AW G22 | WH |

Abbildung 16: Hall-Sensoren Eingänge

Quelle:

easy-inexpensive-hoverboard-motor-controller

|   |    |        |    |
|---|----|--------|----|
| 1 | H1 | AW G26 | YE |
| 2 | H3 | AW G26 | BN |
| 3 | H2 | AW G26 | GN |

Abbildung 17: Hall-Sensoren-Motor

Quelle: eigene Abbildung

### Hall-Sensoren Verbindungen (orange)

Die Hall-Sensoren des Motors werden mit der Treiberplatine verbunden und erfassen die aktuelle Rotorposition. Diese Informationen werden an die Steuerelektronik übermittelt, die daraufhin die Motorphasen entsprechend schaltet, um die gewünschte Drehbewegung zu erzeugen. Dabei wird der gelbe Hall-Sensor mit HA, der grüne mit HB und der braune mit HC auf dem Treiber verbunden.



Abbildung 18: Motor Phasen

Quelle: eigene Abbildung

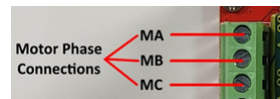


Abbildung 19: Treiber Phasen

Quelle: eigene Abbildung

### Versorgung (rot und schwarz)

- Der Treiber wird mit 12 Volt von der Batterie versorgt.
- Mit diesen 12 Volt steuert der Treiber den Motorstrom an den Phasen.

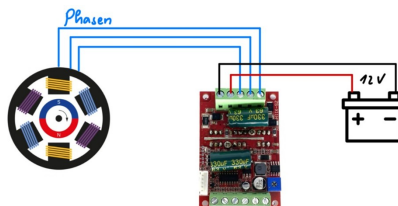


Abbildung 20: Versorgung der Motoren und Treiberplatine

Quelle: eigene Abbildung



---

### Steuerverbindung (grün und grau)

Der ESP32 ist mit dem Motortreiber verbunden, um Geschwindigkeit zu regeln. Das funktioniert über ein PWM-Signal.

Das Expander Modul steuert die Richtung und die Bremsfunktion. Der Richtungswechsel erfolgt, indem das Expander Modul den entsprechenden Eingang auf LOW oder HIGH schaltet. Die Bremse reagiert auf eine HIGH Signal. Das heißt, wenn das Expander Modul den Eingang auf der Steuerplatine HIGH schaltet, wird der Motor gebremst.

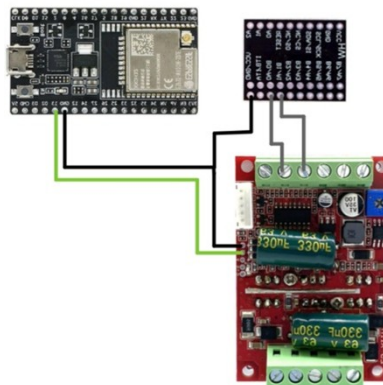


Abbildung 21: Ansteuerung der Treiberplatine  
Quelle: eigene Abbildung

## 3.4 Schaltungsaufbau mit vier Motoren

### 3.4.1 Komponenten

- 4 BLDC-Motor
- 4 Motortreiber (ZS-X11H)
- Mikrocontroller (ESP32)
- Expander Modul (MCP23017)

---

### **3.4.2 Schaltungserweiterung für vier Motoren:**

Der Lastenroboter wird von vier BLDC-Motoren angetrieben, die jeweils über einen eigenen Motortreiber gesteuert werden. Jeder Motortreiber wird über einen PWM-Pin des Esp32 verbunden, um die Geschwindigkeit zu regulieren. Zusätzlich wird jeder Motortreiber über das Expander-Modul angesteuert, um die Richtung zu ändern und die Bremse zu aktivieren. Alle Treiber werden mit 12 Volt von der Batterie versorgt.

---

## 3.5 Code

### 3.5.1 Initialisierung

Um die Motoren beziehungsweise die Treiberplatinen anzusteuern, gibt es pro Motor drei Pins. Einer dieser Pins wird mit einem PWM-fähigen Pin am ESP32 verbunden. Die anderen beiden werden mit dem Expander Modul verbunden. Da das Expander Modul nur Digitale Ein-/Ausgänge besitzt, ist es nicht möglich die PWM-Signale auch über dieses Modul zu realisieren. Der Hauptgrund für diese Aufteilung auf den Expander ist, dass wir Analoge Pins am ESP32 einsparen müssen, um diese für andere Sensoren oder Aktoren einzusetzen.

```
int leftfrontwheel_pwm = 32;
const int leftfrontwheel_brake = 1;
const int leftfrontwheel = 2;

int leftrearwheel_pwm = 18;
const int leftrearwheel_brake = 3;
const int leftrearwheel = 4;

int rightfrontwheel_pwm = 27;
const int rightfrontwheel_brake = 5;
int rightfrontwheel = 6;

int rightrearwheel_pwm = 25;
const int rightrearwheel_brake = 7;
const int rightrearwheel = 8;
```

Abbildung 22: Motoren-GPIO Konfiguration  
Quelle: eigene Abbildung

Um das Expander-Board zu nutzen, wird die Bibliothek Adafruit\_MCP23X17 verwendet. Diese Bibliothek ermöglicht eine einfachere Initialisierung und Ansteuerung der MCP-Pins, sodass die GPIOs ähnlich wie die des ESP32 genutzt werden können. Außerdem wird ein Objekt der Klasse Adafruit MCP23X17 erstellt, über das man später auf den Expander zugreifen kann.

```
#include <Adafruit_MCP23X17.h>
Adafruit_MCP23X17 mcp;
```

Abbildung 23: Adafruit\_MCP23X17.h-Bibliothek  
Quelle: eigene Abbildung

---

### 3.5.2 Setup-Code

Am Anfang des Setup-Codes wird überprüft, ob ein Expander Modul über die I2C Schnittstelle verbunden ist und dieses auch funktionstüchtig ist. Falls das nicht der Fall ist, wird ein Error über die Serielle Schnittstelle ausgegeben und das Setup wird unterbrochen.

```
if (!mcp.begin_I2C())
{
  Serial.println("MCP Error.");
  while (1);
}
```

Abbildung 24: MCP-Setup

Quelle: eigene Abbildung

Danach werden die Pins des ESP32 sowie die des Expander-Moduls als Ausgänge initialisiert. Die Pins des Expander-Ports müssen mit dem Befehl mcp. vor pinMode initialisiert werden. Zudem wird die Motorbremse direkt über den MCP mit digitalWrite aktiviert, sodass sich beim Starten kein Motor ungewollt dreht.

```
mcp.pinMode(leftfrontwheel, OUTPUT);
pinMode(leftfrontwheel_pwm, OUTPUT);
mcp.pinMode(leftfrontwheel_brake, OUTPUT);
mcp.digitalWrite(leftfrontwheel_brake, HIGH);

mcp.pinMode(rightfrontwheel, OUTPUT);
pinMode(rightfrontwheel_pwm, OUTPUT);
mcp.pinMode(rightfrontwheel_brake, OUTPUT);
mcp.digitalWrite(rightfrontwheel_brake, HIGH);

mcp.pinMode(leftrearwheel, OUTPUT);
pinMode(leftrearwheel_pwm, OUTPUT);
mcp.pinMode(leftrearwheel_brake, OUTPUT);
mcp.digitalWrite(leftrearwheel_brake, HIGH);

mcp.pinMode(rightrearwheel, OUTPUT);
pinMode(rightrearwheel_pwm, OUTPUT);
mcp.pinMode(rightrearwheel_brake, OUTPUT);
mcp.digitalWrite(rightrearwheel_brake, HIGH);
```

Abbildung 25: Initialisierung Motoren

Quelle: eigene Abbildung

### 3.5.3 Handling der Steuerbefehle

Um den Roboter zu steuern, werden die verschiedenen Steuerbefehle von der Webseite durch spezifische Funktionen bearbeitet. Die Richtungsbefehle zur Steuerung des Roboters werden mit einer Art State-Maschine beziehungsweise mit einer switch-case Struktur realisiert. Dafür wird zunächst ein enum (Enumeration) erstellt, dass die möglichen Bewegungsrichtungen des Roboters definiert und die Direction wird sofort auf „HALT“ gesetzt. Außerdem wird die Variable speed\_cb für die Geschwindigkeit initialisiert.

```
enum Direction
{
    HALT,
    UP,
    DOWN,
    LEFT,
    RIGHT,
    CAM_RIGHT,
    CAM_LEFT
};
int speed_cb = 0;
Direction dir = HALT;
```

Abbildung 26: Enumeration von Bewegungsrichtungen  
Quelle: eigene Abbildung

#### Befehle zum Steuern:

- HALT: Der Roboter bremst.
- UP: Bewegt den Roboter vorwärts.
- DOWN: Bewegt den Roboter rückwärts.
- LEFT: Lässt den Roboter nach links drehen.
- RIGHT: Lässt den Roboter nach rechts drehen.

---

Um die Befehle von der Webseite für die Steuerung zu benutzen, werden zunächst die erhaltenen Strings, mit einer Funktion in die vorhin definierte Richtungsvariable umgewandelt.

```
Direction stringToDirection(const String &str)
{
    if (str == "up")
    {
        return UP;
    }
    else if (str == "down")
    {
        return DOWN;
    }
    else if (str == "left")
    {
        return LEFT;
    }
    else if (str == "right")
    {
        return RIGHT;
    }
    else
    {
        return HALT;
    }
}
```

Abbildung 27: Strings zu Enum Variable umwandeln  
Quelle: eigene Abbildung

#### 3.5.4 Ansteuerung der Treiberplatinen:

Die Geschwindigkeit und die Richtung des Roboters werden schlussendlich in der Funktion „void navigate()“ bearbeitet. Diese Funktion wird in der „void loop()“ die ganze Zeit ausgeführt.

```
void loop()
{
    websocket.loop();
    navigate();
}
```

Abbildung 28: navigate() in loop()  
Quelle: eigene Abbildung

---

Zu Beginn werden zwei statische Variablen definiert, die dazu dienen, die zuletzt empfangene Fahrtrichtung (lastDir) und Geschwindigkeit (lastSpeed) zu speichern. Anschließend wird überprüft, ob sich entweder die Richtung oder die Geschwindigkeit im Vergleich zum vorherigen Wert geändert hat. Falls sich die Geschwindigkeit im Vergleich zum vorherigen Wert geändert hat, wird der empfangene String von der Webseite in einen Integer umgewandelt. Der Wert von „speed“ liegt im Bereich von 0–100 % und wird in einen Wert von 0–255 skaliert, der später für die PWM-Signale verwendet wird.

```
void navigate()
{
    static Direction lastDir = HALT;
    static int lastSpeed = -1;

    if (dir != lastDir || speed_cb != lastSpeed)
    {
        speed_cb = speed.toInt() * 2.55;
        lastSpeed = speed_cb;
    }
}
```

Abbildung 29: Überprüfung auf Änderung und Speed  
in Integer umwandeln in navigate()-Funktion  
Quelle: eigene Abbildung

Außerdem wird die Motorbremse deaktiviert, sodass sich der Roboter überhaupt bewegt und sich steuern lässt.

```
mcp.digitalWrite(leftfrontwheel_brake, LOW);
mcp.digitalWrite(rightfrontwheel_brake, LOW);
mcp.digitalWrite(leftrearwheel_brake, LOW);
mcp.digitalWrite(rightrearwheel_brake, LOW);
```

Abbildung 30: Motorbremse deaktivieren in navigate()-Funktion  
Quelle: eigene Abbildung

Die Steuerung der Richtung des Roboters erfolgt durch eine switch-case-Struktur, die anhand der aktuellen Fahrtrichtung (dir) die entsprechende Bewegungsfunktion aufruft. Diese Steuerlogik basiert auf dem zuvor definierten enum Direction, der die verschiedenen Zustände wie UP, DOWN, LEFT, RIGHT und HALT enthält. Diese Struktur funktioniert so ähnlich wie eine Gangschaltung, sie legt nämlich nur fest in welche Richtung sich der Motor drehen soll.

```

switch (dir)
{
case UP:
    moveForward();
    break;

case DOWN:
    moveBackward();
    break;

case LEFT:
    turnLeft();
    break;

case RIGHT:
    turnRight();
    break;

case HALT:
    stop();
    break;
default:
    stop();
    break;
}

void moveForward()
{
    mcp.digitalWrite(leftfrontwheel, HIGH);
    mcp.digitalWrite(rightfrontwheel, LOW);
    mcp.digitalWrite(leftrearwheel, HIGH);
    mcp.digitalWrite(rightrearwheel, LOW);
    Serial.println("Vorwärts");
}

void moveBackward()
{
    mcp.digitalWrite(leftfrontwheel, LOW);
    mcp.digitalWrite(rightfrontwheel, HIGH);
    mcp.digitalWrite(leftrearwheel, LOW);
    mcp.digitalWrite(rightrearwheel, HIGH);
    Serial.println("Rückwärts");
}

void turnLeft()
{
    mcp.digitalWrite(leftfrontwheel, LOW);
    mcp.digitalWrite(rightfrontwheel, LOW);
    mcp.digitalWrite(leftrearwheel, LOW);
    mcp.digitalWrite(rightrearwheel, LOW);
    Serial.println("Links");
}

void turnRight()
{
    mcp.digitalWrite(leftfrontwheel, HIGH);
    mcp.digitalWrite(rightfrontwheel, HIGH);
    mcp.digitalWrite(leftrearwheel, HIGH);
    mcp.digitalWrite(rightrearwheel, HIGH);
    Serial.println("Rechts");
}

void stop()
{
    mcp.digitalWrite(leftfrontwheel_brake, HIGH);
    mcp.digitalWrite(rightfrontwheel_brake, HIGH);
    mcp.digitalWrite(leftrearwheel_brake, HIGH);
    mcp.digitalWrite(rightrearwheel_brake, HIGH);
    Serial.println("Stop");
}

```

Abbildung 31: switch-case Struktur in navigate()-Funktion

Quelle: eigene Abbildung

Die Steuerung der Geschwindigkeit, wird mithilfe von PWM-Signalen umgesetzt. Zunächst wird eine statische Variable „lastPWM“ definiert, die später den zuletzt verwendeten PWM-Wert speichert. Bevor die Treiber-Platinen mit einem PWM-Signal angesteuert werden, wird überprüft, ob sich die Geschwindigkeit tatsächlich geändert hat. Falls sich „speed\_cb“ vom vorherigen Wert abweicht, werden die neuen PWM-Werte mit analogWrite() an die Motorsteuerungsplatinen gesendet, um die Geschwindigkeit der Motoren entsprechend anzupassen.



---

Danach wird der Wert von „lastPWM“ aktualisiert, um sicherzustellen, dass die gleiche Geschwindigkeit nicht unnötig erneut gesetzt wird. Zusätzlich wird die Variable „lastDir“ aktualisiert, um die letzte Richtung zu speichern.

```
static int lastPWM = -1;
if (lastPWM != speed_cb)
{
    analogWrite(leftfrontwheel_pwm, speed_cb);
    analogWrite(rightfrontwheel_pwm, speed_cb);
    analogWrite(leftrearwheel_pwm, speed_cb);
    analogWrite(rightrearwheel_pwm, speed_cb);
    lastPWM = speed_cb;
}

lastDir = dir;
}
```

Abbildung 32: PWM mit analogWrite übertragen und  
letzte Werte speichern in navigate()-Funktion  
Quelle: eigene Abbildung

---

# 4 Webserver

## 4.1 Grundlegende Ziele

In diesem Kapitel wird die geplante Website thematisiert, die die Steuerung des Lastenroboters ermöglichen soll. Zunächst werden die grundlegenden Funktionen definiert, die die Website erfüllen muss. Sobald diese Anforderungen umgesetzt sind, liegt der Fokus auf einer möglichst einfachen und benutzerfreundlichen Gestaltung des User Interfaces. Ein weiterer zentraler Aspekt ist die Darstellung spezifischer Messwerte und Daten, sodass diese über den Webserver schnell und unkompliziert zugänglich sind.

Für die Entwicklung der Website kommen HTML, CSS und JavaScript zum Einsatz, um sowohl die funktionalen als auch die optischen Anforderungen zu erfüllen.

### **Videübertragung**

Die Website ermöglicht die Echtzeit-Videübertragung des Kamerabilds der ESP32-CAM. Die Bildfrequenz und die Qualität der Videübertragung sollte ausgeglichen sein, so dass in der Übertragung alle Objekte und ggf. Hindernisse frühzeitig erkennbar sind und noch Reaktionszeit zum Manövrieren besteht.

### **Steuerung**

Auf der Website soll eine grafische Steuereinheit implementiert werden, mit der der Roboter gesteuert und navigiert werden kann. Diese soll dann die entsprechenden Steuerbefehle bzw. Richtungen an den ESP32 senden, wo sie dann in Steuerungsbefehle für die Motoren übersetzt werden.

### **Anzeigen von Daten**

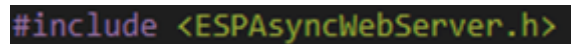
Auf der Website sollen bestimmte Messwerte und Daten, wie zum Beispiel Akkustand oder zurzeit aufliegende Last, die vom ESP32 durch Sensoren oder Messungen ausgewertet werden, übersichtlich und leicht zugänglich angezeigt werden.

---

## 4.2 Webserver

### 4.2.1 Webserver Setup

Der Webserver wird auf einem ESP32-CAM Mikrocontroller mithilfe der Bibliothek ESPAsyncWebServer eingerichtet. Diese Bibliothek ermöglicht einen nicht-blockierenden Betrieb, wodurch parallele Anfragen effizient verarbeitet werden können.<sup>1</sup>



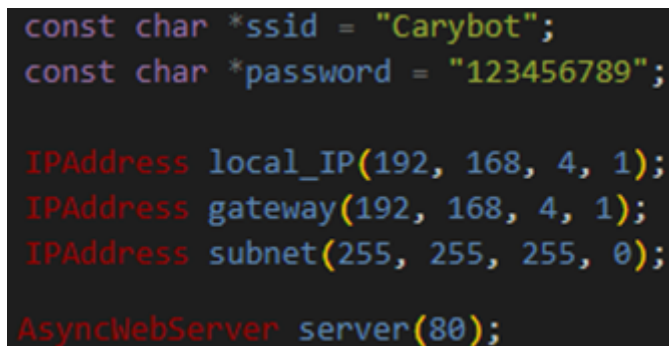
```
#include <ESPAsyncWebServer.h>
```

Abbildung 33: ESPAsyncWebServer.h-Bibliothek  
Quelle: eigene Abbildung

### Netzwerkkonfiguration

Die ESP32-CAM fungiert als Access Point mit folgenden Konfigurationen:

- SSID (Service Set Identifiert) : “Carybot” – dient zur Identifikation des drahtlosen Netzwerkes
- Passwort: “123456789” – dient zum Schutz des Netzwerkes
- Lokale IP-Adresse: 192.168.4.1 – dient zum Zugriff auf die Webserver-Oberfläche
- Gateway-Adresse: 192.168.4.1 - ESP32-CAM fungiert als Access Point
- Subnetzmaske: 255.255.255.0 - ermöglicht die Kommunikation zwischen Geräten im Bereich 192.168.4.x
- Port: 80 - Standardport für HTTP-Dienste



```
const char *ssid = "Carybot";  
const char *password = "123456789";  
  
IPAddress local_IP(192, 168, 4, 1);  
IPAddress gateway(192, 168, 4, 1);  
IPAddress subnet(255, 255, 255, 0);  
  
AsyncWebServer server(80);
```

Abbildung 34: WebServer Netzwerkkonfiguration  
Quelle: eigene Abbildung

---

<sup>1</sup><https://github.com/lacamera/ESPAsyncWebServer>

---

In der Setup Funktion wird danach überprüft, ob der Access Point erfolgreich konfiguriert worden ist und ob er erfolgreich gestartet werden kann. Falls ein Fehler auftreten sollte, wird der Setup unterbrochen und die jeweilige Fehlermeldung in der seriellen Konsole ausgegeben. Wenn alles erfolgreich konfiguriert ist und starten kann, wird im Seriellen Monitor die IP-Adresse in der seriellen Konsole ausgegeben. Anschließend wird der Webserver gestartet.

```
if (!WiFi.softAPConfig(local_IP, gateway, subnet)) {  
    Serial.println("AP-Konfiguration fehlgeschlagen.");  
    return;  
}  
  
if (!WiFi.softAP(ssid, password)) {  
    Serial.println("AP konnte nicht gestartet werden.");  
    return;  
}  
  
Serial.println("Access Point gestartet!");  
Serial.print("IP-Adresse: ");  
Serial.println(WiFi.softAPIP());  
server.begin();
```

Abbildung 35: Webserver Setup  
Quelle: eigene Abbildung

#### 4.2.2 SPIFFS Setup

SPIFFS (SPI Flash File System) ist ein leichtgewichtiges Dateisystem für Mikrocontroller mit SPI-Flash-Speicher. Es ermöglicht das Speichern und Verwalten von Dateien direkt im Flash-Speicher des Mikrocontrollers. SPIFFS wird in unserem Projekt benötigt, um statische Dateien für unseren Webserver (HTML-, CSS-, JavaScript Anwendungen) bereitzustellen.

In unserem Code wird zuallererst einmal überprüft, ob SPIFFS beim Start der ESP32-CAM richtig initialisiert werden kann. Falls es fehlschlägt, wird eine Fehlermeldung in der seriellen Konsole ausgegeben und das Programm gestoppt. Ansonsten werden die Webserver-Endpunkte über HTTP-GET-Routen definiert, über die unsere statischen Dateien aus SPIFFS an Clients gesendet werden.

---

“/“ ist die Default-Route des Servers. Somit wird dpad.html als Startseite angezeigt, wenn ein Client sich verbindet.

“menu-icon.svg“ und “Fernlicht.svg“ werden als SVG-Bilder (Scalable Vector Graphics) an den Browser gesendet. Image/svg+xml sorgt dafür, dass der Browser die Dateien als SVG-Bilder erkennt.

“mystyles.css“ wird mit text/css als CSS-Datei gesendet und dient zur Formatierung der Website.

“carybot.js“ wird mit application/javascript als Javascript-Datei gesendet und verarbeitet die Eingaben von Clients auf der Website.

Wenn alle Dateien erfolgreich geladen sind, wird eine Nachricht in der seriellen Konsole ausgegeben.

```
if (!SPIFFS.begin(true)) {  
    Serial.println("Fehler beim Mounten von SPIFFS");  
    return;  
}  
  
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {  
    String dpad = readFile(SPIFFS, "/dpad.html");  
    request->send(200, "text/html", dpad);  
});  
  
server.on("/menu-icon.svg", HTTP_GET, [](AsyncWebServerRequest *request) {  
    String icon = readFile(SPIFFS, "/menu-icon.svg");  
    request->send(200, "image/svg+xml", icon);  
});  
  
server.on("/Fernlicht.svg", HTTP_GET, [](AsyncWebServerRequest *request) {  
    String fernlicht = readFile(SPIFFS, "/Fernlicht.svg");  
    request->send(200, "image/svg+xml", fernlicht);  
});  
  
server.on("/mystyles.css", HTTP_GET, [](AsyncWebServerRequest *request) {  
    String css = readFile(SPIFFS, "/mystyles.css");  
    request->send(200, "text/css", css);  
});  
  
server.on("/carybot.js", HTTP_GET, [](AsyncWebServerRequest *request) {  
    String js = readFile(SPIFFS, "/carybot.js");  
    request->send(200, "application/javascript", js);  
});  
  
Serial.println("SPIFFS-Dateien erfolgreich geladen");
```

Abbildung 36: SPIFFS Initialisierung

Quelle: eigene Abbildung

---

Die `readFile()` Funktion wird benötigt, um die jeweiligen Dateien aus SPIFFS lesen zu können. Die Funktion liest eine Datei aus dem SPIFFS-Speicher und gibt den Inhalt als String zurück.

```
String readFile(fs::FS &fs, const char *path) {  
    File file = fs.open(path, "r");  
    if (!file || file.isDirectory()) {  
        Serial.println("- failed to open file for reading");  
        return String();  
    }  
  
    String fileContent;  
    while (file.available()) {  
        fileContent += String((char)file.read());  
    }  
    return fileContent;  
}
```

Abbildung 37: `readFile()`-Funktion

Quelle: eigene Abbildung

## 4.3 WebSocket Kommunikation

### 4.3.1 Kommunikation Setup

Für die Kommunikation zwischen dem Webserver und dem ESP32 wird die `ArduinoJson` und die `ArduinoWebSockets` Bibliothek benötigt. Die `ArduinoJson` Bibliothek wird für die Umwandlung der JSON-Steuerbefehle benötigt. Die `ArduinoWebSockets` Bibliothek wird für die Kommunikation über das WebSocket Protokoll benötigt.

```
#include "ArduinoJson.h"  
#include "WebSocketsServer.h"
```

Abbildung 38: Bibliotheken für die Kommunikation

Quelle: eigene Abbildung

Für die Netzwerkkonfiguration als Client werden die gleichen Parameter wie in der Access Point Konfiguration gewählt (siehe 34), mit Ausnahmen der IP-Adresse und dem WebSocket Port. Als Lokale IP-Adresse wurde 192.168.4.3 und als Port 8080 gewählt.

---

In der Setup Funktion des Programmes wird überprüft, ob die IP-Konfiguration erfolgreich abgeschlossen wurde. Ansonsten kommt es zu einer Fehlermeldung und das Setup wird abgebrochen. Danach wird versucht, sich mit dem WLAN-Netzwerk zu verbinden. Wenn sich der ESP32 erfolgreich mit dem WLAN verbunden hat, wird eine Nachricht und die IP-Adresse des ESP32 in der seriellen Konsole ausgegeben. Danach wird die WebSocket Konfiguration gestartet. Es wird definiert, dass die Funktion onWebSocketEvent aufgerufen wird, wenn Events über den WebSocket registriert werden. In der loop Funktion wird ständig überprüft, ob neue Events am WebSocket registriert werden.

```
if (!WiFi.config(local_IP, gateway, subnet))
{
    Serial.println("Fehler bei der IP-Konfiguration");
    return;
}

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.println(".");
}
Serial.println("WLAN verbunden");
Serial.print("IP-Adresse: ");
Serial.println(WiFi.localIP());

websocket.begin();
websocket.onEvent(onWebSocketEvent);

void loop()
{
    websocket.loop();
}
```

Abbildung 39: Setup ESP32

Quelle: eigene Abbildung

### 4.3.2 Message Handling

In der Funktion onWebSocketEvent() werden die WebSocket-Ereignisse verarbeitet. Sie wird aufgerufen, wenn sich ein WebSocket-Client verbindet, eine Nachricht sendet oder die Verbindung getrennt wird. Der Parameter num steht für die ID des Clients, der das Event ausgelöst hat. Der Parameter type gibt die Art des WebSocket-Event an. Der Parameter payload sind die empfangenen Daten. Der Parameter length steht für die Größe des payload-Buffers. In der Funktion werden die Events mit einem switch-case verarbeitet

---

Übersicht der WebSocket-Ereignisse:

| Ereignistyp         | Beschreibung                       | Verarbeitung                                      |
|---------------------|------------------------------------|---------------------------------------------------|
| WStype_CONNECTED    | Ein neuer Client verbindet sich.   | Ausgabe der Client-ID & IP-Adresse in der Konsole |
| WStype_TEXT         | Eine Textnachricht wird empfangen. | Übergabe an <code>handleWebSocketMessage()</code> |
| WStype_BIN          | Binärdaten werden empfangen.       | Nachricht in der Konsole (wird nicht verarbeitet) |
| WStype_DISCONNECTED | Ein Client trennt die Verbindung.  | Meldung mit der Client-ID in der Konsole.         |

Tabelle 3: Übersicht der WebSocket-Ereignisse

```
void onWebSocketEvent(uint8_t num, WStype_t type, uint8_t *payload, size_t length)
{
    switch (type)
    {
        case WStype_TEXT:
            handleWebSocketMessage(num, payload, length);
            break;

        case WStype_BIN:
            Serial.println("Binärdaten empfangen (nicht unterstützt)");
            break;

        case WStype_DISCONNECTED:
            Serial.printf("[WS] Client %u disconnected.\n", num);
            break;

        case WStype_CONNECTED:
            IPAddress ip = websocket.remoteIP(num);
            Serial.printf("[WS] Client %u connected from %s.\n", num, ip.toString().c_str());
            break;
    }
}
```

Abbildung 40: onWebSocketEvent()-Funktion

Quelle: eigene Abbildung

Die Funktion `handleWebSocketMessage()` verarbeitet eingehende WebSocket-Nachrichten, die als JSON-Objekte gesendet wird. Sie nimmt drei Parameter entgegen:

1. `num` - Client-ID der Websocket Verbindung
2. `payload` - die empfangene Nachricht als Byte-Array
3. `length` - die Länge der empfangenen Nachricht



---

Zuerst wird das payload-Byte-Array in einen String konvertiert und anschließend zu Debugging-Zwecken auf der seriellen Konsole ausgegeben. Danach wird ein `StaticJsonDocument` mit einer maximalen Größe von 200 Bytes erstellt, um die empfangenen JSON-Daten zu speichern. Anschließend wird die Nachricht mit `deserializeJson()` geparkt. Falls das Parsen fehlschlägt, wird die Verarbeitung abgebrochen. Je nachdem, welche Schlüssel in der JSON-Nachricht enthalten sind, werden unterschiedliche Funktionen ausgeführt.

Falls die JSON-Nachricht den Schlüssel `robot_direction` enthält, wird der zugehörige Wert ausgelesen und mithilfe der Funktion `stringToDirection()` in eine interne Richtungsvariable (`dir`) umgewandelt. Zusätzlich wird der Wert für `speed` aus der Nachricht extrahiert und als String gespeichert.

```
{
  "robot_direction": "up",
  "speed": 100
}
```

Abbildung 41: JSON-Beispiel für Steuerung

Quelle: eigene Abbildung

Falls die JSON-Nachricht den Schlüssel `camera_position` enthält, wird der Wert als Ganzzahl in die Variable `camera_pos` gespeichert. Anschließend wird die Funktion `cam_turn()` aufgerufen, um die Kamera entsprechend zu bewegen.

```
{
  "camera_position": 45
}
```

Abbildung 42: JSON-Beispiel für Kamerasteuerung

Quelle: eigene Abbildung

Falls die JSON-Nachricht den Schlüssel `light_status` enthält, wird der Wert der Nachricht in die boolesche Variable `light_status` gespeichert. Dieser Variable wird dann in die numerische Variable `light_st` umgewandelt (1 = an, 0 = aus). Wenn `light_st` True ist, wird die Funktion `lights_on()` aufgerufen, ansonsten wird die Funktion `lights_off()` aufgerufen.

```
{
  "light_status": true
}
```

Abbildung 43: JSON-Beispiel für Lichtsteuerung

Quelle: eigene Abbildung

```

void handleWebSocketMessage(uint8_t num, uint8_t *payload, size_t length)
{
    String message = String((char *)payload).substring(0, length);
    Serial.println("WebSocket-Nachricht empfangen: " + message);

    StaticJsonDocument<200> jsonDoc;
    DeserializationError error = deserializeJson(jsonDoc, message);

    if (!error)
    {
        if (jsonDoc.containsKey("robot_direction"))
        {
            const char *robot_direction = jsonDoc["robot_direction"];
            if (robot_direction)
            {
                dir = stringToDirection(robot_direction);
            }
            speed = jsonDoc["speed"].as<String>();
        }
        else if (jsonDoc.containsKey("camera_position"))
        {
            const char *camera_position = jsonDoc["camera_position"];
            if (camera_position)
            {
                camera_pos = atoi(camera_position);
                cam_turn();
            }
        }
        else if (jsonDoc.containsKey("light_status"))
        {
            bool light_status = jsonDoc["light_status"];
            light_st = light_status ? 1 : 0;

            if (light_st == 1)
            {
                lights_on();
            }
            else
            {
                lights_off();
            }
        }
    }
}

```

Abbildung 44: handleWebSocketMessage()-Funktion

Quelle: eigene Abbildung

---

## 4.4 Website

### 4.4.1 Implementierung der Steuerung

#### Steuerkreuz

Um den Roboter überhaupt steuern zu können, wird ein Steuerkreuz implementiert. Das Steuerkreuz befindet sich mittig am rechten Bildschirmrand. Das Steuerkreuz besteht aus fünf Tasten, nämlich: Vorwärts (UP), (DOWN ), links (LEFT), rechts (RIGHT) und in der Mitte Stop (HALT).



Abbildung 45: Steuerkreuz auf der Website

Quelle: eigene Abbildung

Im HTML-Code wird das Steuerkreuz als HTML `<div>`-Tag im body Bereich definiert. Für die Formatierung wird die CSS-Klasse `dpad-container` verwendet. Die einzelnen Richtungstasten des Steuerkreuze werden auch als HTML `<div>`-Tags definiert. Jede Taste wird mit der CSS-Klasse `button` und der jeweiligen Zusatzklasse `up/down/left/right/center` formatiert. Beim jeweiligen Tastendruck wird außerdem immer das Attribut `data-direction` mit der jeweiligen Richtung belegt.

```
<div class="dpad-container">
  <div class="button up" data-direction="up">↑</div>
  <div class="button left" data-direction="left">←</div>
  <div class="button center" data-direction="halt"></div>
  <div class="button right" data-direction="right">→</div>
  <div class="button down" data-direction="down">↓</div>
</div>
```

Abbildung 46: Steuerkreuz HTML-Code

Quelle: eigene Abbildung

---

Die CSS-Klasse `dpad-container` ist das übergeordnete Element, welches die Steuerkreuztasten enthalten. Es wird als CSS Grid mit einer 3x3 Struktur definiert. Es gibt Lücken (.) an den Ecken, damit man die Form eines Steuerkreuzes erhält. Der Abstand zwischen den Tasten wird mit 10px festgelegt. Der Container hat eine feste Breite von 120px und ist vertikal so wie horizontal zentriert.

Die CSS-Klasse `button` sorgt für eine einheitliche Gestaltung der Tasten des Steuerkreuzes. Jede Taste hat eine Größe von 60x60px. Eine Flexbox wird verwendet, um die Tasten jeweils mittig in den einzelnen Positionen des Grids zu positionieren. Jede Taste hat einen dunkelgrauen Hintergrund, eine weiße Textfarbe, einen 2px dicken dunklen Rahmen und abgerundete Ecken. Die Optionen `-webkit-tap-highlight-color: transparent` und `user-select: none` sorgen dafür, dass auf mobilen Geräten der Text in den Tasten nicht blau markiert werden kann, damit die Steuerung nicht blockiert wird.

```
.dpad-container {
  display: grid;
  grid-template-areas:
    ". up ."
    "left center right"
    ". down .";
  gap: 10px;
  width: 120px;
  margin: 50px auto;
  position: absolute;
  right: 100px;
  top: 45%;
  transform: translateY(-50%);
}

.button {
  width: 60px;
  height: 60px;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #333;
  color: white;
  font-size: 18px;
  cursor: pointer;
  border: 2px solid #444;
  border-radius: 5px;

  -webkit-tap-highlight-color: transparent;
  user-select: none;
}
```

Abbildung 47: CSS-Klassen `.dpad-container` und `.button`

Quelle: eigene Abbildung

---

Die fünf Richtungsklassen sorgen dafür, dass die Tasten des Steuerkreuzes in den zuvor definierten Grid-Bereichen zugewiesen und richtig platziert werden. Die mittlere Taste bekommt außerdem eine leicht hellere Farbe zugewiesen, um ihn optisch von den anderen abzuheben.

Außerdem bekommen alle Tasten einen Hover-Effekt, damit sie etwas heller werden, wenn eine Taste gedrückt wird.

```
.up {  
  |   grid-area: up;  
}  
  
.down {  
  |   grid-area: down;  
}  
  
.left {  
  |   grid-area: left;  
}  
  
.right {  
  |   grid-area: right;  
}  
  
.center {  
  |   grid-area: center;  
  |   background-color: #555;  
}  
  
.button:hover {  
  |   background-color: #666;  
}
```

Abbildung 48: CSS-Richtungsklassen

Quelle: eigene Abbildung

Im JavaScript-Code sorgen zwei EventListener dafür, dass die Elemente auf der Seite nicht ziehbar und verschiebbar sind und dass das Rechtsklick-Menü nicht geöffnet werden kann. Diese Maßnahmen verhindern unerwünschte Benutzerinteraktionen und sorgen für eine reibungslose Bedienung.

```

document.addEventListener('dragstart', event => {
|   event.preventDefault();
});

document.addEventListener('contextmenu', event => {
|   event.preventDefault();
});

```

Abbildung 49: EventListener für ungewünschte Aktionen

Quelle: eigene Abbildung

Vier weitere EventListener kümmern sich um die Eingabe des Steuerkreuzes. Es wird zuerst jeder mousedown (Linksklick) bzw. ein touchstart (klick auf mobile Geräte) abgefangen. Danach wird überprüft, ob das geklickte Element innerhalb eines .button Elements (Steuerkreuztaste) liegt. Falls ja, wird die Richtung aus dem data-direction-Attribut gelesen und an die send() Funktion übergeben. Wenn ein mouseup (Maus loslassen) oder ein touchend (Finger loslassen) abgefangen wird, wird die Funktion stop() aufgerufen.

```

document.addEventListener('mousedown', (event) => {
|   const target = event.target.closest('.button');
|   if (target) {
|       const direction = target.dataset.direction;
|       if (direction) {
|           send(target.dataset.direction);
|       }
|   }
});

document.addEventListener('mouseup', (event) => {
|   const target = event.target.closest('.button');
|   if (target) {
|       stop();
|   }
});

document.addEventListener('touchstart', (event) => {
|   const target = event.target.closest('.button');
|   if (target) {
|       const direction = target.dataset.direction;
|       if (direction) {
|           send(direction);
|       }
|   }
});

document.addEventListener('touchend', (event) => {
|   const target = event.target.closest('.button');
|   if (target) {
|       stop();
|   }
});

```

Abbildung 50: EventListener für Eingabe

Quelle: eigene Abbildung

---

Die JavaScript Funktion `send()` kümmert sich um das Senden der Steuerungsbefehle. Zuerst wird überprüft, ob die aktuelle Richtung nicht bereits die gewünschte Richtung ist. Falls ja, wird ein JSON-Objekt erstellt, welche die Richtung sowie die Geschwindigkeit enthält. Zu debug-Zwecken wird die Richtung sowie die Geschwindigkeit in der Webkonsole ausgegeben. Danach wird überprüft, ob die WebSocket-Verbindung zur ESP32-CAM offen ist. Wenn ja, wird die Nachricht auch für debug-Zwecken an die ESP32-CAM gesendet, ansonsten wird eine Fehlermeldung ausgegeben. Dasselbe geschieht für die WebSocket-Verbindung zum ESP32. Dort werden jedoch die Steuerbefehle weiterverarbeitet.

```
let currentdir_robot = Directions.HALT;

function send(direction) {
  if (currentdir_robot !== direction) {
    currentdir_robot = direction;

    const message = JSON.stringify({
      robot_direction: currentdir_robot,
      speed: speed
    });

    console.log("Direction: " + direction + " Speed: " + speed);

    if (websocket_cam.readyState === WebSocket.OPEN) {
      websocket_cam.send(message);
      console.log("An CAM gesendet");
    } else {
      console.error("Senden fehlgeschlagen. WebSocket Cam is not open");
    }

    if (websocket_carybot.readyState === WebSocket.OPEN) {
      websocket_carybot.send(message);
      console.log("An Carybot gesendet");
    } else {
      console.error("Senden fehlgeschlagen. WebSocket Carybot is not open");
    }
  }
}
```

Abbildung 51: `send()`-Funktion

Quelle: eigene Abbildung

Die JavaScript Funktion `stop()` sorgt dafür, dass der Roboter anhält, wenn eine Taste des Steuerkreuzes losgelassen wird. Zuerst wird überprüft, ob die aktuelle Richtung nicht bereits `HALT` ist, ansonsten wird sie überschrieben. Danach wird ein Stopp-Befehl als JSON-Nachricht vorbereitet. Zu debug-Zwecken wird wieder ein `halt` in der Webkonsole ausgegeben. Danach wird der Stopp-Befehl wieder an die WebSocket-Verbindung zur ESP32-CAM für debug-Zwecke gesendet. Danach wird sie auch an die WebSocket-Verbindung zum ESP32 gesendet, wo dieser weiterverarbeitet wird.

```
function stop() {
  if (currentdir_robot !== Directions.HALT) {
    currentdir_robot = Directions.HALT;

    const message = JSON.stringify({
      robot_direction: Directions.HALT,
      speed: speed
    });

    console.log('Message sent: halt');

    if (websocket_cam.readyState === WebSocket.OPEN) {
      websocket_cam.send(message);
    } else {
      console.error("Senden fehlgeschlagen. WebSocket Cam is not open");
    }

    if (websocket_carybot.readyState === WebSocket.OPEN) {
      websocket_carybot.send(message);
    } else {
      console.error("Senden fehlgeschlagen. WebSocket Carybot is not open");
    }
  }
}
```

Abbildung 52: stop()-Funktion

Quelle: eigene Abbildung

## Geschwindigkeitseinstellung

Um die Fortbewegungsgeschwindigkeit des Roboters kontrollieren zu können, gibt es auf der linken Seite neben dem Steuerkreuz einen vertikalen Geschwindigkeitsslider, mit der die Geschwindigkeit der Motoren gesteuert werden kann. Wenn der Slider nach oben gezogen wird, beschleunigt der Roboter, wenn der Slider nach unten gezogen wird, wird die Geschwindigkeit verlangsamt.

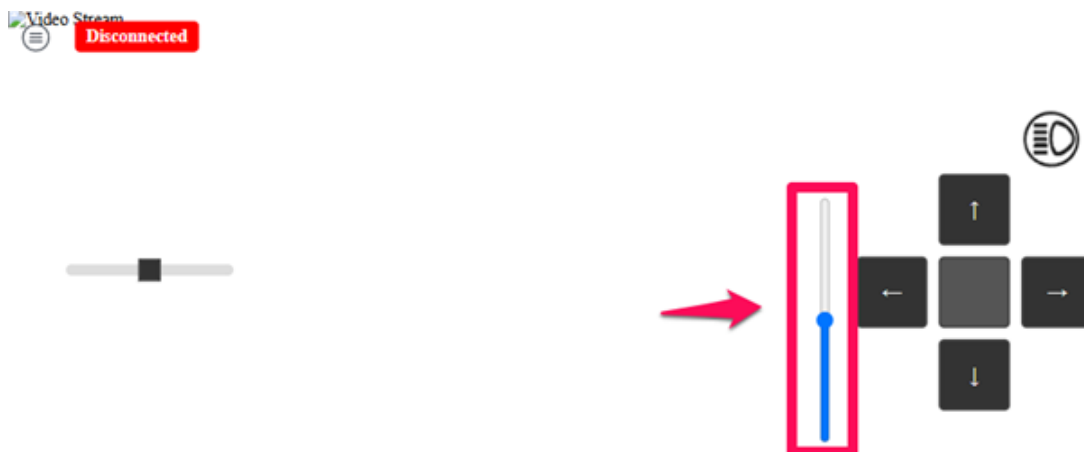


Abbildung 53: Geschwindigkeitssteuerung auf der Website

Quelle: eigene Abbildung



---

Im HTML-Code wird mit der CSS-Klasse `slider-container` ein Bereich im Hauptbereich definiert. Der Slider wird als HTML `<input>`-Tag mit dem `type` `range` von 0 bis 100 definiert. Für das Design wird die CSS-Klasse `slider` verwendet und beim Verschieben des Sliders wird die JavaScript Funktion `speed_change()` mit der aktuellen Position des Sliders aufgerufen.

```
<div class="slider-container">
  <input type="range" min="0" max="100" value="50" class="slider" id="rangeSlider"
    oninput="speed_change(this.value)">
</div>
```

Abbildung 54: Geschwindigkeitssteuerung HTML-Code

Quelle: eigene Abbildung

In der CSS-Klasse `slider-container` wird ein Bereich mit der Größe 50x220 Pixel definiert, in dem der Slider angezeigt werden soll. In `slider` wird definiert, dass der Slider vertikal und nicht horizontal angezeigt wird. Außerdem wird die Richtung auf `Right to Left` gesetzt, damit beim Hochschieben des Sliders die aktuelle Position erhöht und beim Runterschieben vermindert wird.

```
.slider-container {
  display: grid;
  width: 50px;
  position: absolute;
  right: 220px;
  top: 35%;
}

.slider {
  width: 80%;
  height: 220px;
  margin-top: 10px;
  writing-mode: vertical-lr;
  direction: rtl;
  vertical-align: middle;
}
```

Abbildung 55: CSS-Klassen für die Geschwindigkeitssteuerung

Quelle: eigene Abbildung

---

Im JavaScript-Code wird die aktuelle Position des Sliders in die eigene Variable `speed` gespeichert. Die Geschwindigkeit wird immer bei einem Steuerbefehl des Steuerkreuzes mitübertragen. (siehe Steuerkreuz 52)

```
var speed = 50;

function speed_change(input_speed) {
    speed = input_speed;
}
```

Abbildung 56: `speed-change()`-Funktion

Quelle: eigene Abbildung

## Kamerasteuerung

Um mit der Videoübertragung besser Objekte und Hindernisse zu erkennen, ist die Kamera leicht schwenkbar. Um nun die Kamera auf unserer Website bewegen zu können, gibt es auf der linken Seite einen horizontalen Slider, mit der die Kamera ein Stück links und rechts geschwenkt werden kann.



Abbildung 57: Kamerasteuerung auf der Website

Quelle: eigene Abbildung

Im HTML-Code wird ein mit der CSS-Klasse `camera-container` ein Bereich im Hauptbereich definiert. Der Slider wird als HTML `<input>`-Tag mit dem type `range` von 0 bis 180 definiert. Für das Design wird die CSS-Klasse `cam_slider` verwendet und beim Verschieben des Sliders wird die JavaScript Funktion `camera_change()` mit der aktuellen Position des Sliders aufgerufen.

---

```
<div class="camera-container">
  <input type="range" min="0" max="180" value="90" class="cam_slider" id="cameraSlider"
  |   oninput="camera_change(this.value)">
</div>
```

Abbildung 58: Kamerasteuerung HTML-Code

Quelle: eigene Abbildung

In der CSS-Klasse `camera-container` wird ein Bereich erstellt, in dem der Slider angezeigt werden soll. In `cam_slider` wird die Slidespur mit einer Breite von 150 Pixel und einer Höhe von 10 Pixel definiert. Der Hintergrund ist hellgrau und die Ecken werden leicht abgerundet. In `webkit-slider-thumb` wird der Sliderknopf mit 20x20 Pixel definiert. Der Hintergrund ist dunkelgrau mit einem 2px breiten, dunkleren Rand.

```
.camera-container {
  display: grid;
  position: absolute;
  top: 50%;
  left: 50px;
}

.cam_slider {
  -webkit-appearance: none;
  appearance: none;
  width: 150px;
  height: 10px;
  background: #ddd;
  outline: none;
  border-radius: 5px;
}

.cam_slider::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 20px;
  height: 20px;
  background: #333;
  cursor: pointer;
  border: 2px solid #444;
}
```

Abbildung 59: CSS-Klassen für die Kamerasteuerung

Quelle: eigene Abbildung

Im JavaScript-Code wird die aktuelle Position des Sliders verarbeitet. Dazu wird die Position in eine JSON-Nachricht gespeichert. Zu debug-Zwecken wird die Position noch in der Website Konsole ausgegeben. Wenn der WebSocket Verbindung zum ESP32 offen ist, wird die aktuelle Position übermittelt.

```
function camera_change(input_position) {  
  const message = JSON.stringify({  
    camera_position: input_position  
  });  
  
  console.log("Camera Position: " + input_position);  
  
  if(websocket_carybot.readyState === WebSocket.OPEN){  
    websocket_carybot.send(message);  
  }  
}
```

Abbildung 60: camera-change()-Funktion

Quelle: eigene Abbildung

## Fernlicht

Die Funktion des Fernlichts dient bei unserem Roboter zur Beleuchtung bei schlechter Sicht bzw. Dunkelheit. Wenn eingeschaltet, leuchtet auf der Vorder- sowie auf der Hinterseite des Roboters ein Scheinwerfer die Umgebung aus.

Um das Fernlicht auf unserer Website ein- bzw. auszuschalten gibt es ein eigenes Fernlicht-Icon rechts über dem Steuerkreuz. Im ausgeschalteten Zustand ist das Icon ausgegraut. Im aktiven Zustand ist das Scheinwerfer Symbol im Icon blau und das Icon hat einen blassgrünen Hintergrund.



Abbildung 61: Fernlicht-Icon auf der Website

Quelle: eigene Abbildung

---

Im HTML-Code wird das Icon als HTML `<img>`-Tag eingefügt. Als Quelle wird die im SPIFFS hochgeladene `Fernlicht.svg` Grafik verwendet. Für das Design wird die CSS-Klasse `fernlicht-icon` verwendet und beim `onclick` Event wird die JavaScript Funktion `togglelight()` aufgerufen.

```

```

Abbildung 62: HTML-Code für Fernlicht

Quelle: eigene Abbildung

In der CSS-Klasse `fernlicht-icon` wird das Icon 20% vom oberen Rand und 2% von rechten Rand mit einer Größe von 50x50 Pixel positioniert. Es liegt in der zweiten Ebene über dem Hintergrund, so dass es nicht von anderen Elementen überdeckt wird. Der Hintergrund, der außerhalb des Icons liegt, wird transparent angezeigt. Beim Ändern der Farbe wird eine sanfte Animation über 0,3 Sekunden angezeigt.

Die `grayscale` Klasse wandelt das Icon in die ausgegraute Darstellung um, um anzuzeigen, dass das Fernlicht deaktiviert ist. Die `active-light` Klasse ändert die Hintergrundfarbe zu einem transparenten grünen Ton, um anzuzeigen, dass das Fernlicht aktiv ist.

```
.fernlicht-icon {
    position: fixed;
    top: 20%;
    right: 2%;
    width: 50px;
    height: 50px;
    cursor: pointer;
    z-index: 2;
    background-color: transparent;
    transition: filter 0.3s ease;
}

.grayscale {
    filter: grayscale(100%)
}

.active-light {
    background-color: rgba(0, 255, 0, 0.3);
    border-radius: 10px;
}
```

Abbildung 63: CSS-Klassen für das Fernlicht

Quelle: eigene Abbildung

---

Im JavaScript Code wird bei jedem onclick Event die Variable `light` getoggled. Wenn die Variable `light` `true` ist, wird die CSS-Klasse `active-light` angewendet, ansonsten die CSS-Klasse `grayscale`. Anschließend wird eine JSON-Nachricht mit dem aktuellen Zustand der `light` Variable erstellt. Wenn die WebSocket Verbindung zum ESP32 offen ist, wird die JSON-Nachricht versendet, ansonsten wird eine Fehlermeldung ausgegeben.

```
var light = false;

function togglelight() {
  light = !light;
  const fernlichtIcon = document.getElementById("Fernlicht");

  if(light) {
    fernlichtIcon.classList.remove("grayscale");
    fernlichtIcon.classList.add("active-light")
  } else {
    fernlichtIcon.classList.add("grayscale");
    fernlichtIcon.classList.remove("active-light");
  }

  const message = JSON.stringify({
    light_status: light
  });

  if(websocket_carybot.readyState === WebSocket.OPEN) {
    websocket_carybot.send(message);
  } else {
    console.error("Senden fehlgeschlagen. WebSocket Carybot is not open");
  }
}
```

Abbildung 64: `togglelight()`-Funktion

Quelle: eigene Abbildung

#### 4.4.2 Anzeige von Sensordaten und Verbindungsstatus

##### Sensordaten

In der linken oberen Ecke der Website befindet sich das Menü-icon. Ein Klick darauf ruft die Sidebar für die Daten auf. Die Sidebar klappt sich am linken Bildschirmrand auf. Darin ist der aktuelle Akkustand sowie das aktuelle aufliegende Gewicht auslesbar. Um das Menü wieder schließen zu können, ist ein erneuter Klick auf das Icon erforderlich und die Sidebar wird wieder zugeklappt.

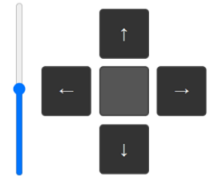


Abbildung 65: Menü-Icon auf der Website

Quelle: eigene Abbildung

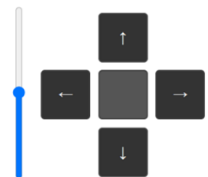
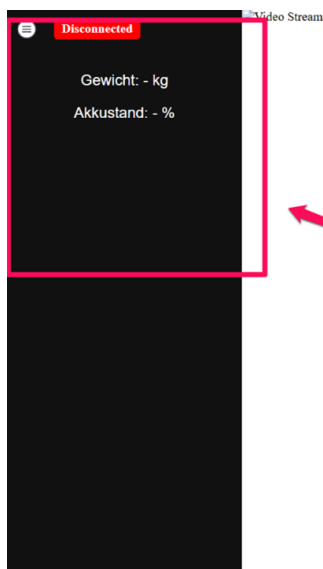


Abbildung 66: Sidebar mit Sensordaten

Quelle: eigene Abbildung

Im HTML-Code wird im Body-Bereich wird das Icon als HTML `<img>`-Tag eingefügt. Als Quelle wird die im SPIFFS hochgeladene Grafik `menu-icon.svg` Grafik verwendet. Für das Design wird die CSS-Klasse `menu-icon` verwendet und beim onclick Event wird die JavaScript Funktion `togglemenu()` aufgerufen.

Im HTML `<div>`-Tag werden die einzelnen Daten festgelegt und mit der CSS-Klasse `mymenu` formatiert. Das Gewicht und der Akkustand werden als HTML `<p>`-Tag angezeigt und mit der CSS-Klasse `daten` formatiert.

---

```
<div id="menu" class="mymenu">
  <p id="Gewicht" class="daten">Gewicht: - kg</p>
  <p id="Akkustand" class="daten">Akkustand: - %</p>
</div>

```

Abbildung 67: Sensordaten HTML-Code

Quelle: eigene Abbildung

In der CSS-Klasse mymenu wird die sidebar formatiert. Das Menü ist fixiert und nimmt 100% der Höhe ein. Die anfängliche Höhe ist 0, da es versteckt bzw. eingeklappt ist. Die Hintergrundfarbe ist schwarz und das Menü liegt auf der ersten Ebene über dem Hauptinhalt, damit es diesen überlagern kann, wenn es eingeblendet wird. Beim Ein- und Ausblenden dauert 0.5 Sekunden, um einen sanften Übergang zu ermöglichen.

Der Hauptinhalt #main hat eine Animation, damit er beim Öffnen des Menüs nach rechts verschoben werden kann.

Falls der Bildschirm kleiner als 450px Höhe hat, werden die Abstände der angezeigten Daten verkleinert. Das dient dazu, um besonders auf Handys die Ansicht zu optimieren.

Die CSS-Klasse menu-icon positioniert das Icon in der linken oberen Ecke mit einer 30x30px Format. Cursor: pointer sorgt dafür, dass das Icon anklickbar ist.



```

.mymenu {
  height: 100%;
  width: 0;
  position: fixed;
  z-index: 1;
  top: 0;
  left: 0;
  background-color: #111;
  overflow-x: hidden;
  padding-top: 60px;
  transition: 0.5s;
}

#main {
  transition: margin-left .5s;
  padding: 0;
  height: 100%;
}

@media screen and (max-height: 450px) {
  .mymenu {
    padding-top: 15px;
  }
}

.menu-icon {
  position: fixed;
  top: 10px;
  left: 10px;
  width: 30px;
  height: 30px;
  cursor: pointer;
  z-index: 2;
  background-color: transparent
}

```

Abbildung 68: CSS-Klassen für das Menü

Quelle: eigene Abbildung

Die CSS-Klasse `daten` formatiert die einzelnen Daten im Menü. Sie werden in der Schriftart Arial, Helvetica oder sans-serif angezeigt und in der Farbe Weiß, mit der Schriftgröße 1.2em (20% größer) mittig angezeigt.

```
.daten {
  font-family: Arial, Helvetica, sans-serif;
  text-align: center;
  color: ■ white;
  font-size: 1.2em;
}
```

Abbildung 69: CSS-Klasse .daten

Quelle: eigene Abbildung

In der JavaScript Funktion `connectWebSocket_carybot()` wird im `onmessage()`-event die übertragenen Daten verarbeitet. Für debug-Zwecken werden die angekommen Daten in der WebKonsole ausgegeben. Danach werden die JSON-Nachrichten extrahiert. Wenn die Nachricht den Akkustand beinhaltet, wird dieser im HTML `<p>`-Tag mit der id Akkustand im Menü angezeigt. Wenn die Nachricht das Gewicht beinhaltet, wird dieses im HTML `<p>`-Tag mit der id Gewicht im Menü angezeigt. Falls beim Umwandeln ein Fehler auftreten sollte, wird dieser in der Webkonsole ausgegeben.

```
websocket_carybot.onmessage = (event) => {
  console.log("Received:", event.data);

  try {
    const data = JSON.parse(event.data);
    if (data.battery) {
      document.getElementById('Akkustand').innerText = "Akkustand: " + data.battery + "%";
    }
    if (data.weight) {
      document.getElementById('Gewicht').innerText = "Gewicht: " + data.weight + "kg";
    }
  } catch (e) {
    console.error("Error parsing JSON:", e);
  }
};
```

Abbildung 70: JavaScript-Verarbeitung der Sensordaten

Quelle: eigene Abbildung

## Verbindungsstatus

In der WebSocket Status-Box wird der aktuelle Verbindungszustand zum ESP32 angezeigt. Wenn keine Verbindung vorhanden ist, ist die Box rot. Wenn die Verbindung erfolgt, wird die Box grün.



Abbildung 71: Statusbox auf der Website  
Quelle: eigene Abbildung



Abbildung 72: Statusbox:  
Verbunden



Abbildung 73: Statusbox:  
Verbindung getrennt

Im HTML-Code wird die Status-Box als HTML `<span>`-Tag definiert. Für das Design wird die CSS-Klasse `status-box` kombiniert je nach Verbindungsstatus mit der Klasse `connected` oder `disconnected`.

```
<span id="connectionStatus" class="status-box disconnected">Disconnected</span>
```

Abbildung 74: Statusbox HTML-Code

Quelle: eigene Abbildung

In der CSS-Klasse `status-box` wird die grundlegende Box am linken oberen Bildschirmrand positioniert. Die Box liegt auf der zweiten z-Ebene. Die Schrift wird weiß definiert und wird fett dargestellt. Wenn eine Verbindung hergestellt wurde, wird der Hintergrund auf grün festgelegt (`status-box.connected`), ansonsten ist der Hintergrund rot (`status-box.disconnected`).

```

.status-box {
  top: 10px;
  left: 50px;
  z-index: 2;
  position: fixed;
  padding: 5px 10px;
  margin-left: 10px;
  border-radius: 5px;
  font-weight: bold;
  color: white;
}

.status-box.connected {
  background-color: green;
}

.status-box.disconnected {
  background-color: red;
}

```

Abbildung 75: CSS-Klassen für die Statusbox

Quelle: eigene Abbildung

Im JavaScript-Code werden die `onopen()` und `onclose()` Events der WebSocket Verbindung gehandelt. Bei einem Verbindungsaufbau (`onopen()`) wird die Status-Box grün und der Text wechselt zu `Connected`. Außerdem wird eine Nachricht für Debug-Zwecke ausgegeben. Bei einem Verbindungsabbruch wird die Status-Box wieder rot und der Text wechselt zu `Disconnected`. Es wird automatisch alle 5 Sekunden ein neuer Versuch zur Verbindungsaufbau gestartet.

```

websocket_carybot.onopen = function () {
  console.log("WebSocket Carybot verbunden");
  document.getElementById('connectionStatus').innerText = 'Connected';
  document.getElementById('connectionStatus').className = 'status-box connected';
}

websocket_carybot.onclose = function () {
  document.getElementById('connectionStatus').innerText = 'Disonnected';
  document.getElementById('connectionStatus').className = 'status-box disconnected';

  console.log('WebSocket Carybot getrennt. Erneuter Versuch in 5 Sekunden...');
  setTimeout(connectWebSocket_carybot, 5000); // Versuchen, die Verbindung erneut herzustellen
}

```

Abbildung 76: JavaScript Funktionen für den Verbindungsstatus

Quelle: eigene Abbildung

---

## 4.5 Herausforderungen und Optimierungen

### 4.5.1 Latenz- und Performance Optimierungen

#### Kameraoptimierungen

Als Bildformat wird JPEG verwendet, um Speicherplatz zu sparen. Alternativ wäre RGB565 oder YUV422 möglich, jedoch benötigen diese aufgrund fehlender Kompression mehr Speicher und verlängern die Übertragungsdauer.

Für die Bildauflösung wird QVGA (320x240 Pixel) definiert. Alternativ wären noch VGA (640x480), SVGA (800x600) oder UXGA (1600x1200) möglich, jedoch benötigen diese mehr Speicher und mehr Bandbreite und verlängern dadurch die Übertragungsdauer.

Den Wert für die Bildqualität kann von 0 (= beste Qualität) bis 63 (= schlechteste Qualität) definiert werden. Wir definierten die Bildqualität mit 12, was ein guter Kompromiss zwischen Qualität und Speicherverbrauch ist.

Wir wählten für die Anzahl der Framebuffer 3. Die Anzahl sagt aus, wie viele Bilder gleichzeitig gespeichert werden können. Mehr Framebuffer erhöhen die Bildrate, benötigen jedoch mehr RAM. (siehe Kamera-Initialisierung 124)

---

# 5 Gehäuse

## 5.1 Planung und Design

Das Gehäuse des Lastenroboters wurde entwickelt, um die Sicherheit und Stabilität beim Transport von schweren Lasten zu gewährleisten. Dabei lag der Fokus auf einer stabilen Bauweise, damit der Lastenroboter ein Gewicht von zirka 25 Kilo aushaltet.

### **Wichtigsten Aspekte bei der Planung:**

1. Stabilität: Das Gehäuse muss das Eigengewicht des Roboters aber auch das Gewicht der Last aushalten.
2. Größe: Bei der Größe ist es wichtig darauf zu achten das alle Komponenten problemlos ins Gehäuse passen.
3. Fahrverhalten: Durch die Verwendung von vier Räder, die jeweils über einen Motor angetrieben werden, soll eine gute Manövrierfähigkeit erreicht werden.
4. Integration der Elektronik: Alle Komponenten müssen sicher im Gehäuse untergebracht werden.
5. Gewichtsmessung: Die Wägezelle muss in das Gehäuse integriert werden, um eine präzise Messung zu ermöglichen.
6. Wartungsfreundlichkeit: Bauteile sollten leicht zugänglich sein, um Reparaturen oder Anpassungen durchführen zu können.

### **Mechanische Konstruktion**

Nach der Planung der wichtigsten Aspekte wurden zwei 3D-Modelle des Lastenroboters in Fusion 360 entworfen.

---

## Erstes Modell

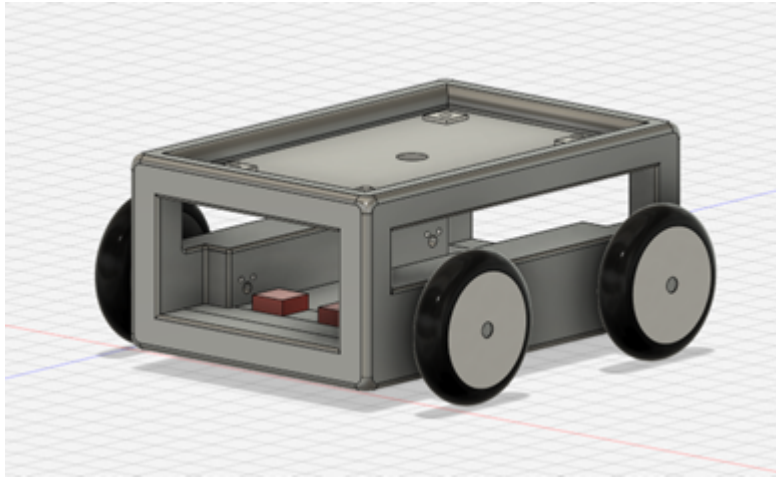


Abbildung 77: Erstes Modell

Quelle: eigene Abbildung

Das erste Modell des Lastenroboters diente hauptsächlich als Prototyp. Es wurden keine genauen Maße beachtet, sondern dient als grobe Vorstellung wie der Lastenroboter aussehen könnte. Wichtig dabei war es die grundlegenden Komponenten wie Motoren, Räder und Wägezelle zu skizzieren, um die allgemeine Funktionalität sicherzustellen.

In diesem frühen Entwicklungsstadium lag der Fokus auf der Mechanik und der Stabilität des Designs. Das erste Modell war sehr wichtig, um Schwachstellen und Optimierungsmöglichkeiten zu identifizieren und half dabei, das zweite Modell präzise und durchdacht zu planen.

## Zweites Modell

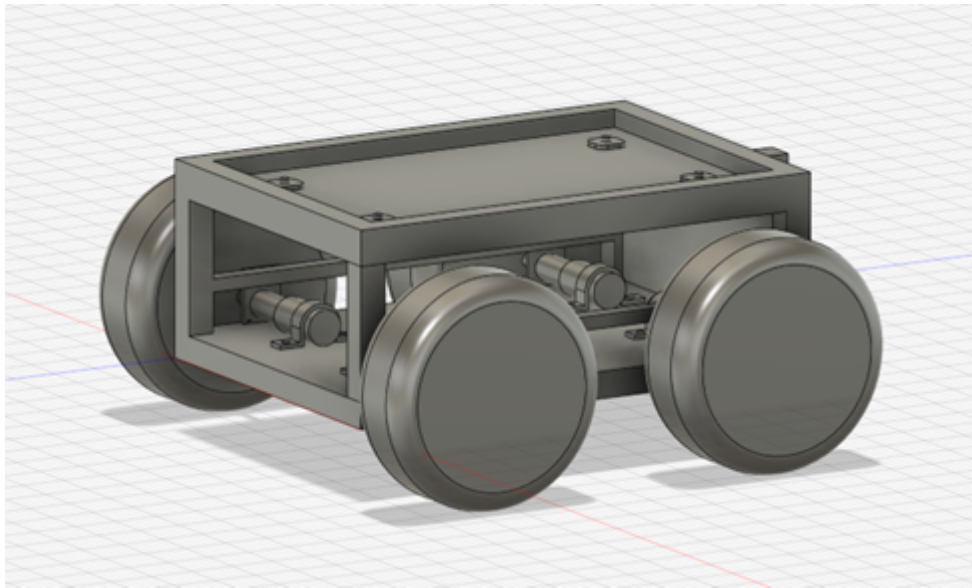


Abbildung 78: Zweites Modell

Quelle: eigene Abbildung

Anhand der Erkenntnisse aus dem ersten Modell wurde das zweite Modell entwickelt, das als fertiges Modell für den Bau des Lastenroboters dient. In dieser Version wurden exakte Maße festgelegt und die genaue Position der Komponenten wie Räder, Motoren, Kamera und Wägezelle festgelegt.

Das Wichtigste beim Entwickeln ist die Größe des Gehäuses. Das Gehäuse basiert auf einem rechteckigen Stahlrahmen. Dieser Rahmen ist wichtig, weil er die Grundstruktur und Stabilität gewährleistet. Der rechteckige Rahmen des Lastenroboters hat eine Höhe von 220mm, eine breite von 400mm und eine Länge von 560mm. Die Vierkantstahlrohre haben eine Größe von 30mm mal 40mm.

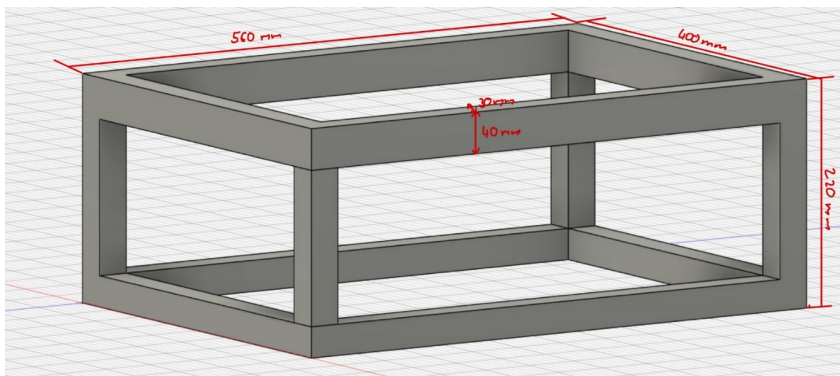


Abbildung 79: Rahmen

Quelle: eigene Abbildung



---

Im nächsten Schritt wurde Halterung für die Elektronik Komponenten entwickelt. Dabei handelt es sich um eine Platte, die auf der Unterseite mit zwei Querstreben verstärkt wird. Die Größe der Platte beträgt 34mm mal 50mm und hat eine Materialstärke von 5mm. Die Querstreben haben eine Länge von 50mm, sind 25mm hoch und 25mm breit.

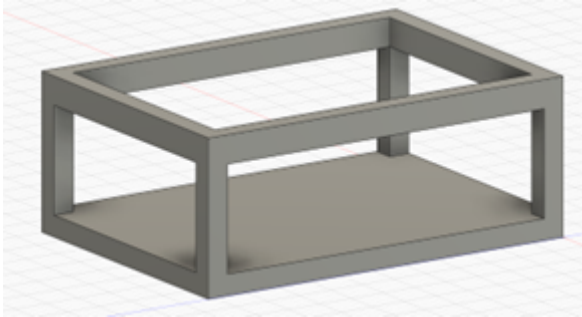


Abbildung 80: Halterung

1

Quelle: eigene Abbildung

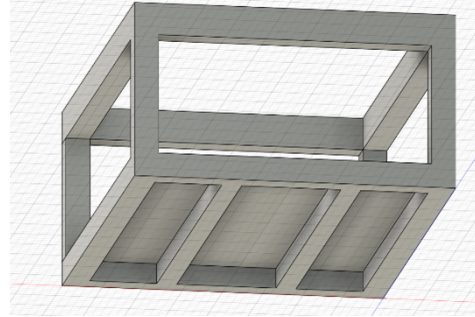


Abbildung 81: Halterung

2

Quelle: eigene Abbildung

Danach wurde die Halterung für die Motoren entwickelt. Dabei ist zu beachten, dass sich die Motoren während des Betriebs weder verdrehen noch verrutschen. Dies ist entscheidend, da die Motoren ein hohes Drehmoment erzeugen, das ohne eine stabile Befestigung zu Instabilitäten führen könnte. Die Motoren sind auf einer Querstange befestigt damit sie sich nicht mehr verdrehen können. Die Querstangen haben eine Länge von 50mm und sind 15mm hoch und 20mm breit. Zusätzlich sind die Motoren auch mit einer Schelle am Boden fixiert, um ungewolltes verrutschen zu vermeiden.

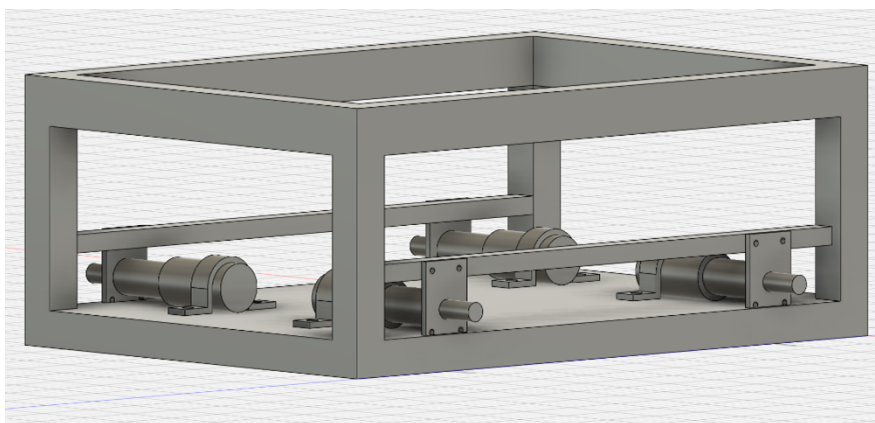


Abbildung 82: Motorenhalterung

Quelle: eigene Abbildung

---

Nach dem die Halterung für die Motoren entwickelt wurde, erfolgt die Auswahl und Befestigung der Reifen. Die Reifen spielen eine entscheidende Rolle für das Fahrverhalten.

Die Räder bestehen aus Gummi für eine gute Bodenhaftung. Jedes der vier Reifen ist direkt an der Motorachse befestigt. Die Motorachse hat einen Durchmesser von 16mm. Die Motorachse verfügt über ein M8-Gewinde, das für die Befestigung der Räder verwendet wird. Die Räder werden mit einer M8-Schraube in die Motorachse geschraubt.

Wichtig bei der Entwicklung war die Radgröße damit das Gehäuse genug Bodenabstand hat. Es wurden Reifen gewählt mit einem Durchmesser von 260mm und einer Breite von 85mm.

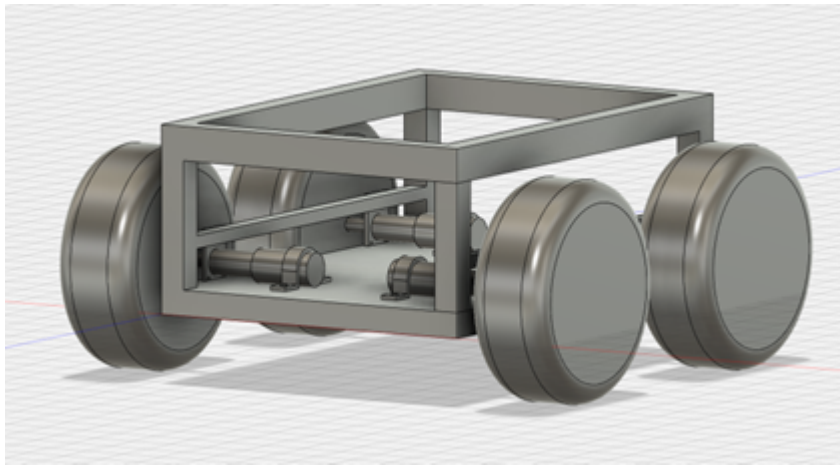


Abbildung 83: Räder

Quelle: eigene Abbildung

Ein weiterer großer Punkt bei der Planung war die Gewichtsmessung. Die Gewichtsmessung erfolgt über vier Wägezellen die in einem Rechteck auf einer Platte angeordnet sind. Auf die Wägezellen wird eine Platte montiert, auf die die Last gestellt wird.

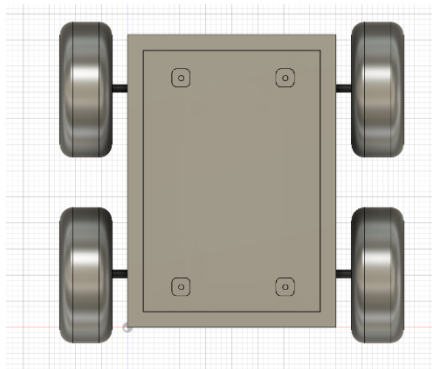


Abbildung 84: Wägezellen

Quelle: eigene Abbildung

---

Damit die Wägezellen sicher fixiert sind und nicht verrutschen, werden sie in einer 3D-gedruckten Halterung befestigt. Die Halterung hat eine Einkerbung, die genau für die Wägezelle abgestimmt ist, damit die Wägezelle nicht mehr verrutschen kann. Es gibt zusätzlich eine Einkerbung für die Leitungen, damit diese nicht beschädigt werden. Am Rand der Halterung sind zwei Löcher damit die Halterung auf der Platte angeschraubt werden kann.

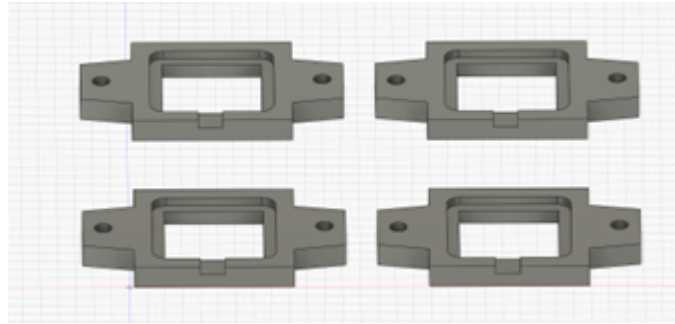


Abbildung 85: Wägezellenhalterung

Quelle: eigene Abbildung

Im letzten Schritt des Designs wurden zwei Platten an der Vorderseite und Rückseite des Lastenroboters angebracht. Die hintere Platte dient für die Befestigung des An/Aus Schalters und des Ladeanschlusses für den Lastenroboter. Auf der vorderen Platte soll die Kamera montiert werden.

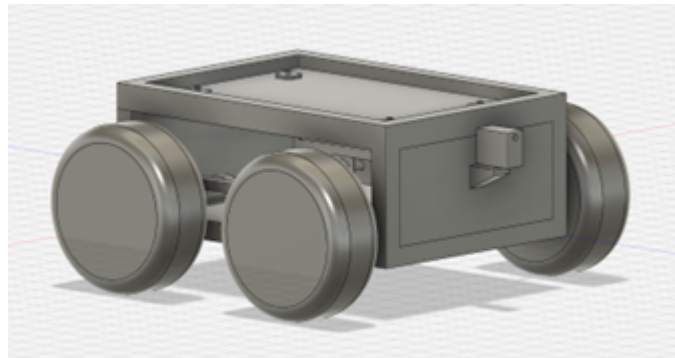


Abbildung 86: Fertiges Modell

Quelle: eigene Abbildung

### Materialwahl für das Gehäuse

Für das Gehäuse wurde Stahl als Hauptmaterial gewählt, weil es sehr robust und widerstandsfähig gegenüber äußeren Einflüssen ist. Ein weiter Grund, warum Stahl als Hauptmaterial verwendet wurde, ist die einfache Verarbeitung. Stahl kann man gut schweißen aber auch verschrauben.

---

## 5.2 Realisierung

Nach der abgeschlossenen Planungsphase begann die schrittweise Umsetzung des Baus des Lastenroboters. Dabei wurde besonders darauf geachtet, dass alle Komponenten präzise gefertigt und montiert wurden, um die gewünschte Stabilität, Funktionalität und Sicherheit zu gewährleisten. Der Bau des Gehäuses erfolgte in mehreren aufeinanderfolgenden Arbeitsschritten.

### Schritt 1: Zuschnitt der Stahlprofile

Im ersten Schritt werden die Vierkantrohre anhand der geplanten Maße zugeschnitten. Bevor die Vierkantrohre zugeschnitten werden, müssen sie sorgfältig vermessen und an den Schnittstellen präzise markiert werden. Nachdem die Vierkantrohre markiert wurden, wurden sie in einen Schraubstock eingespannt und mit einer Flex an den Markierungen zugeschnitten. Die einzelnen Rohre mussten genau zugeschnitten werden, um eine stabile Konstruktion zu erhalten.



Abbildung 87: Stahlprofile

Quelle: eigene Abbildung

Nachdem alle Vierkantrohre zugeschnitten waren, wurden die einzelnen Stücke erneut vermessen, um zu überprüfen, ob alle Maße stimmen und die Teile passgenau zueinanderpassen, bevor sie endgültig befestigt wurden.

### Schritt 1: Aufbau des Rahmens

Nachdem alle Stahlteile zugeschnitten waren, wurde das Gehäuse verschweißt. Zuerst wurde der rechteckige Stahlrahmen ausgerichtet, um sicherzustellen, dass alle Teile korrekt positioniert sind. Um den Zugang zur Elektronik im Inneren des Lastenroboters zu erleichtern, wurden am oberen Rahmen zwei kurze Stangen angebracht, sodass der obere Teil leicht abgenommen werden kann.



Abbildung 88: Oberer Rahmen

Quelle: eigene Abbildung

Für das Verschweißen wurde das Elektrodenschweißverfahren (Lichtbogenhandschweißen) verwendet.

#### **Funktionsweise des Elektrodenschweißens:**

Beim Elektrodenschweißen wird ein elektrischer Lichtbogen zwischen der Stabelektrode und Material erzeugt. Durch die hohe Temperatur des Lichtbogens schmilzt die Elektrode aber auch das Material. Die Elektrode besteht aus einem Metallkern und einer Umhüllung und liefert somit den Zusatzwerkstoff gleich mit.

Zum Schweißen wird die Stabelektrode in den Elektrodenhalter eingespannt und vom Schweißer an der Nahtstelle entlanggeführt. Die Umhüllung schmilzt während des Schweißens ab und schützt durch freiwerdende Gase und Schlacke das Schmelzbad und den Lichtbogen vor der Außenluft.<sup>2</sup>

---

<sup>2</sup><https://www.lorch.eu/de/produkte/manuelles-schweissen/elektroden-schweissen>

---

## 5.3 Vor- und Nachteile von Elektrodenschweißen

### Vorteile

- Einfache Handhabung
- Unabhängig von Stromnetz und Wetter
- Relativ robust gegen Verunreinigungen wie Rost
- Schweißen von nahezu allen Werkstoffen
- Geringe Lärmbelastung der Schweißer

### Nachteile

- Mechanisierung nicht möglich
- Hohe Rauchentwicklung
- Geringe Geschwindigkeit und hohe Rüstzeiten möglich
- Elektrodendurchmesser wird von Blechdicke und Schweißposition bestimmt
- Endkrater und Ansatzstellen können zu Fehlerquellen werden

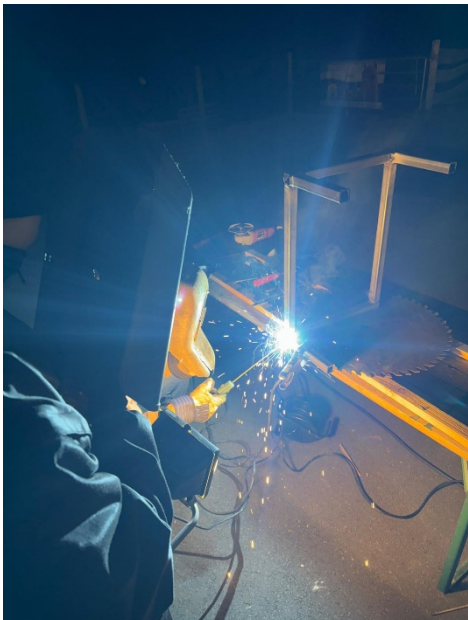


Abbildung 89:  
Elektrodenschweißen  
Quelle: eigene Abbildung



Abbildung 90:  
Elektrodenschweißen  
Quelle: eigene Abbildung



---

Nach dem der Rahmen fertig war, wurden die Querstreben verschweißt. Zuerst wurden die Querstreben für die Platte verschweißt, auf der die Elektronik platziert wird. Anschließend folgten die Querstreben für die Platte, auf der die Wägezelle befestigt wird.

### **Schritt 3: Montage der BLDC-Motoren**

In diesem Schritt erfolgt die Montage der BLDC-Motoren. Dabei ist zu beachten das die Motoren exakt ausgerichtet sind und sich während dem Betrieb nicht verdrehen oder lockern.

Um zu verhindern das sich die BLDC-Motoren verdrehen wurde auf beide Seiten eine Querstrebe verschweißt. Nachdem die Querstange montiert war, wurde für jeden Motor zwei Löcher gebohrt damit sie dort angeschraubt werden können. Die Befestigung erfolgte mithilfe von M4-Schrauben, die mit einer Mutter festgezogen wurden.



Abbildung 91:  
Motorbefestigung

Quelle: eigene Abbildung

Da BLDC-Motoren bei hohen Drehzahlen ein starkes Drehmoment erzeugen können, wurden sie zusätzlich mit Schellen am Boden befestigt. Diese verhindern, dass sich die Motoren während des Betriebs lockern.

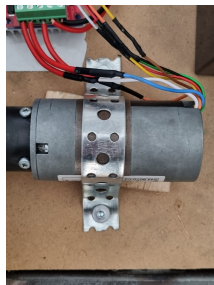


Abbildung 92:  
Motorbefestigung

Quelle: eigene  
Abbildung

---

#### Schritt 4: Montage der Räder

Nachdem die Motoren montiert waren, wurden die Räder angebracht. Es wurden aufblasbare Räder des Typs "WERKA PRO 10" 260x85" mit einer 16 mm Bohrung gewählt, weil die Achse der Motoren 16 mm beträgt. Die Reifen wurden auch gewählt, weil sie eine gute Bodenhaftung bieten und durch ihre Größe für ausreichend Bodenfreiheit sorgen.



Abbildung 93: WERKA PRO 10

Quelle: <https://amzn.eu/d/5EHdhJk>

Die Räder wurden direkt mit einer Schraube in die integrierte Gewindebohrung der Motorachse geschraubt.

Dabei ist aber ein Problem aufgetreten. Durch die mechanische Kraft, die beim Reiben der Räder auf dem Boden auf die Schrauben wirkt, lockerten sich die Schrauben immer wieder, sodass die Räder nicht mehr angetrieben wurden.

Der erste Lösungsversuch war, die Schrauben mit Schraubensicherungsmittel einzuschmieren und dann in die Achse des Motors zu schrauben. Dieser Versuch scheiterte jedoch, da die mechanische Kraft zu groß war, um eine stabile Verbindung zu gewährleisten.

Die endgültige Lösung ist, dass in jedem Reifen ein 3D-gedrucktes Verbindungselement eingesetzt wird, das als Verbindung zwischen Reifen und Motorachse dient. Dieses Verbindungselement hat in der Mitte ein 16 mm großes Loch, das für die Motorachse passt. Auf der Seite, die am Motor anliegt, befindet sich eine sechseckige Einkerbung, die genau zur Sechskantform der Motorachse passt. Da sich der Sechskant auf der Motorachse dreht, wird die Drehbewegung über das Verbindungselement direkt auf den Reifen übertragen. Zusätzlich wird eine M8-Schraube verwendet, um das Rad sicher an das Bauteil und den Motor zu ziehen.



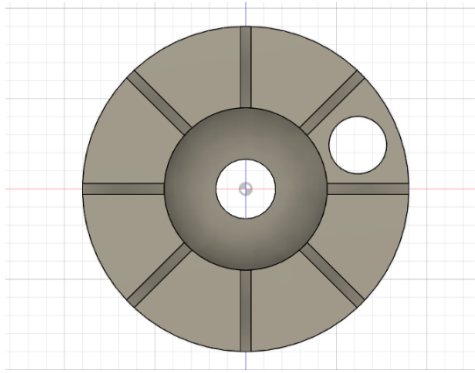


Abbildung 94:  
Verbindungselement  
Quelle: eigene Abbildung

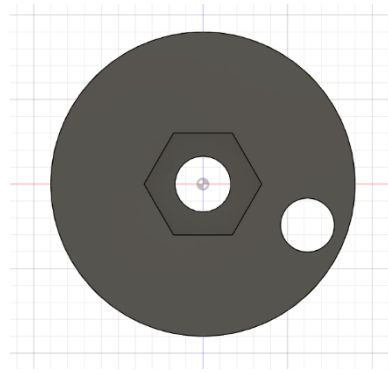


Abbildung 95:  
Verbindungselement  
Quelle: eigene Abbildung

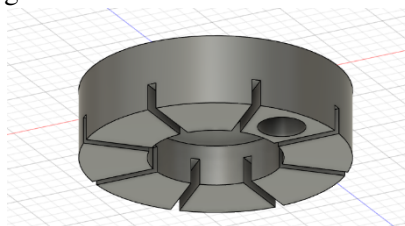


Abbildung 96:  
Verbindungselement  
Quelle: eigene Abbildung

## Schritt 5: Integration der Wägezelle

Die Wägezelle wurde zwischen dem Hauptrahmen und der oberen Plattform installiert.

- Die Platzierung wurde sorgfältig gewählt, um eine gleichmäßige Gewichtsverteilung sicherzustellen.
- Die Wägezelle wurde so montiert, dass sie möglichst direkt das gesamte Gewicht der Last erfasst, ohne dass zusätzliche Kräfte durch die Rahmenstruktur einwirken.

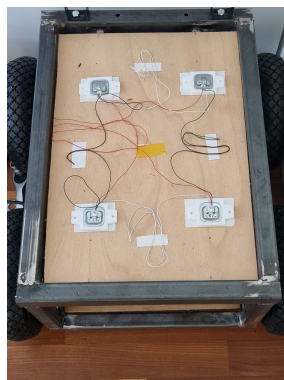


Abbildung 97: Integration  
der Wägezelle  
Quelle: eigene Abbildung

---

## Schritt 6: Elektronik-Integration

Nach der mechanischen Konstruktion wurde die Elektronik montiert. Dazu gehören:

- Steuerplatine
- Motortreiber für jeden BLDC-Motor
- 12V-Akku als Energiequelle
- Dino KRAFTPAKET 4A-6V/12V Ladegerät

Die elektronischen Komponenten wurden auf einer Holzplatte montiert. Die Holzplatte wurde zusätzlich mit zwei Querstreben verstärkt. Beim Platzieren der Komponenten wurde darauf geachtet, dass alle Bauteile sicher befestigt sind und ausreichend Platz für die Verkabelung bleibt. Zusätzlich wurden die Beleuchtungselemente montiert und in die Verkabelung integriert.

Bei der Verkabelung wurde darauf geachtet, dass alles sorgfältig verlegt wird. Die Kabel wurden in verschiedenen Farben gekennzeichnet. Die roten Kabel sind für 12 Volt und 5 Volt, die schwarzen Kabel sind die GND-Leitungen, und die gelben und weißen Kabel sind die Signalleitungen.

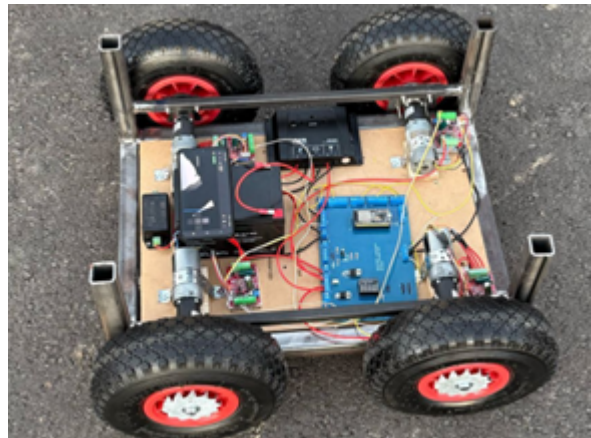


Abbildung 98: Erster funktionierender Prototyp

Quelle: eigene Abbildung

---

## 5.4 Materialliste

### 5.4.1 Stahlliste

| Element (Vierkant) | Länge       |
|--------------------|-------------|
| 30*40              | 2m          |
| 30*30              | 3.5m        |
| 25*25              | 1m          |
| 15*15              | 2m          |
| <b>Gesamtlänge</b> | <b>8.5m</b> |

### 5.4.2 Komponentenliste

| Komponente                          | Anzahl |
|-------------------------------------|--------|
| Aufblasbares Rad 10" 260x85         | 4x     |
| Motortreiber (ZS-X11H)              | 4x     |
| BLDC-Motoren                        | 4x     |
| Steuerplatine                       | 1x     |
| Halbbrücken Wägezelle               | 1x     |
| AGM 12V Batterie                    | 1x     |
| Dino KRAFTPAKET 4A-6V/12V Ladegerät | 1x     |

**Befestigungselemente:** Es wurden außerdem verschiedene Befestigungselemente verwendet, darunter Schrauben, Muttern, Schellen, Unterlegscheiben sowie Steckverbinder für die elektrische Verkabelung. Diese sind essenziell, um die mechanischen und elektrischen Komponenten sicher zu montieren und zu verbinden.

---

## 6 Hardwaredesign

In diesem Kapitel wird die Entwicklung und Herstellung der Steuerplatine sowie die Integration der externen Komponenten beschrieben. Alle Schritte vom ersten Aufbau auf einem Steckbrett bis hin zur fertigen Platine werden verständlich erläutert. Die Platine wurde mehrfach getestet, um mögliche Fehler frühzeitig zu erkennen und einen reibungslosen Betrieb sicherzustellen. Bei der Gestaltung wurde darauf geachtet, die Bauteile strukturiert und übersichtlich anzuordnen.

Zusätzlich zur Steuerplatine umfasst das System auch externe Komponenten wie die Batterie und den Dino KRAFTPAKET 4A-6V/12V Batterieladegerät. Diese wurden separat verbaut, sind jedoch nahtlos in das Gesamtsystem integriert, um eine stabile Energieversorgung, effizientes Laden und die notwendige Spannungsumwandlung zu gewährleisten. Diese externen Bauteile sind ebenfalls ein wichtiger Bestandteil des Hardwaredesigns, auch wenn sie nicht direkt auf der Steuerplatine untergebracht sind.

### 6.1 Zielsetzung

Das Ziel der Hardwareentwicklung besteht darin, eine Steuerplatine zu entwerfen, die alle erforderlichen Funktionen zur Steuerung der Komponenten übernimmt, sowie eine effiziente Energieversorgungslösung zu entwickeln, die die nötige Leistung für das System bereitstellt.

Die Steuerplatine soll unter anderem die Ansteuerung der vier Motortreiber, Sensoren und Lichter mithilfe eines ESP32 übernehmen. Zur Erweiterung der verfügbaren Pins wird ein Expander-Board eingesetzt. Die Platine soll mithilfe eines LM317-Spannungsreglers, die Spannung von 12 Volt auf 5 Volt übernehmen. Zusätzlich sollen Anschlüsse für GND, 5 Volt und 12 Volt für die externe Verkabelung bereitgestellt werden, um die Stromversorgung weiterer Komponenten außerhalb der Platine zu ermöglichen.

Für die Energieversorgung wird eine AGM 12V-Batterie verwendet, die den Betrieb des Systems sicherstellt. Das Dino KRAFTPAKET 4A-6V/12V Ladegerät übernimmt das Laden der Batterie und sorgt für eine geregelte Ladespannung, um eine sichere und effiziente Energieversorgung zu gewährleisten. Die Batterie versorgt direkt die angeschlossenen Verbraucher, einschließlich der Steuerungsplatine, mit 12V Gleichspannung. Dadurch wird sichergestellt, dass das gesamte System stabil und zuverlässig mit Energie betrieben wird.

---

## 6.2 Grundsaltungen

Bevor die Steuerplatine entwickelt wurde, wurden alle Bauteile und Schaltungen auf einem Steckbrett getestet. Dabei war es wichtig, Fehler vorzeitig zu erkennen und nicht erst dann, wenn die Platine fertig designt war. Nach der Überprüfung der einzelnen Komponenten wurden alle Schaltungen verbunden und getestet. Nachdem das erledigt war, wurde die Platine mithilfe von Eagle designt.

Die Elektronik für die Energieversorgung, die sich nicht auf der Platine befindet, wurde ebenfalls zuerst separat getestet. Nachdem alle Komponenten fehlerfrei funktionierten, wurden sie miteinander verbunden und getestet. Anschließend wurde die gesamte Elektronik im Lastenroboter verbaut.

## 6.2.1 Schaltplan Steuerplatine

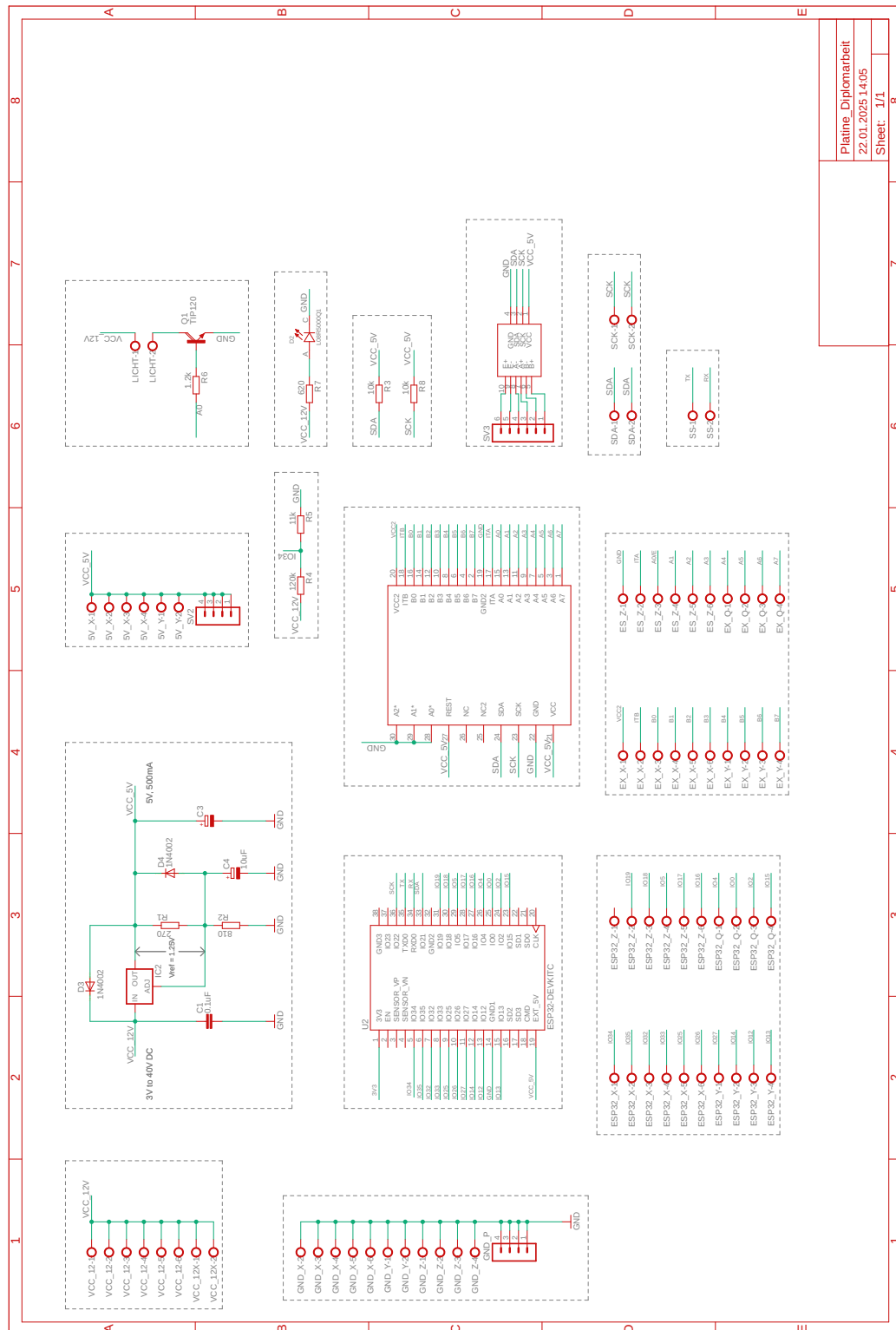


Abbildung 99: Schaltplan Steuerplatine

Quelle: eigene Abbildung

---

## Schaltungszusammenfassung

Das gezeigte Schaltbild stellt eine elektronische Schaltung dar, die verschiedene Komponenten wie einen ESP32-Mikrocontroller, einen MCP23017 I/O-Expander, eine Spannungsregelung sowie mehrere Anschlussmodule integriert. Die Schaltung beinhaltet verschiedene Spannungsversorgungen (12V, 5V) und ermöglicht eine Vielzahl von Funktionen, darunter Lichtsteuerung, Batterieüberwachung, Pull-up-Widerstände für I2C-Kommunikation sowie erweiterte Ein- und Ausgänge.

## Einzelne Komponenten und ihre Beschreibung:

### 1) VCC\_12V (12V Spannungsversorgung)

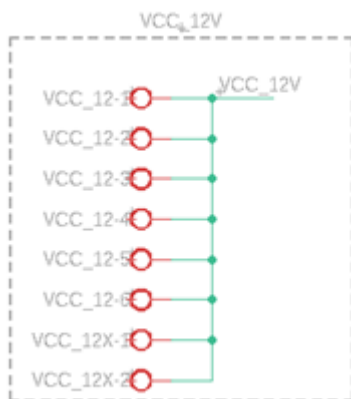


Abbildung 100: Schaltplan  
Spannungsversorgung  
Quelle: eigene Abbildung



Abbildung 101: Schraubklemmen  
Quelle: eigene Abbildung

Die Platine wird über eine Batterie mit 12V Gleichspannung versorgt, die an einer speziellen Schraubklemme angeschlossen wird. Diese 12V-Spannung dient als Hauptversorgung und wird direkt für bestimmte Komponenten genutzt, wie beispielsweise die Lichtsteuerung, die mit 12V arbeitet. Zusätzlich gibt es weitere sieben Schraubklemmen, über die externen Komponenten mit 12V versorgt werden können.

## 2) GND (Masseanschlüsse)



Abbildung 102: Schaltplan  
Masseanschlüsse

Quelle: eigene Abbildung



Abbildung 103: Pinheader

Quelle: eigene Abbildung

Die Platine verfügt über einer gemeinsamen Masse (GND). Alle Bauteile auf der Platine sind mit dieser gemeinsamen Masse verbunden. Für externe Komponenten stehen 11 Schraubklemmen zur Verfügung, darunter eine für die Batteriemasse (12V). Zudem gibt es 4 Pin-Header-Anschlüsse für weitere Verbindungen. Eine saubere Masseführung sorgt für stabile Spannungsverhältnisse und eine zuverlässige Funktion der Schaltung.

## 3) 5V-Regelung (Spannungswandlung von 12V auf 5V)

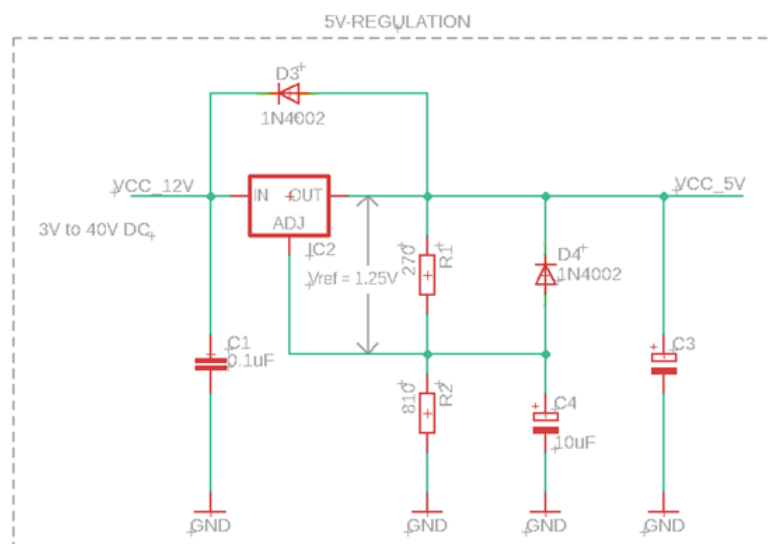


Abbildung 104: 5V-Spannungsregelung

Quelle: eigene Abbildung



---

Da viele der verwendeten Komponenten, insbesondere der ESP32-Mikrocontroller, mit einer Betriebsspannung von 5V arbeiten, wird die 12V-Spannung in eine stabile 5V-Versorgung umgewandelt. Dies geschieht mithilfe eines LM317-Spannungsreglers.

### Funktionsweise LM317-Spannungsregler:

Der LM317 ist ein einstellbarer Linearregler, der eine höhere Eingangsspannung (z. B. 12V) auf eine stabilisierte Ausgangsspannung (z. B. 5V) reduziert. Dies geschieht durch eine kontinuierliche Regelung der Ausgangsspannung mithilfe eines internen Verstärkers und einer Referenzspannung von 1,25V.

### Der LM317 besitzt drei Anschlüsse:

1. **Adjust:** Über diesen wird die Ausgangsspannung mit zwei Widerständen (**R1 & R2**) eingestellt.
2. **Vout:** Hier wird die geregelte Ausgangsspannung (5V) ausgegeben.
3. **Vin:** Hier wird die unregelte Eingangsspannung (12V) angelegt.

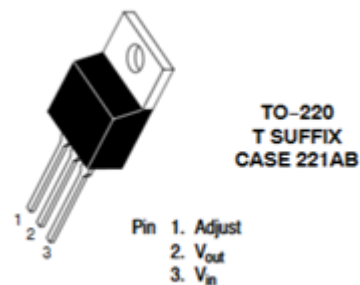


Abbildung 105: LM317

Quelle:

<https://www.alldatasheet.com/datasheet-pdf/view/611120/ONSEMI/LM317.html>

### Berechnung der Widerstände:

Mit dieser Formel kann man die Widerstände so bestimmen damit die gewünschte Ausgangsspannung erreicht wird:

$$V_{out} = 1.25V \times \left(1 + \frac{R2}{R1}\right) \quad (1)$$

Für die Platine wurde ein Spannungsregler entwickelt der 12 Volt auf 5 Volt herunterregelt. Das heißt Vout soll 5 Volt betragen. Damit man die Widerstände bestimmen kann, muss bei einem Widerstand ein Wert angenommen werden. In diesem Fall wurde für R1 der Wert von 270 Ohm festgelegt.

Formel mit den eingesetzten Werten:

$$5V = 1.25V \times \left(1 + \frac{R2}{270}\right) \quad (2)$$

Durch Umformung und Berechnen des Widerstandes R2 ergibt sich:

$$R2 = 810\Omega \quad (3)$$

In der Schaltung wurden daher die Widerstandswerte  $R1 = 270\Omega$  und  $R2 = 810\Omega$  gewählt, was eine Ausgangsspannung von 5V ergibt.

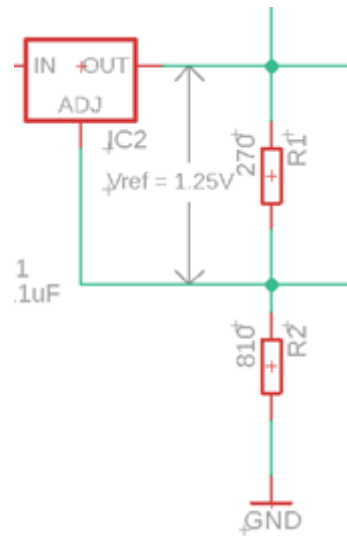


Abbildung 106: Spannungsregler  
Quelle: eigene Abbildung

#### Dioden (D3, D4):

- **D3:** Verhindert Rückströme in den Eingang des LM317, um Schäden zu vermeiden.
- **D4:** Schützt den Regler vor Spannungsrückkopplungen vom Ausgang.

#### Kondensatoren:

- **C1:** Stabilisiert die Eingangsspannung und filtert Störsignale.
- **C3 & C4:** Filtern Spannungsschwankungen am Ausgang und verbessern die Stabilität der 5V-Versorgung.

#### 4) VCC\_5V (5V Spannungsversorgung)

Die Platine liefert geregelte 5V-Spannung zur Versorgung der entsprechenden Komponenten auf der Platine. Für externe Verbindungen stehen 6 Schraubklemmen zur Verfügung, zusätzlich gibt es 4 Pin-Header-Anschlüsse für weitere Verbindungen.

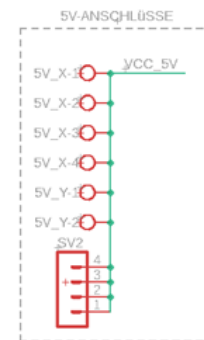


Abbildung 107: Schaltplan Spannungsversorgung

Quelle: eigene Abbildung

#### 5) ESP32

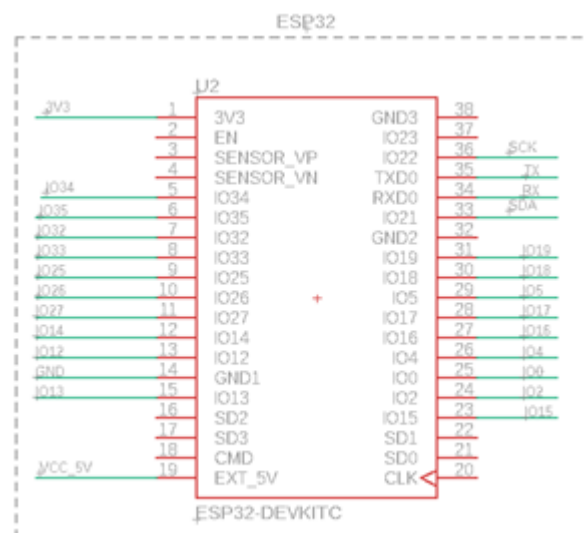


Abbildung 108: Schaltplan ESP32

Quelle: eigene Abbildung

Der ESP32 ist die wichtigste Komponente auf der Platine. Er übernimmt die Kommunikation zwischen den anderen Komponenten. Der ESP32 verfügt über Wi-Fi, wodurch er sich ideal für IoT-Anwendungen eignet.

Der ESP32 misst die Batteriespannung und liest die Daten des HX711 ein. Er verfügt auch über eine I2C-Schnittstelle, die für die Wägezelle und das Expander-Board verwendet wird. Zusätzlich steuert der ESP32 die PWM-Signale für die Motorentreiber, um die Drehzahl der Motoren präzise zu regulieren.

## 6. ESP32-Anschlüsse



Abbildung 109: ESP32-Anschlüsse

Quelle: eigene Abbildung

Um den Zugriff auf die GPIO-Pins des ESP32 zu erleichtern, werden sie mit Schraubklemmen herausgeführt. Das ermöglicht eine einfache Verbindung mit Sensoren und Aktoren, ohne dass direkt auf den Mikrocontroller zugegriffen werden muss.

## 7. MCP23017 (I/O-Expander mit I2C-Anbindung)

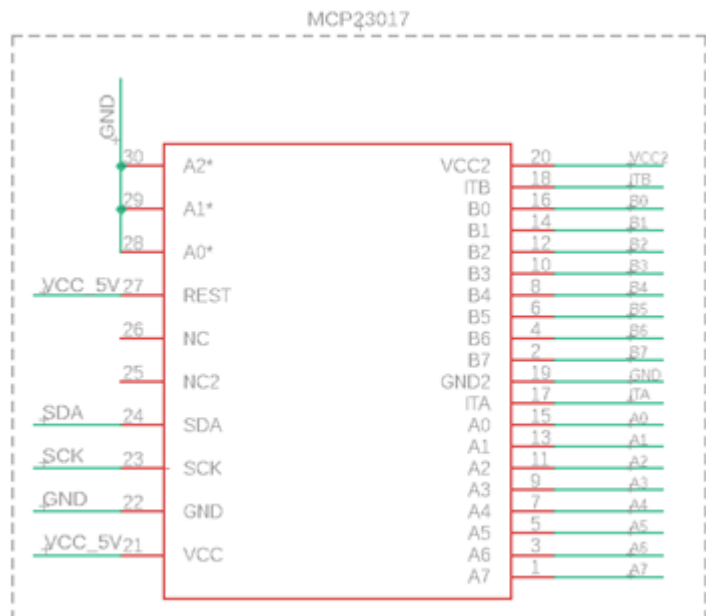


Abbildung 110: MCP23017

Quelle: eigene Abbildung

---

Der MCP23017 ist ein I/O-Expander, der über den I2C-Bus mit dem ESP32 kommuniziert. Da der ESP32 nur eine begrenzte Anzahl an GPIOs besitzt, bietet der MCP23017 eine Erweiterung um 16 zusätzliche digitale Ein- und Ausgänge. Diese zusätzlichen GPIOs steuern die Richtung und die Bremsfunktion der Motoren. Aber auch das Licht wird über das Expander-Board gesteuert.

Die Ausgänge des MCP23017 sind in zwei Gruppen unterteilt: A0–A7 und B0–B7.

### 6.2.2 8. Expander\_Anschlüsse (GPIO-Erweiterung über MCP23017)



Abbildung 111: expander-Anschlüsse

Quelle: eigene Abbildung

Die 16 digitalen I/O-Pins des MCP23017 werden über Schraubklemmen ausgegeben. Dies ermöglicht eine einfache Nutzung der zusätzlichen GPIOs, indem externe Module direkt an diese Schraubklemmen angeschlossen werden können.

## 9) Lichtsteuerung (Transistor-Schaltung zur Steuerung einer LED)

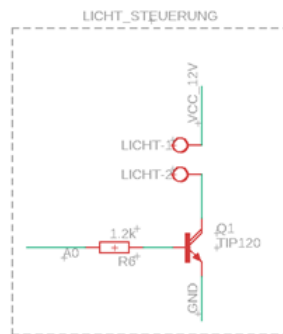


Abbildung 112: Lichtsteuerung

Quelle: eigene Abbildung

Diese Schaltung dient zur Steuerung der Scheinwerfer des Lastenroboters. Ein NPN-Leistungstransistor (TIP120) übernimmt das Schalten der Beleuchtung. Der Basiswiderstand R6 begrenzt den Basisstrom des Transistors. Die Ansteuerung des Transistors erfolgt über das Expander-Board.

### Berechnung des Basiswiderstands:

Für die Berechnung des Basisstrom braucht man die Formel:

$$I_b = \frac{I_c}{h_{FE}} \quad (4)$$

- $I_c$  ... Strom, den der Verbraucher benötigt
- $h_{FE}$  ... Stromverstärkung des Transistors

Die Scheinwerfer haben einen  $I_c$  von 1,5 A und der Transistor hat eine  $h_{FE}$  von 1000.

$$I_b = \frac{I_c}{h_{FE}} = \frac{1.5}{1000} = 1.5mA \quad (5)$$

Die Formel für den Basiswiderstand ist:

$$R_b = \frac{U_e - 0.7V}{I_b} \quad (6)$$

- $U_e$  ... Ausgangsspannung des Expander-Boards

Das Expander-Board hat eine Ausgangsspannung von 3,3 V.

$$R_b = \frac{U_e - 0.7V}{I_b} = \frac{3.3V - 0.7V}{1.5mA} = 1.73k\Omega \quad (7)$$

Gewählt wurde ein Widerstand von **1.7 kΩ**.<sup>3</sup>

<sup>3</sup><https://www.mikrocontroller.net/articles/Basiswiderstand>

## 10. Batteriemessung (Spannungsteiler zur Messung der Batteriespannung)

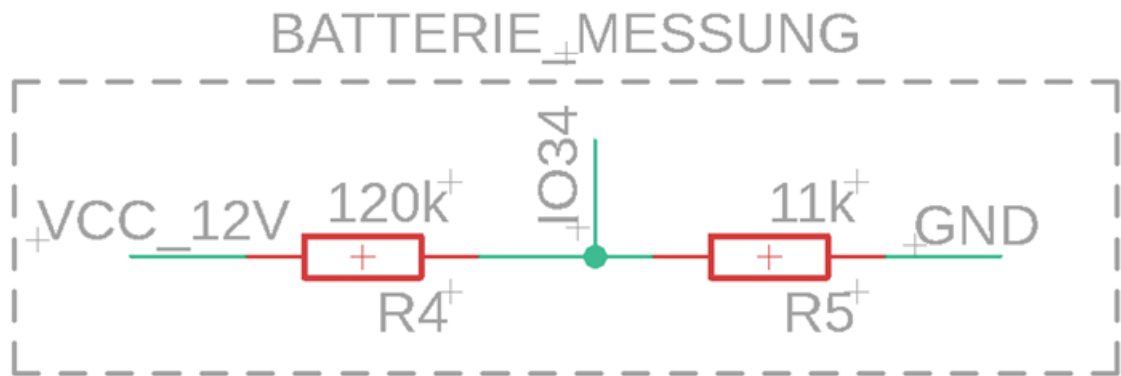


Abbildung 113: Batteriemessung

Quelle: eigene Abbildung

Um die Batteriespannung zu überwachen, kommt ein Spannungsteiler bestehend aus den Widerständen R6 und R7 zum Einsatz. Da der ESP32 nur Spannungen bis maximal 3.3V messen kann, reduziert der Spannungsteiler die 12V Batteriespannung auf einen für den Mikrocontroller sicheren Bereich.

Der ESP32 kann die Batteriespannung über seinen IO34 auslesen.

## 11) Power-LED

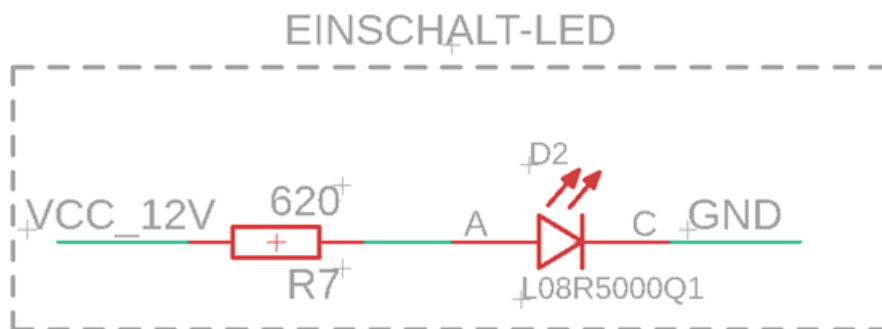


Abbildung 114: Power-LED

Quelle: eigene Abbildung

Diese Schaltung dient dazu, anzuzeigen, dass die Platine mit 12 Volt versorgt wird. Sobald die 12V-Spannung anliegt, leuchtet die Power-LED auf. Ein Vorwiderstand begrenzt den Strom durch die LED, um eine sichere und langlebige Funktion zu gewährleisten.

## 12) Pull-Up-Widerstände für I2C (Widerstände zur Stabilisierung des I2C-Busses)

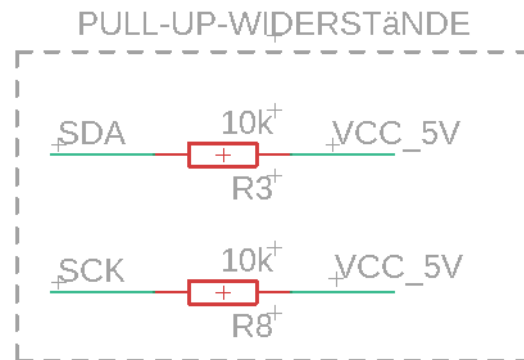


Abbildung 115: pullup-widerstände

Quelle: eigene Abbildung

Da der I2C-Bus eine aktive Pegelsteuerung benötigt, kommen Pull-Up-Widerstände mit 10 kOhm zum Einsatz. Diese Widerstände sind zwischen den SDA/SCL-Leitungen und der Versorgungsspannung (5V) geschaltet und sorgen für eine stabile Signalübertragung. Ohne diese Widerstände könnten Kommunikationsfehler auftreten, insbesondere bei längeren I2C-Leitungen.

## 13) HX711 (Wägezellen)

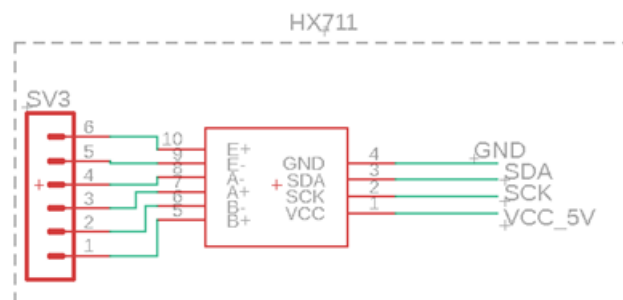


Abbildung 116: schaltplan-wägezellen

Quelle: eigene Abbildung

Der HX711 ist ein spezialisierter IC, der für die Messung von Gewichtssensoren (Wägezellen) entwickelt wurde. Er verstärkt und digitalisiert das analoge Ausgangssignal der Wägezelle, sodass der ESP32 präzise Gewichts- oder Kraftmessungen durchführen kann.

Die Kommunikation mit dem HX711 erfolgt über den I2C-Bus (SDA und SCL).



#### 14) I2C-Anschlüsse (Anschlussleiste für I2C-Kommunikation)



Abbildung 117: i2c-anschlüsse

Quelle: eigene Abbildung

Damit externe I2C-Geräte einfach angeschlossen werden können, gibt es eine eigene Schraubklemme für das I2C-Signale. Es stehen zwei SDA und zwei SCL Schraubklemmen zu Verfügung.

#### 15) RX/TX (Serielle Schnittstelle für UART-Kommunikation)

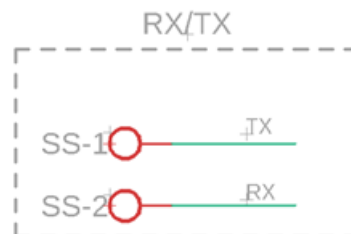


Abbildung 118: RX/TX

Quelle: eigene Abbildung

Die Platine verfügt über eine TX-Schraubklemme und eine RX-Schraubklemme, die für die serielle Kommunikation verwendet werden. Diese Anschlüsse ermöglichen den Datenaustausch zwischen der Platine und externen Geräten.

## 6.3 Steuerplatine PCB

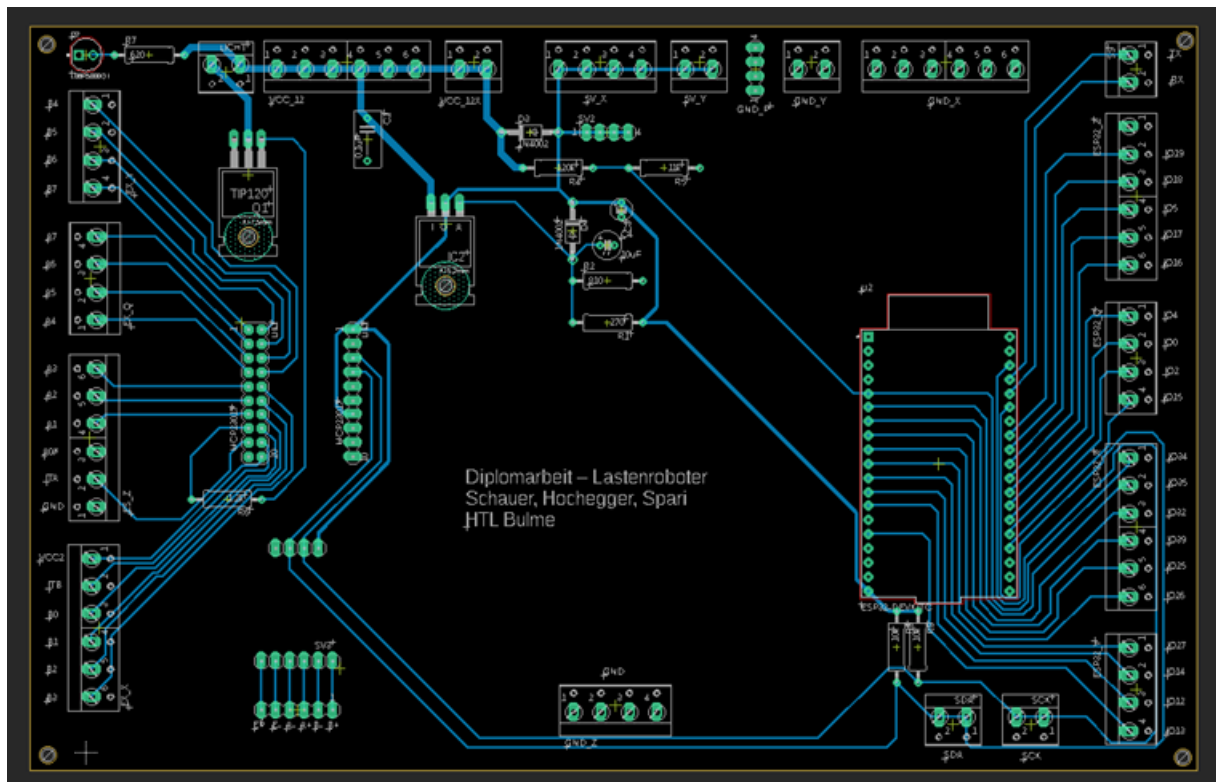


Abbildung 119: PCB

Quelle: eigene Abbildung

Für die Steuerplatine wurde eine einseitige Leiterplatte entwickelt, um eine einfache und kosteneffiziente Fertigung zu gewährleisten. Die Leiterbahnen sind entsprechend der erforderlichen Stromstärken dimensioniert, insbesondere in den Bereichen der 12V-Versorgung, um Spannungsabfälle und übermäßige Erwärmung zu vermeiden. Dabei wurde darauf geachtet, dass die Bahnen für höhere Ströme ausreichend breit ausgeführt sind.

Die Platine verfügt über mehrere beschriftete Anschlussklemmen, um externe Sensoren, Aktoren und weitere elektronische Bauteile sicher und übersichtlich integrieren zu können. Durch die klare Kennzeichnung der Klemmen wird die Verkabelung vereinfacht, was den Aufbau und die Wartung erleichtert.

Zusätzlich wurden in jeder Ecke der Platine Befestigungslöcher vorgesehen, um eine stabile Montage innerhalb des Lastenroboters zu ermöglichen. Diese sorgen dafür, dass die Platine sicher fixiert ist und während des Betriebs nicht verrutscht oder beschädigt wird.

## 6.4 Leiterplatten herstellen mit JLCPCB

JLCPCB ist ein weltweit führender Anbieter für die Herstellung von Leiterplatten und bietet eine benutzerfreundliche Online-Plattform zur individuellen Fertigung von PCBs. Kunden können ihre eigenen Designs hochladen, verschiedene Produktionsoptionen wählen und die Platinen nach ihren Anforderungen konfigurieren. Für unsere Diplomarbeit haben wir uns für JLCPCB entschieden, da der Hersteller eine schnelle, zuverlässige und kostengünstige Lösung für die Fertigung unserer Platinen bietet. Im Vergleich zu anderen Anbietern ermöglicht JLCPCB eine kurze Produktionszeit, sodass die bestellten Leiterplatten bereits nach wenigen Tagen versandfertig sind. Zudem sind die Fertigungskosten im Vergleich zu vielen europäischen Herstellern deutlich günstiger, ohne dabei Einbußen in der Qualität hinnehmen zu müssen.

## 6.5 Bestückte Platine

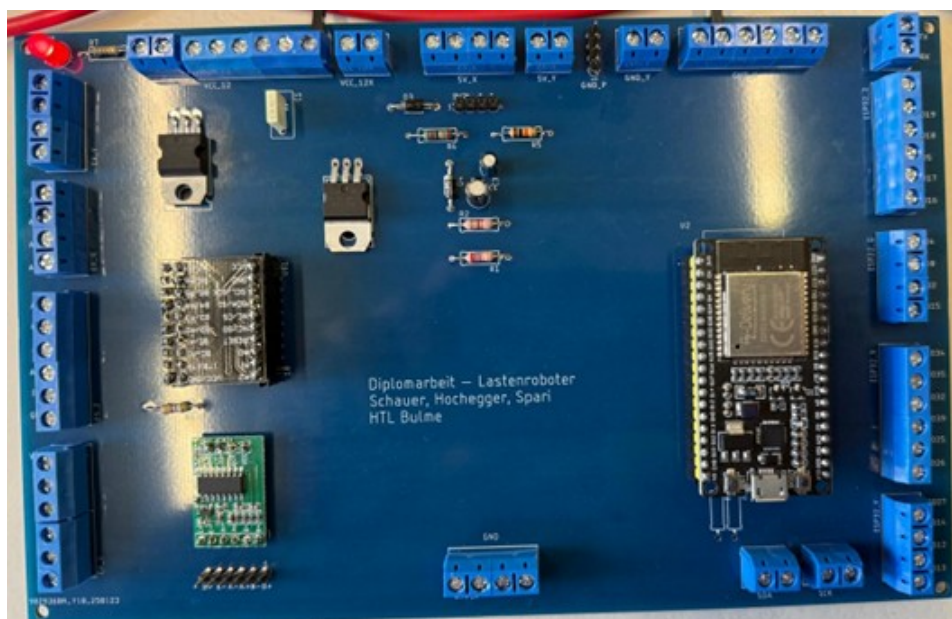


Abbildung 120: Steuerungsplatine

Quelle: eigene Abbildung

Nach der Fertigung der Platinen wurden alle erforderlichen Bauteile darauf montiert und verlötet, um die Schaltung funktionsfähig zu machen. Dabei wurde besonders darauf geachtet, die Bauteile präzise zu platzieren und richtig auszurichten, um eine einwandfreie Funktion zu gewährleisten. Nach dem Lötvorgang erfolgte eine gründliche Kontrolle, bei der alle Verbindungen überprüft und die Platinen getestet wurden, um mögliche Fehler frühzeitig zu erkennen und zu beheben.

---

# 7 Kamera

## 7.1 Kamera im Überblick

Die ESP32-CAM ist ein kompaktes Kameramodul, das auf dem ESP32-Mikrocontroller basiert. Dieses Modul kombiniert WLAN- und Bluetooth-Funktionen, wodurch die Kamera Bilder und Videos drahtlos übertragen kann. Das Grundprinzip der ESP32-Kamera besteht darin, dass der ESP32-Chip Bilddaten vom Kamerasensor verarbeitet und über eine drahtlose Verbindung an ein Endgerät, in diesem Fall ein Webserver, sendet.

Bei der ESP32-CAM wird häufig die OV2640-Kamera verwendet, die eine Auflösung von bis zu  $1600 \times 1200$  Pixeln bietet. Das Modul verfügt zudem über einen microSD-Kartenslot zur Speicherung von Aufnahmen.

Das Modul kann so programmiert werden, dass es als eigenständiger Webserver agiert, der einen Videostream bereitstellt. Über eine IP-Adresse kann auf den Stream zugegriffen werden, wodurch eine Echtzeitübertragung entsteht.

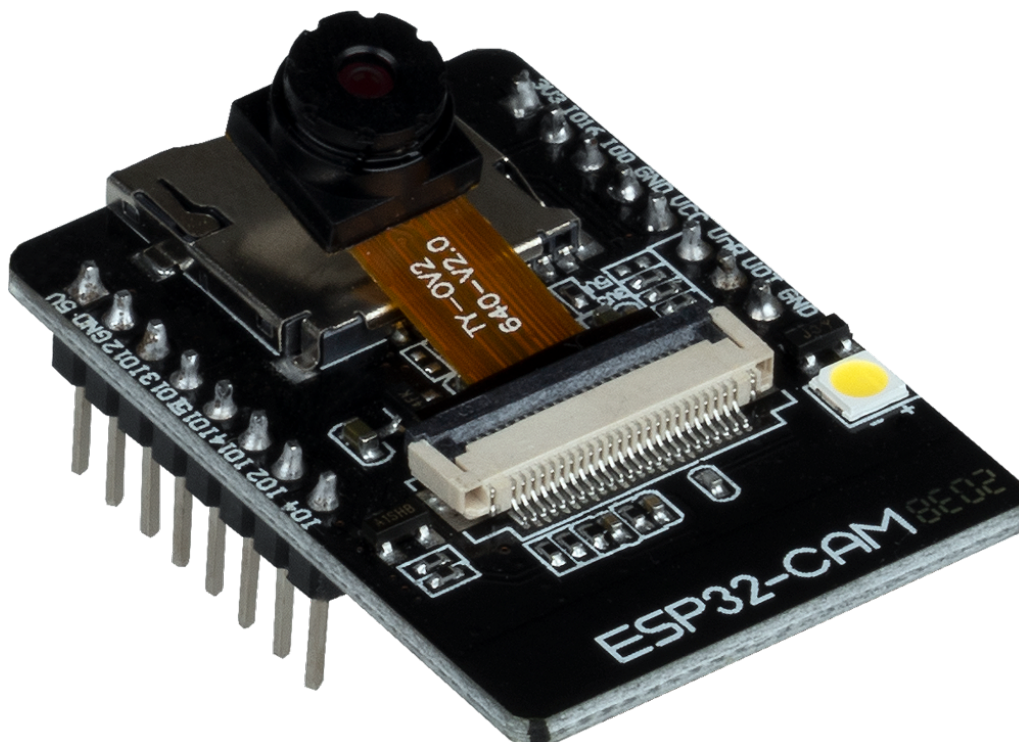


Abbildung 121: ESP32-CAM Modul

Quelle: <https://joy-it.net/de/products/SBC-ESP32-Cam>

---

### 7.1.1 Technische Daten

In der folgenden Tabelle sind die wichtigsten technischen Daten der ESP32-Kamera aufgeführt.

| Eigenschaft            | Details                                                 |
|------------------------|---------------------------------------------------------|
| Kamera                 | 2MP OV2640                                              |
| Bildformate            | JPEG, BMP, Grayscale                                    |
| Prozessor              | Dual Core-ESP32 160 MHz                                 |
| Special Features       | Helle Blitz-LED, integrierte Antenne                    |
| Schnittstellen         | UART, SPI, I2C, PWM, Wi-Fi, BT-Funkverbindung           |
| SD-Kartenslot          | Bis zu 4 GB                                             |
| SPI Flash              | 32 Mbit                                                 |
| RAM                    | 52KB SRAM + 4MB PSRAM                                   |
| Versorgungsspannung    | 5 V                                                     |
| Betriebstemperatur     | -20°C 85°C                                              |
| UART Standard Baudrate | 115200 bps                                              |
| Stromverbrauch         | 6mA im Tiefschlaf; 310 mA mit Blitz auf max. Helligkeit |
| Verschlüsselung        | WPA, WPA2, WPA2-Enterprise, WPS                         |

Quelle: <https://joy-it.net/de/products/SBC-ESP32-Cam>

### 7.1.2 Nutzung

Der ESP32 wird als Webserver verwendet, auf dem der Kamerastream gehostet wird. Über eine zugewiesene IP-Adresse kann direkt auf den Webserver zugegriffen werden, wo der Livestream angezeigt wird.

Die Videoübertragung dient dazu, den Roboter über größere Entfernungen autonom steuern zu können. Mit der Videoübertragung werden Objekte und Hindernisse erkennbar und kann diese somit ausweichen.

Die Videoübertragung wird über der ganzen Website im Hintergrund angezeigt.



Abbildung 122: Videoübertragung auf der Website

Quelle: eigene Abbildung

## 7.2 Code

### 7.2.1 Kamera Setup

Um die Kamera programmieren zu können, muss zunächst das richtige Board `AI Thinker ESP32-CAM` ausgewählt werden. Danach müssen die ESP32-Kamera-Treiber mit der Bibliothek `esp_camera.h` inkludiert werden. Dann muss das passende ESP32-CAM-Modell festgelegt werden. Da wir uns für das `Ai-Thinker` Modell entschieden haben, muss diese nun definiert werden. Falls ein anderes Modell genutzt wird, muss es entsprechend angepasst werden.

Als nächstes werden die GPIO-Pins der ESP32-CAM für die Kamera OV2640 konfiguriert. Diese Zuordnung ist spezifisch für das `Ai-Thinker`-Modell und muss für jedes Modell individuell angepasst werden.

```

#include "esp_camera.h"

#define CAMERA_MODEL_AI_THINKER

#if defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM 32 // Kamera-Power-Down
#define RESET_GPIO_NUM -1 // Reset-Pin
#define XCLK_GPIO_NUM 0 // Externer Taktgeber
#define SIOD_GPIO_NUM 26 // I2C-Daten (SDA)
#define SIOC_GPIO_NUM 27 // I2C-Takt (SCL)

#define Y9_GPIO_NUM 35 // Kamera-Datenbit 9
#define Y8_GPIO_NUM 34 // Kamera-Datenbit 8
#define Y7_GPIO_NUM 39 // Kamera-Datenbit 7
#define Y6_GPIO_NUM 36 // Kamera-Datenbit 6
#define Y5_GPIO_NUM 21 // Kamera-Datenbit 5
#define Y4_GPIO_NUM 19 // Kamera-Datenbit 4
#define Y3_GPIO_NUM 18 // Kamera-Datenbit 3
#define Y2_GPIO_NUM 5 // Kamera-Datenbit 2
#define VSYNC_GPIO_NUM 25 // Vertikale Synchronisation
#define HREF_GPIO_NUM 23 // Horizontale Synchronisation
#define PCLK_GPIO_NUM 22 // Pixeltakt
#endif

```

Abbildung 123: Kamera-GPIO Konfiguration

Quelle: eigene Abbildung

Als nächstes muss die ESP32-CAM mit der ESP-IDF `esp_camera` Bibliothek konfiguriert und initialisiert werden. Als erstes muss mit `camera_config_t` eine Struktur definiert werden, mit der verschiedene Parameter und Eigenschaften wie GPIO-Pins, Bildgröße und Qualität festgelegt werden können.

LEDC-Kanal und Timer werden für das Taktsignal benötigt, um die Kamera zu betreiben.

Zum Schluss wird die Kamera mit den konfigurierten Einstellungen initialisiert. Falls bei der Initialisierung ein Fehler auftreten soll, wird dieser in der seriellen Konsole ausgegeben und das Setup abgebrochen.



```

camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;           // Kamera-Datenbit 2
config.pin_d1 = Y3_GPIO_NUM;           // Kamera-Datenbit 3
config.pin_d2 = Y4_GPIO_NUM;           // Kamera-Datenbit 4
config.pin_d3 = Y5_GPIO_NUM;           // Kamera-Datenbit 5
config.pin_d4 = Y6_GPIO_NUM;           // Kamera-Datenbit 6
config.pin_d5 = Y7_GPIO_NUM;           // Kamera-Datenbit 7
config.pin_d6 = Y8_GPIO_NUM;           // Kamera-Datenbit 8
config.pin_d7 = Y9_GPIO_NUM;           // Kamera-Datenbit 9
config.pin_xclk = XCLK_GPIO_NUM;       // Externer Taktgeber (24 MHz)
config.pin_pclk = PCLK_GPIO_NUM;       // Pixeltakt
config.pin_vsync = VSYNC_GPIO_NUM;     // Vertikale Synchronisation
config.pin_href = HREF_GPIO_NUM;       // Horizontale Synchronisation
config.pin_sccb_sda = SIOD_GPIO_NUM;    // I2C-Datenleitung (SDA)
config.pin_sccb_scl = SIOC_GPIO_NUM;    // I2C-Taktleitung (SCL)
config.pin_pwdn = PWDN_GPIO_NUM;       // Power-Down
config.pin_reset = RESET_GPIO_NUM;     // Reset-Pin
config.xclk_freq_hz = 24000000;        // Taktfrequenz
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_QVGA;
config.jpeg_quality = 12;
config.fb_count = 3;

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Kamera-Init fehlgeschlagen mit Fehler 0x%x", err);
    return;
}

```

Abbildung 124: Kamera-Initialisierung

Quelle: eigene Abbildung

### 7.2.2 Videoübertragung

In unserem Projekt werden die Live-Bilder per WebSocket an den Webserver gesendet. Um dies umzusetzen, wird zuerst eine Webserver-Route benötigt. Dazu wird eine http-GET-Anfrage für die Hauptseite (“/“) definiert. Im Code wird ein JavaScript Skript benutzt, um eine WebSocket-Verbindung herzustellen. Die IP-Adresse wird automatisch durch `windows.location.hostname` erkannt. Port 81 wird für das WebSocket-Streaming verwendet. Wenn die ESP32-CAM ein neues Bild als WebSocket-Nachricht versendet, wird es als JPEG-Blob gespeichert. Danach wird ein temporär URL-Link erstellt. Das Bild wird schlussendlich in einem `<img>`-Tag mit der id `stream` angezeigt.

Die WebSocket Verbindung wird die ganze Zeit überwacht. In der Konsole wird ausgegeben, wenn



die WebSocket-Verbindung aktiv ist. Falls die Verbindung abbrechen sollte, wird nach 5 Sekunden automatisch ein erneuter Verbindungsversuch gestartet. Zum Schluss wird der HTML-Code mit HTTP-Status 200 (OK) an den Browser gesendet.

```
server.on("/", HTTP_GET, [(AsyncWebServerRequest *request) {
  String html = R"rawliteral(
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>ESP32-CAM WebSocket Stream</title>
      <script>
        let websocket;
        function connectWebSocket() {
          websocket = new WebSocket('ws://' + window.location.hostname + ':81');

          websocket.onmessage = function(event) {
            const blob = new Blob([event.data], { type: 'image/jpeg' });
            const url = URL.createObjectURL(blob);
            const img = document.getElementById('stream');
            img.src = url;
          };

          websocket.onopen = function() {
            console.log('WebSocket verbunden');
          };

          websocket.onclose = function() {
            console.log('WebSocket getrennt. Erneuter Versuch in 5 Sekunden...');
            setTimeout(connectWebSocket, 5000);
          };
        }
        connectWebSocket();
      </script>
    </head>
    <body>
      <h1>ESP32-CAM WebSocket Stream</h1>
      <img id="stream" alt="Live Stream" style="width: 100%; max-width: 640px;" />
    </body>
  </html>
)rawliteral";
  request->send(100, "text/html", html);
});

server.begin();
```

Abbildung 125: Halterung der Kamera

Quelle: eigene Abbildung

Im HTML-Code wird die Videoübertragung als HTML <img>-Tag mit der id dynamicimage im Hauptbereich definiert.

```
<img id="dynamicimage" alt="Video Stream" />
```

Abbildung 126: Videoübertragung HTML-Code

Quelle: eigene Abbildung

Im CSS-Code wird das Element mit der id dynamicimage formatiert. Es wird definiert, dass die Größe 100% der Breite sowie 100% der Höhe einnimmt. Die Position wird auf absolut gesetzt.

```
#dynamicimage {
  width: 100%;
  height: 100%;
  position: absolute;
}
```

Abbildung 127: CSS-Formatierung  
für dynamic image

Quelle: eigene Abbildung

Im JavaScript-Code wird die WebSocket-Verbindung zur ESP32-CAM gehandelt. In der Funktion `connectWebSocket_cam()` wird zuerst ein WebSocket mit der eigenen IP-Adresse (da Webserver auf ESP32-CAM gehostet wird) und dem Port 81 geöffnet. Wenn nun eine Nachricht auf der WebSoccke-Verbindung ankommt, wird diese extrahiert. Das mitgesendete BLOB wird in einem neuem BLOB als jpeg-Bild gespeichert. Danach wird eine URL (Adresse) erstellt, die zu diesem BLOB führt. Dann wird eine Variable für die Videoübertragung im HTML `<img>`-Tag erstellt. Die Quelle dieses Bildes wird dann mit dem neuen, erhaltenen Bild ersetzt. Da die ESP32-CAM die ganze Zeit neue Blobs sendet, wird das Bild die ganze Zeit ersetzt, was dazu führt, dass es aussieht, als wäre es ein Video.

Wenn sich die ESP32-CAM verbindet, wird zu Debug-Zwecken eine Nachricht in der Webkonsole ausgegeben. Wenn die Verbindung zur ESP32-CAM verloren geht, wird wieder zu Debug-Zwecke eine Nachricht in der Webkonsole ausgegeben und ein neuer Verbindungsversuch wird nach 5 Sekunden gestartet.

```
let websocket_cam;

function connectWebSocket_cam() {
  websocket_cam = new WebSocket('ws://' + window.location.hostname + ':81'); // WebSocket-Verbindung zur ESP32 CAM

  websocket_cam.onmessage = function (event) {
    const blob = new Blob([event.data], { type: 'image/jpeg' });
    const url = URL.createObjectURL(blob);
    const img = document.getElementById('dynamicimage');
    img.src = url;
  };

  websocket_cam.onopen = function () {
    console.log('WebSocket cam verbunden');
  };

  websocket_cam.onclose = function () {
    console.log('WebSocket Cam getrennt. Erneuter Versuch in 5 Sekunden...');
    setTimeout(connectWebSocket_cam, 5000); // Versuchen, die Verbindung erneut herzustellen
  };
}

connectWebSocket_cam();
```

Abbildung 128: Videoübertragung im JavaScript-Code

Quelle: eigene Abbildung

---

## 7.3 Schwenkmechanismus der Kamera

### 7.3.1 Servomotor

Der SG90 ist ein kompakter Servomotor, der häufig in Modellbauprojekten eingesetzt wird. Aufgrund seiner geringen Größe und seines leichten Gewichts ist der Servomotor ideal für Anwendungen geeignet, die präzise Bewegungen erfordern.

Seine definierte Winkelsteuerung ermöglicht eine exakte Positionierung. Über den ESP32 kann der Motor so programmiert werden, dass er die Kamera auf eine gewünschte Position ausrichtet.

Der SG90 ist ein per PWM-Signal gesteuerter Servomotor, dessen Position durch einen Winkel zwischen  $0^\circ$  und  $180^\circ$  festgelegt wird. Zusätzlich gibt es Servomotoren mit einem Drehbereich von  $360^\circ$ , die in der Richtung als auch in der Drehgeschwindigkeit gesteuert werden können.

**Die Anschlussbelegung des SG90-Servomotors ist wie folgt:**

- **Rot:** VCC (ca. 5 Volt)
- **Orange:** PWM-Signal
- **Braun:** GND (0 Volt)

Die Betriebsspannung des SG90 liegt zwischen 4,8 und 6,0 Volt. Der Stromverbrauch variiert je nach Belastung zwischen 0,5 und 2,0 Ampere.

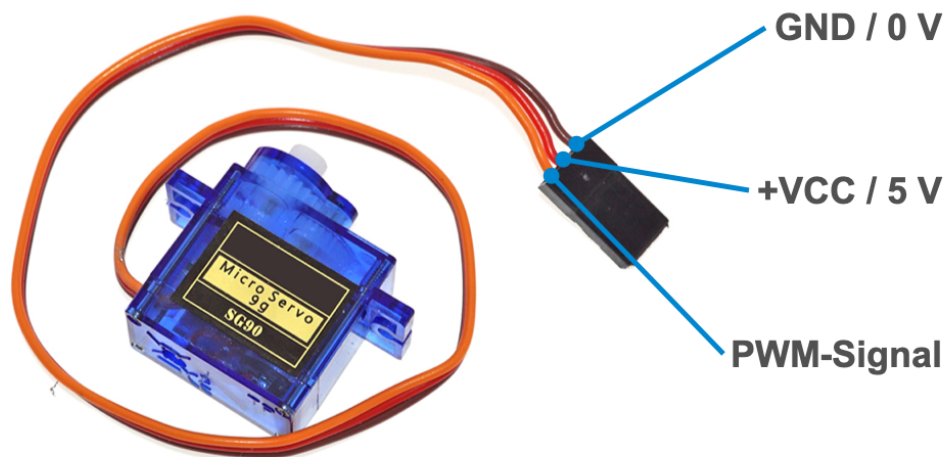


Abbildung 129: SG90-Servomotor

Quelle: [https://www.elektronik-kompandium.de/sites/praxis/bauteil\\_sg90.htm](https://www.elektronik-kompandium.de/sites/praxis/bauteil_sg90.htm)

### 7.3.2 Nutzung

Der SG90-Servomotor wird genutzt, um die Kamera an der Front des Roboters zu steuern und horizontal von links nach rechts zu schwenken. Dadurch wird das Sichtfeld vergrößert, was eine bessere Übersicht über die Umgebung ermöglicht.

Diese Funktion ist besonders hilfreich, da der Roboter dadurch präziser gesteuert und manövriert werden kann ohne direkten Sichtkontakt zu haben.

Außerdem lassen sich durch das erweiterte Blickfeld Hindernisse besser erkennen und Ausweichmanöver gezielt durchführen.

### 7.3.3 Verbindung und Steuerung des Servomotors

Der Servomotor wurde mit dem ESP32 folgend verbunden:

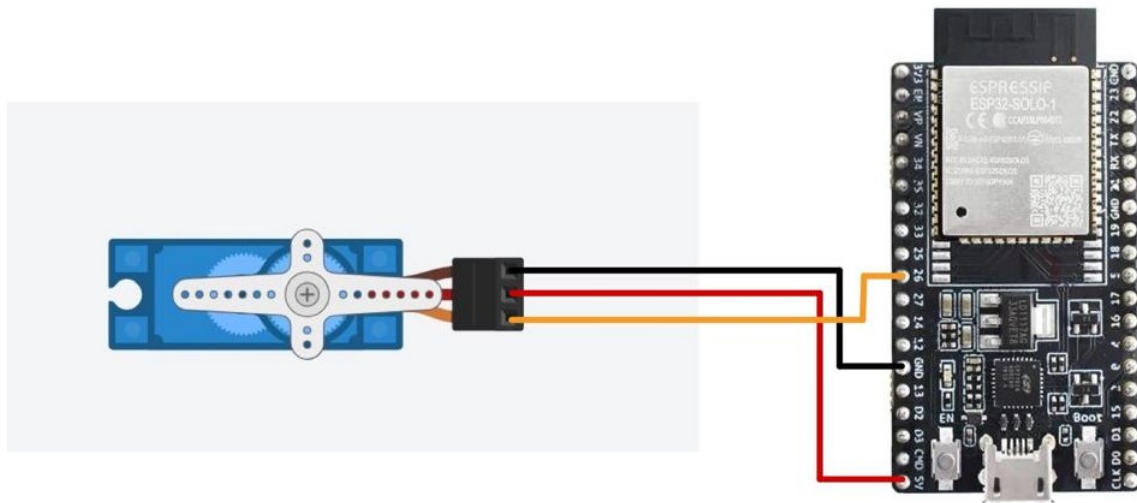


Abbildung 130: Verkabelung des Servomotors

Quelle: eigene Abbildung

Um den Servomotor einfach anzusteuern, wird die ESP32Servo.h Bibliothek eingebunden. Durch diese Bibliothek kann man Servomotoren per ESP32 ähnlich wie mit einem Arduino ansteuern.

```
#include <ESP32Servo.h>
```

Abbildung 131: ESP32Servo-Bibliothek

Quelle: eigene Abbildung

Danach wird die Variable `camera_pos` definiert, die die aktuelle Position des Servomotors speichert. Dann wird ein Objekt der Klasse `Servo` namens `myservo` erstellt, darüber kann man den

---

Servomotor später ansteuern. Der Pin, an den der Servo angeschlossen ist, wird als servoPin mit dem Wert 26 festgelegt. Durch die Verwendung von static const bleibt dieser Wert konstant und kann während der Laufzeit nicht geändert werden.

```
// Servo Motor
int camera_pos = 90;
Servo myservo;
static const int servoPin = 26;
//-----
```

Abbildung 132: Variablen für Servo

Quelle: eigene Abbildung

In der Setup()-Funktion wird das Servo-Objekt myservo mit dem zuvor definierten GPIO-Pin servoPin verbunden, dass ermöglicht die Steuerung des Servomotors über diesen Pin. Danach wird mit myservo.write(camera\_pos); der Servo in die Position bewegt, die in der Variable camera\_pos gespeichert ist. Diese Position wird in Grad angegeben, wobei die Werte zwischen 0° und 180° liegen. Am Anfang wird der Servomotor auf 90° gedreht, sodass er sich mittig im Bewegungsradius befindet.

```
myservo.attach(servoPin);
myservo.write(camera_pos);
```

Abbildung 133: Setup() ; -Code für Servo

Quelle: eigene Abbildung

Wenn der Slider auf der Website bewegt wird, sendet die Anwendung eine Nachricht in Form eines JSON-Objekts an den ESP32, dass die neue Position des Sliders enthält. Sobald der ESP32 eine solche Nachricht empfängt, wird die Funktion handleWebSocketMessage() aufgerufen. Innerhalb dieser Funktion wird überprüft, ob das JSON-Objekt jsonDoc den Schlüssel "camera\_position" enthält. Ist dies der Fall, wird der zugehörige Wert als Zeichenkette (const char \*camera\_position) extrahiert. Danach erfolgt eine Prüfung auf Gültigkeit des Wertes, bevor der Wert mit atoi() in eine Ganzzahl umgewandelt und in der Variable camera\_pos gespeichert wird. Am Ende wird die Funktion cam\_turn(); aufgerufen, um die Kamera entsprechend der neuen Position zu bewegen.

```

else if (jsonDoc.containsKey("camera_position"))
{
    const char *camera_position = jsonDoc["camera_position"];
    if (camera_position)
    {
        camera_pos = atoi(camera_position);
        cam_turn();
    }
}

```

Abbildung 134: camerapos

Quelle: eigene Abbildung

In der Funktion `cam_turn()` wird die Kamera entsprechend der neuen Position ausgerichtet. Zunächst gibt `Serial.println("Cam_Turn");` eine Nachricht im seriellen Monitor aus, um anzuzeigen, dass die Funktion ausgeführt wurde. Danach wird mit `myservo.write(camera_pos);` der Servomotor in die gewünschte Position bewegt, wobei `camera_pos` die zuvor gespeicherte Position enthält. Abschließend wird der Wert von `camera_pos` mit `Serial.println(String(camera_pos));` im seriellen Monitor ausgegeben, um die aktuelle Position des Servos zur Überprüfung anzuzeigen.

```

void cam_turn()
{
    Serial.println("Cam_Turn");
    myservo.write(camera_pos);
    Serial.println(String(camera_pos));
}

```

Abbildung 135: Funktion zum Kamera drehen

Quelle: eigene Abbildung

### 7.3.4 Gehäuse der Keraschwenkung

Das Gehäuse für die Keraschwenkung wurde so konstruiert, dass es eine robuste und sichere Halterung für eine Kamera bietet. Es besteht aus mehreren Bauteilen, die eine einfache Montage und Demontage ermöglichen.

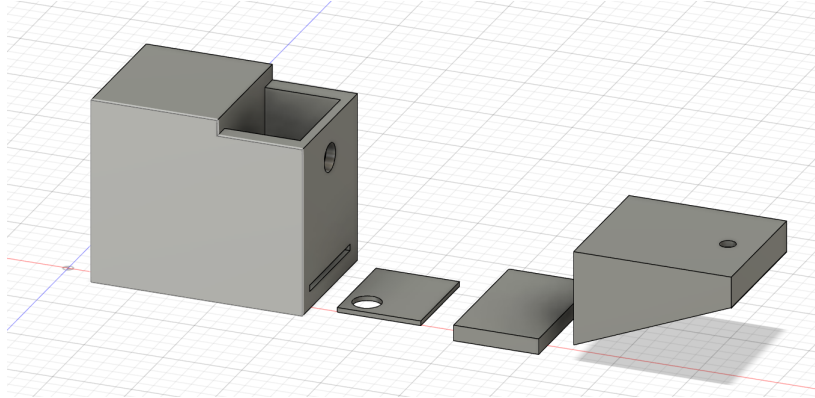


Abbildung 136: Kameragehäuse

Quelle: eigene Abbildung

#### Aufbau und Funktionen:

- **Hauptgehäuse:** Der größte Bauteil bildet das Grundgerüst der Halterung. Es ist als stabiler Kasten ausgeführt, der die Kamera aufnimmt und schützt. Eine Aussparung an der Oberseite ermöglicht die einfache Installation und Fixierung der Kamera. Nachdem die Kamera platziert ist, kann die Öffnung mit einer Platte geschlossen werden.
- **Kabeldurchführung:** Eine separate Abdeckplatte mit einer runden Öffnung dient zur Kabelführung. Dadurch können Stromversorgung und Datenleitungen sicher verlegt werden, ohne die Beweglichkeit der Kamera zu beeinträchtigen.
- **Schwenkmechanismus:** Ein angeschrägtes Bauteil mit einer Befestigungsbohrung dient zur Montage des Schwenkmechanismus. Im Grundgehäuse befindet sich auf der Unterseite eine Aussparung, in der der Servomotor eingesetzt wird. Die Achse des Servomotors wird anschließend in die Befestigungsbohrung des angeschrägten Bauteils geführt, um die Schwenkbewegung der Kamera zu ermöglichen.

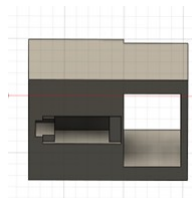


Abbildung 137:  
Aussparung für  
den Servomotor

Quelle: eigene  
Abbildung

---

**Montierung:**

Das Gehäuse für die Kameraschwenkung wird auf der unteren Verkanthrohr-Konstruktion befestigt. Dies sorgt für eine stabile und vibrationsarme Montage. Die Befestigung erfolgt über Schrauben und andere geeignete Verbindungselemente, um eine sichere Fixierung zu gewährleisten. Der Servomotor ist im Inneren des Gehäuses untergebracht und ermöglicht die präzise Schwenkbewegung der Kamera.



---

# 8 Sensoren und Aktoren

## 8.1 Abstandsensor

### 8.1.1 Grundprinzip

Der HC-SR04 ist ein Ultraschallsensor, der mithilfe von Schallwellen Entfernungen messen kann. Sein Funktionsprinzip basiert auf der Laufzeitmessung von Schallwellen. Dabei sendet der Sensor hochfrequente Ultraschallwellen aus, die von Objekten in der Umgebung reflektiert und anschließend wieder vom Sensor empfangen werden. Durch die Messung der Zeit, die der Schall für den Hin- und Rückweg benötigt, kann die Entfernung zum Objekt bestimmt werden.

#### 1. Senden eines Ultraschallimpulses

Der Ultraschallsensor verfügt über zwei Hauptkomponenten, der Trigger und das Echo. Zu Beginn des Messvorgangs sendet der Trigger einen kurzen Ultraschallimpuls aus. Dieser Impuls besteht aus acht Schallwellen mit einer Frequenz von 40 kHz, die sich mit Schallgeschwindigkeit in der Luft ausbreiten.

#### 2. Reflexion des Signals

Die ausgesendeten Ultraschallwellen bewegen sich durch die Luft, bis sie auf ein Hindernis treffen. Sobald dies geschieht, werden die Wellen reflektiert und in Richtung des Sensors zurückgesendet. Je nach Entfernung des Hindernisses dauert diese Reflexion unterschiedlich lange.

#### 3. Empfangen des Echos

Der Echo-Sensor registriert das reflektierte Signal und misst die Zeitspanne zwischen dem Aussenden und dem Empfang des Ultraschallimpulses. Diese Laufzeit des Schalls ist direkt proportional zur Entfernung des Hindernisses. Je weiter das Objekt entfernt ist, desto länger dauert es, bis das Echo zurückkehrt.

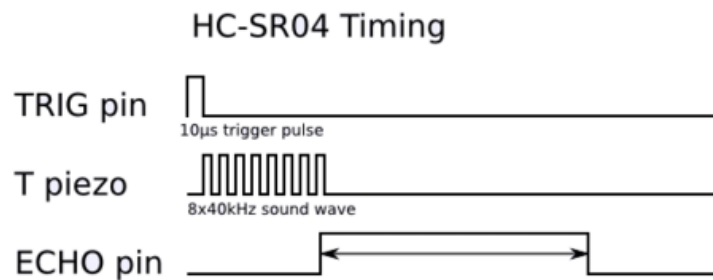


Abbildung 138: HC-SR04 Timing Diagram

Quelle:

[https://wiki.hshl.de/wiki/index.php/Ultraschallsensor\\_HC-SR04](https://wiki.hshl.de/wiki/index.php/Ultraschallsensor_HC-SR04)

#### 4. Berechnung der Entfernung

Die Entfernung zum Hindernis kann mit der folgenden Formel berechnet werden:

$$Distanz = \frac{\text{Schallgeschwindigkeit} \times \text{Laufzeit}}{2}$$

Da sich der Ultraschall mit einer Geschwindigkeit von ca. 343 m/s (bei 20°C) in der Luft ausbreitet und das Signal hin und zurück läuft, wird die Laufzeit durch zwei geteilt. Diese Berechnung ermöglicht eine präzise Abstandsmessung mit einem Ultraschallsensor.

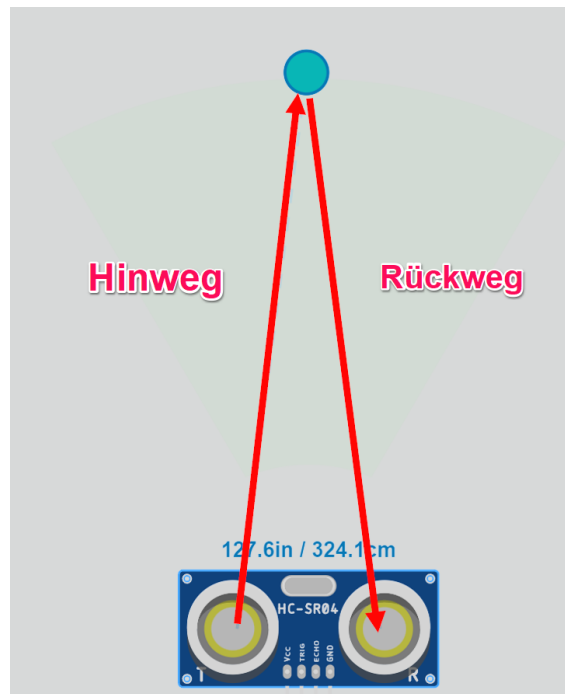


Abbildung 139: Beispiel für Messung

Quelle: <https://www.tinkercad.com/>

### 8.1.2 Schaltungsaufbau

Der Aufbau der Schaltung ist relativ einfach, da der Sensor nur vier Pins hat.

Der Sensor verfügt über folgende Anschlüsse:

- **VCC** → Versorgungsspannung (5V)
- **GND** → Masse
- **Trigger (TRIG)** → Eingangssignal zum Starten der Messung
- **Echo (ECHO)** → Ausgangssignal für die Messzeit

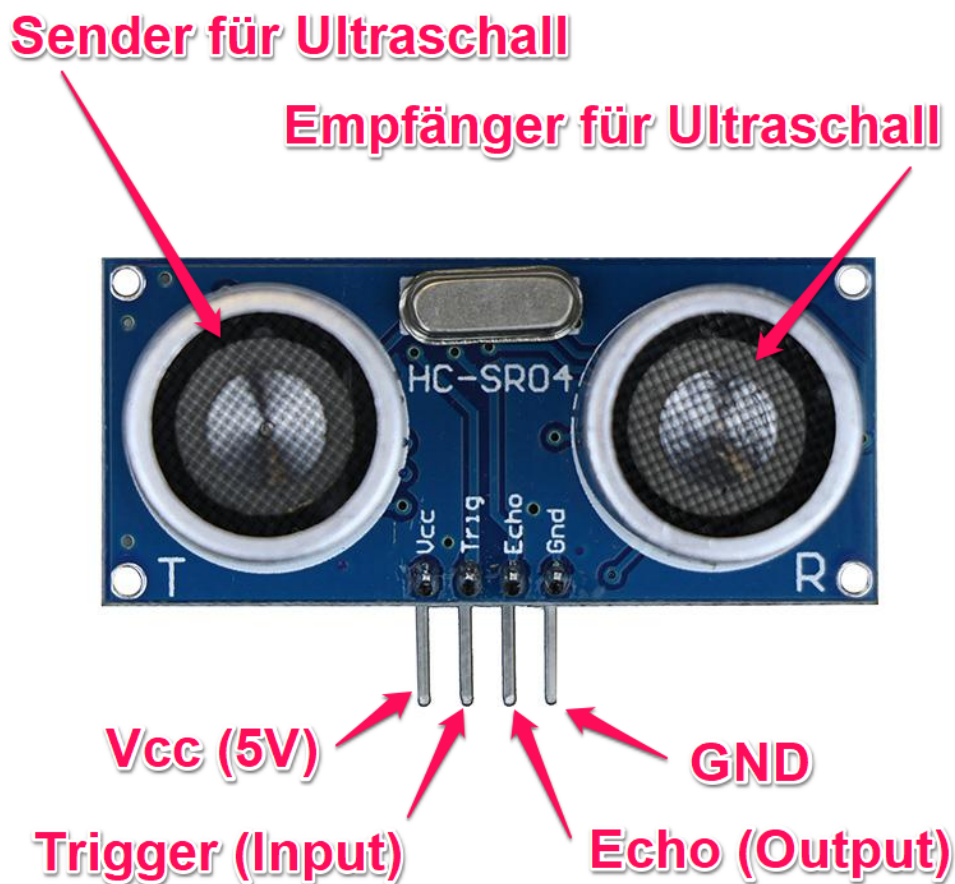


Abbildung 140: HC-SR04 Pinbelegung

Quelle: <https://at.rs-online.com/web/p/bbc-micro-bit-add-ons/2153181>

---

Der HC-SR04 Ultraschallsensor kann an beliebige digitale GPIO-Pins des ESP32 angeschlossen werden, solange man diese als Eingang (für Echo) und Ausgang (für Trigger) benutzen kann.

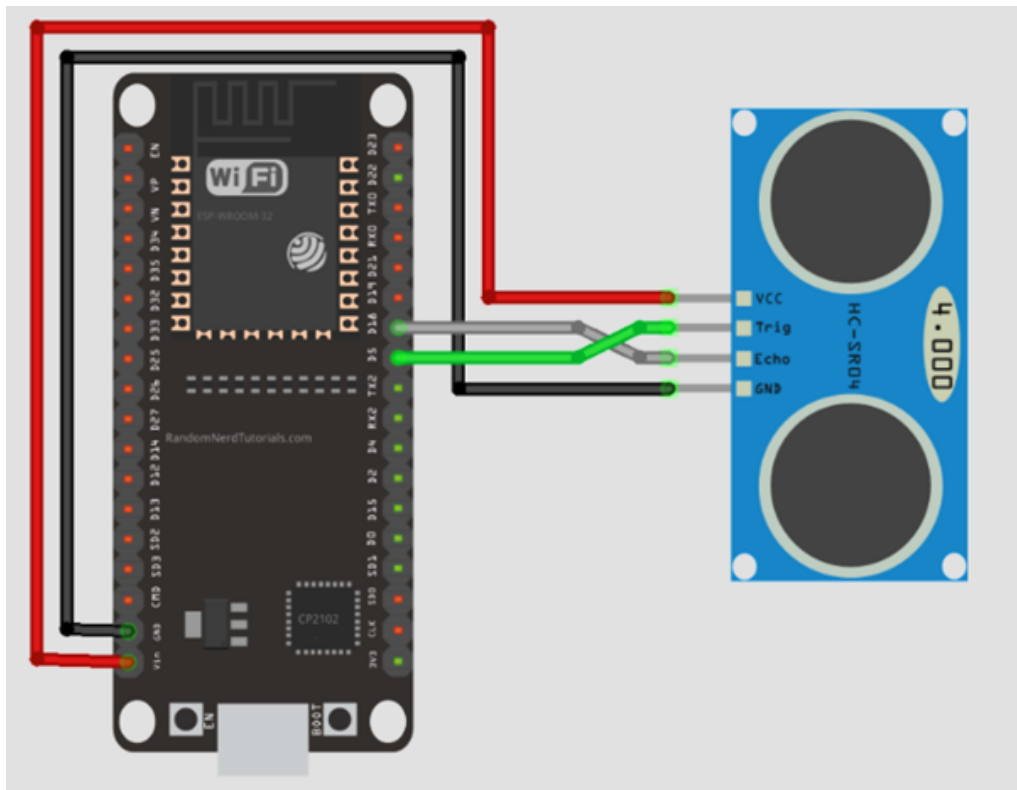


Abbildung 141: HC-SR04 Schaltungs Beispiel

Quelle: <https://randomnerdtutorials.com/esp32-hc-sr04-ultrasonic-arduino/>

### 8.1.3 Nutzung

Unser Lastenroboter hat zwei HC-SR04 Ultraschallsensoren verbaut, diese messen den Abstand vorne und hinten am Roboter. Diese Messungen sind wichtig für eine Sicherheitsfunktion, die verhindert, dass der Roboter unkontrolliert gegen ein Hindernis fährt.

Bei zu hoher Geschwindigkeit bremst der Roboter automatisch, um eine Kollision zu vermeiden. Fährt er jedoch langsam auf ein Objekt zu, bleibt die Bremse deaktiviert.

Dieses Verhalten ist ähnlich wie bei einem Auto, dass präzise Heranfahren an ein Hindernis ist bei langsamer Geschwindigkeit möglich, beispielsweise beim Einparken oder Positionieren. Bei hoher Geschwindigkeit kann es aber passieren das der Roboter jedoch nicht rechtzeitig stoppt, weshalb die automatische Bremse greift. Dadurch wird verhindert, dass der Roboter mit voller Wucht gegen ein Hindernis, wie etwa eine Mauer, fährt.

---

### 8.1.4 Code

Um die Ultraschallsensoren leicht zu verwenden, wird die HCSR04-Bibliothek eingebunden. Diese Bibliothek vereinfacht die Steuerung der HC-SR04-Sensoren, indem sie die Entfernungsmessung automatisch berechnet.

```
#include "HCSR04.h"
```

Abbildung 142: HC-SR04.h Bibliothek

Quelle: eigene Abbildung

Nach dem Einbinden der Bibliothek, werden Objekte der Klasse `UltraSonicDistanceSensor` erstellt, mit dem später auf die Ultraschallsensoren zugegriffen und Entfernungen gemessen werden können.

`distanceSensor_rear(23, 19);` ist der Sensor für die hintere Messung

`distanceSensor_front(13, 12);` ist der Sensor für die vordere Messung.

```
UltraSonicDistanceSensor distanceSensor_rear(23, 19);  
UltraSonicDistanceSensor distanceSensor_front(13,12);
```

Abbildung 143: Erstellung von Objekten der Klasse `UltraSonicDistanceSensor`

Quelle: eigene Abbildung

Danach wird die ganze Zeit in der `loop()` die aktuelle Entfernung für den vorderen, als auch für den hinteren Ultraschallsensor ermittelt. Dazu wird die Methode „`measureDistanceCm()`“ der jeweiligen Sensorobjekte aufgerufen, um die Entfernung in cm zu messen. Die Messergebnisse werden in die dementsprechende float-Variable gespeichert.

```
distance_front = distanceSensor_front.measureDistanceCm();  
distance_rear = distanceSensor_rear.measureDistanceCm();
```

Abbildung 144: Messung der Entfernung in der `loop()`-Funktion

Quelle: eigene Abbildung

Mit einer if-Abfrage prüft der Code, ob der Roboter zu nah an ein Hindernis kommt und dabei zu schnell ist. Wenn die gemessene Entfernung vorne oder hinten zwischen fünf und zwanzig Zentimetern liegt und die Geschwindigkeit mindestens fünfzig beträgt, wird die Funktion `stop()` ausgeführt. Dadurch stoppt der Roboter automatisch, um eine Kollision zu vermeiden.

---

Der Bereich von fünf bis zwanzig cm wurde gewählt, weil der Sensor bei sehr großen Entfernungen, ungenaue oder falsche Signale liefern kann. Dadurch dass nur Messwerte innerhalb dieses Bereichs berücksichtigt werden, wird sichergestellt, dass der Roboter nur dann stoppt, wenn tatsächlich ein Hindernis erkannt wird.

```
if(((distance_front >= 5 && distance_front <= 20) && speed_cb >= 50) ||  
((distance_rear >= 5 && distance_rear <= 20) && speed_cb >= 50)){  
    stop();  
}
```

Abbildung 145: if-Abfrage für Mindestabstand und Geschwindigkeit

Quelle: eigene Abbildung

## 8.2 Gewichtsmessung

### 8.2.1 Grundprinzip

Um das Gewicht, das sich auf dem Roboter befinden zu wiegen, werden Wägezellen benutzt. Wägezellen sind Widerstände, die sich unter Krafteinfluss verändern. Diese Veränderung des Widerstandes kann man dann benutzen, um das darauf drückende Gewicht zu messen. Wägezellen bestehen meistens aus einem Federkörper und einem Dehnmessstreifen. Der Federkörper ist ein biegsames Metallstück, das sich Verbiegt, wenn Gewicht darauf einwirkt. Diese Dehnung wird dann mit dem Dehnmessstreifen gemessen.

Wägezellen gibt es in unterschiedlichen Größen, von kleinen Modellen für geringe Lasten bis hin zu größeren Varianten für hohe Gewichte. Beim Lastenroboter kommen „Halbbrücken-Wägezellen“ zum Einsatz.



Abbildung 146: Halbbrücken-Wägezelle

Quelle: <https://bienenwaage.shop/auswahl-der-waegezelle>

Zum Auslesen der Wägezellen wird ein HX711 verwendet. Der HX711 ist ein präziser 24-Bit-Analog-Digital-Wandler, der speziell für Wägezellen entwickelt wurde. Er verstärkt und digitalisiert die schwachen Signale der Sensoren, sodass sie von Mikrocontrollern verarbeitet werden können. Die Daten des HX711 werden anschließend über eine serielle Datenverbindung an den ESP32 übertragen. Der Mikrocontroller empfängt die digitalen Signale und verarbeitet sie weiter, um das Gewicht präzise zu berechnen.

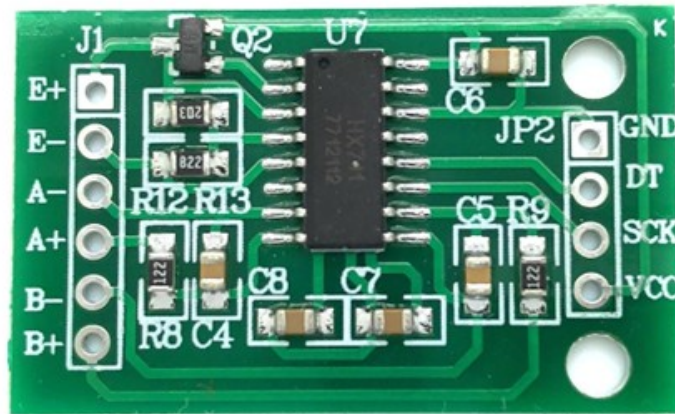


Abbildung 147: HX711 24-bit A/D-Wandler

Quelle: <https://www.berrybase.at/hx711-24-bit-a-d-gewichtssensor>

### 1. Kraftaufnahme durch die Wägezellen

Wird eine Last auf das System aufgebracht, verformen sich die Halbbrücken-Wägezellen minimal. Diese Verformung führt zu einer Änderung des elektrischen Widerstands der Dehnungsmessstreifen in der Wägezelle.

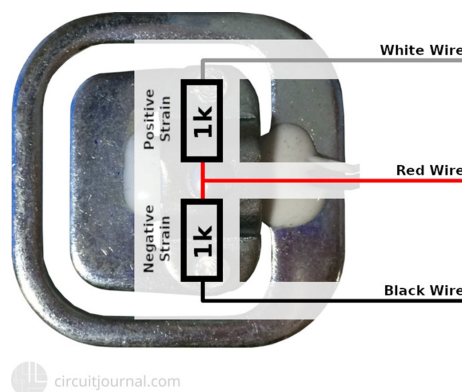


Abbildung 148: Prinzip einer Wägezelle

Quelle:

<https://circuitjournal.com/50kg-load-cells-with-HX711>

## 2. Widerstandsänderung und Signalbildung

Die Halbbrücken-Wägezellen sind Teil einer Wheatstone-Brückenschaltung. Durch die Widerstandsänderung entsteht eine kleine Spannungsdifferenz, die proportional zur aufgetragenen Last ist.

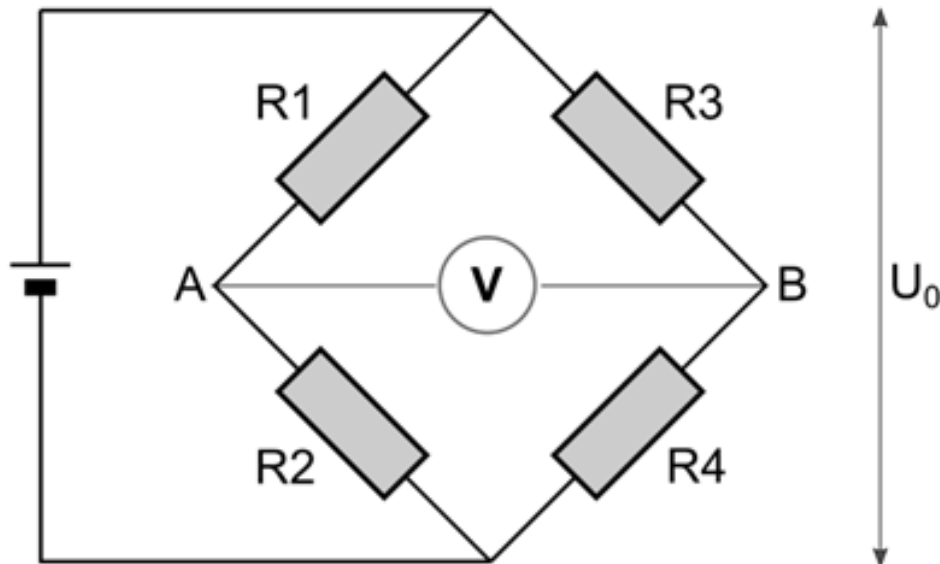


Abbildung 149: Wheatstone-Brückenschaltung

Quelle: <https://wolles-elektronikkiste.de/dehnungsmessstreifen>

## 3. Signalverstärkung durch den HX711

Der HX711 nimmt diese schwache Differenzspannung auf und verstärkt sie mit einem internen Verstärker. Die Verstärkung kann auf 32x, 64x oder 128x eingestellt werden, um auch sehr kleine Laständerungen messbar zu machen.

## 4. Analog-Digital-Wandlung durch den HX711

Die verstärkte Spannung wird im HX711 in ein digitales 24-Bit-Signal umgewandelt. Dadurch kann das Gewicht mit hoher Genauigkeit erfasst werden.

## 5. Analog-Digital-Wandlung durch den HX711

Der ESP32 empfängt die digitalen Werte über eine serielle Datenverbindung. Dann werden die Rohdaten interpretiert und in eine Gewichtseinheit (z. B. Kilogramm) umgerechnet. Eine vorherige Kalibrierung stellt sicher, dass die Messung korrekt erfolgt.



### 8.2.2 Schaltungsaufbau

Die vier Wägezellen sind in einer Wheatstone-Vollbrücke verbunden, dadurch kann die Gewichtsmessung präziser erfolgen und die Belastung auf die Plattform gleichmäßig erfasst werden, unabhängig von der Position der Last. Die vier Wägezellen sind so verbunden, dass ihre Signale zusammengeführt und an die differentiellen Eingänge des HX711 weitergegeben werden.

Für den Anschluss werden die folgenden Pins des HX711 genutzt:

- **E+ (Excitation +)** – Versorgungsspannung für die Wägezellen
- **E- (Excitation -)** – Masse der Wägezellen
- **A+ (Signal +)** – Positives Ausgangssignal der Brückenschaltung
- **A- (Signal -)** – Negatives Ausgangssignal der Brückenschaltung

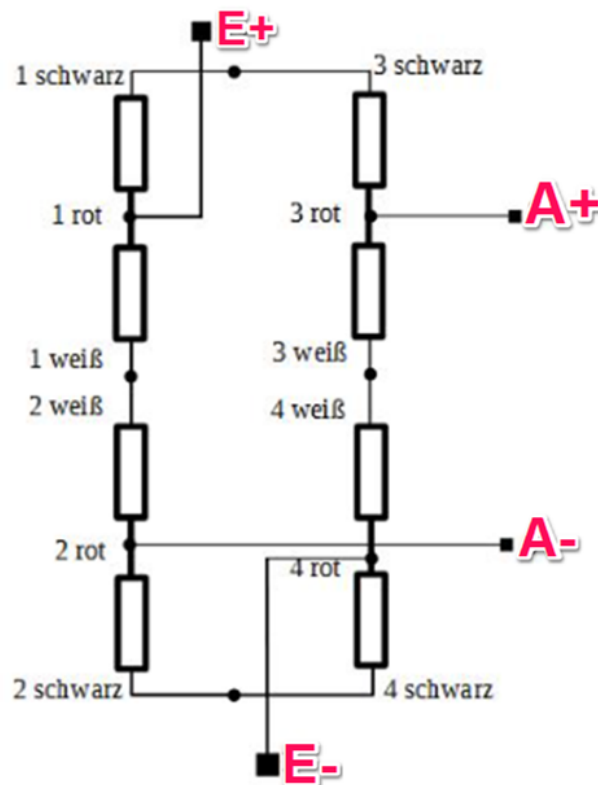


Abbildung 150: Beschaltung der Wägezellen

Quelle:

[https://beelogger.de/stockwaage/aufbauanleitung\\_mit\\_halbbrueckensensoren](https://beelogger.de/stockwaage/aufbauanleitung_mit_halbbrueckensensoren)

Der HX711 überträgt die digitalisierten Messwerte an den ESP32 über eine serielle Datenverbindung.:

- **DT (Data)** – Dieser Pin überträgt die digitalen Messwerte vom HX711 und wird mit dem I2C\_SDA-Pin (GPIO 21) des ESP32 verbunden.
- **SCK (Clock)** – Dieser Pin steuert die Taktung der Datenübertragung und wird mit dem I2C\_SCL-Pin (GPIO 22) des ESP32 verbunden.

Obwohl der HX711 kein echtes I2C-Protokoll verwendet, wird er oft ähnlich angeschlossen und mit einer einfachen seriellen Bit-Kommunikation ausgelesen. Der ESP32 sendet dabei regelmäßig ein Signal über den SCK-Pin, um die Datenübertragung zu starten, und liest anschließend über den DT-Pin die Gewichtswerte aus.

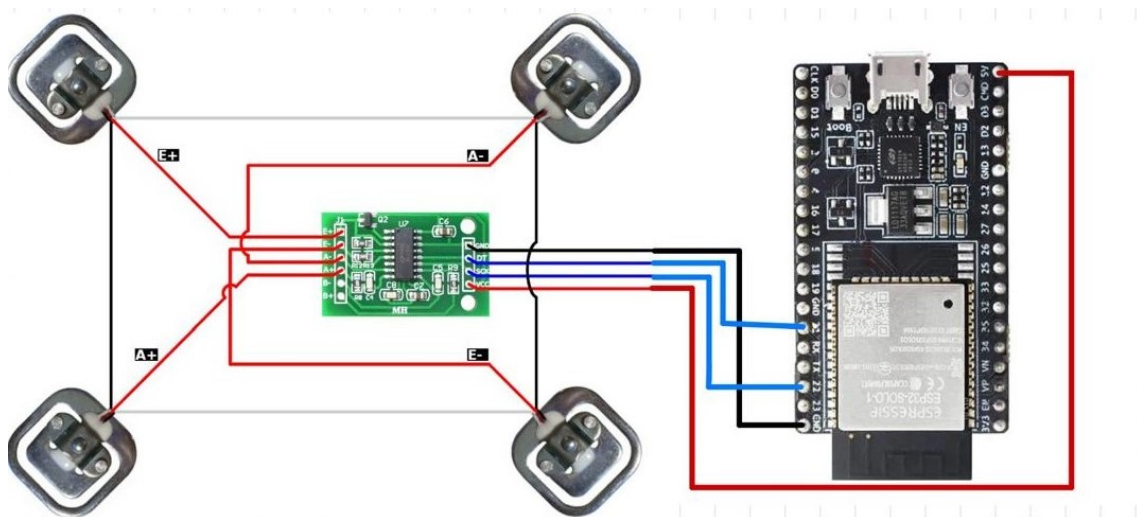


Abbildung 151: Verkabelung der Wägezellen mit HX711  
Quelle: eigene Abbildung

### 8.2.3 Nutzung

Die Wägezellen dienen dazu, das Gewicht auf dem Lastenroboter präzise zu messen. Der Mikrocontroller verarbeitet die vom HX711 empfangenen Daten und sendet sie in Echtzeit an die Website, auf der das aktuelle Gewicht des Roboters in einem Untermenü angezeigt wird.

Dadurch kann jederzeit überprüft werden, wie viel Last sich auf dem Roboter befindet. Diese Funktion trägt zur Überwachung des Gewichts bei und hilft Überlastungen bei den Transportprozessen zu vermeiden.

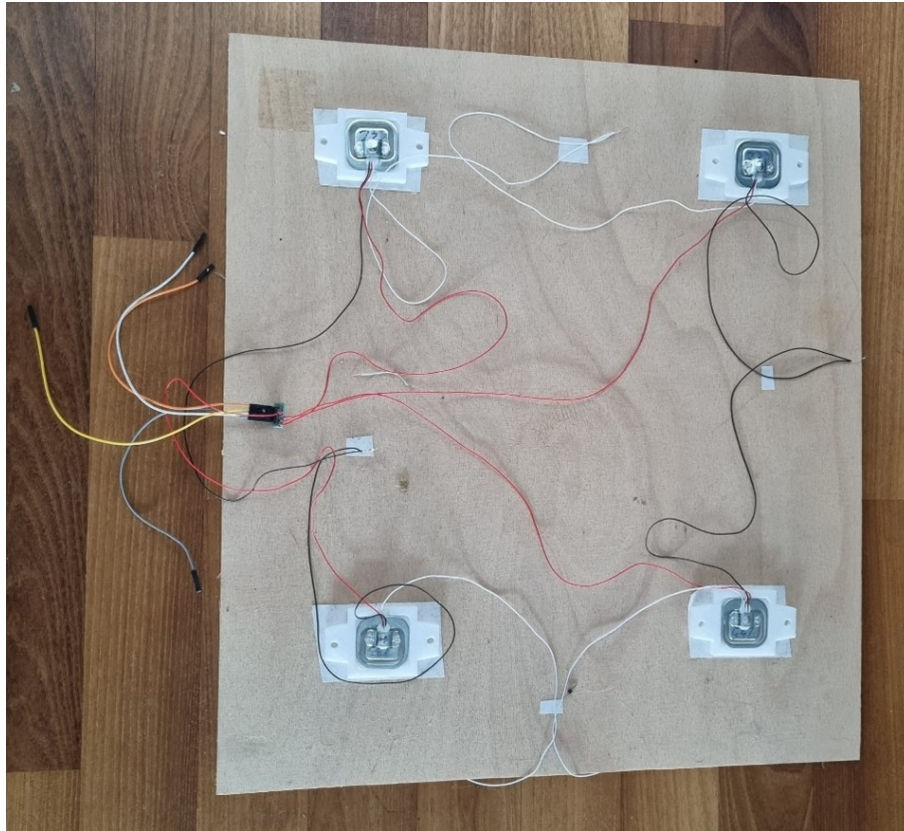


Abbildung 152: Testaufbau der Wägezellen  
Quelle: eigene Abbildung

#### 8.2.4 Code

Um die Wägezellen mit dem HX711 einfach auszulesen, wird die HX711\_ADC- Bibliothek verwendet. Diese Bibliothek vereinfacht durch ihre vielen fertigen Funktionen das Auslesen und das Umrechnen der Daten. Nach Einbinden der Bibliothek, werden die GPIOs für das Auslesen festgelegt. Mit diesen wird dann ein Objekt der Klasse HX711\_ADC erstellt, um auf den HX711 zuzugreifen. Außerdem wird eine Float-Variable für das Gewicht deklariert.

```
#include <HX711_ADC.h>
const int HX711_dout = 21;
const int HX711_sck = 22;
HX711_ADC LoadCell(HX711_dout, HX711_sck);
float weight = 0.0;
```

Abbildung 153: Einbinden der HX711\_ADC-Bibliothek  
Quelle: eigene Abbildung

---

Dannach wird mit `LoadCell.begin()`; der HX711-Sensor in der `Setup()`-Funktion gestartet. Anschließend wird die Variable `calibrationValue` als `float` definiert und auf `696.0` gesetzt. Dieser Wert dient zur Kalibrierung der Messung und muss an die spezifische Wägezelle angepasst werden.

Die Variable `stabilizingtime` vom Typ `unsigned long` wird auf `2000` Millisekunden gesetzt. Diese gibt an, wie lange der Sensor Zeit bekommt, um sich nach dem Start zu stabilisieren.

Die Variable `_tare` ist ein `Boolean` und wird auf `true` gesetzt. Das bedeutet, dass der Sensor beim Start automatisch „tarieren“ soll, also das aktuelle Gewicht als Nullpunkt speichert.

Schließlich wird mit `LoadCell.start(stabilizingtime, _tare)`; der Messvorgang gestartet, wobei die angegebene Stabilisierungszeit und die automatische Tara-Funktion berücksichtigt werden.

```
LoadCell.begin();  
float calibrationValue;  
calibrationValue = 696.0;  
unsigned long stabilizingtime = 2000;  
boolean _tare = true;  
LoadCell.start(stabilizingtime, _tare);
```

Abbildung 154: `Setup()` ; - Code für den HX711  
Quelle: eigene Abbildung

Dann wird noch überprüft, ob die Tara-Funktion des HX711 erfolgreich abgeschlossen wurde oder ob ein Timeout-Fehler aufgetreten ist.

Zuerst wird geprüft, ob die Tara-Funktion zu lange gedauert hat und fehlgeschlagen ist. Falls dies der Fall ist, wird eine Fehlermeldung über die serielle Schnittstelle ausgegeben. Anschließend wird eine Endlosschleife gestartet, die das Programm anhält.

Falls kein Timeout auftritt, wird die Kalibrierung durchgeführt. Danach wird über die serielle Schnittstelle eine Bestätigung ausgegeben, dass die Initialisierung erfolgreich abgeschlossen wurde.

```

if (LoadCell.getTareTimeoutFlag())
{
    Serial.println("Timeout, check MCU>HX711 wiring and pin designations");
    while (1)
    {
        ;
    }
}
else
{
    LoadCell.setCalFactor(calibrationValue);
    Serial.println("Startup is complete");
}

```

Abbildung 155: Timeout Abfrage in der Setup() ;-Funktion  
Quelle: eigene Abbildung

In der loop() -Funktion wird kontinuierlich das Gewicht gemessen. Die getData() -Funktion ruft dabei den aktuellen Messwert vom HX711 A/D-Wandler ab und speichert ihn in der Variablen „weight“.

Dieser Wert stellt das aktuell gemessene Gewicht dar, das mithilfe der Kalibrierung in eine gewichtsspezifische Einheit umgerechnet wird. Dieser Wert kann dann auf die Website übertragen werden.

```

weight = LoadCell.getData();

```

Abbildung 156: Gewichtsmessung in der loop() ;-Funktion  
Quelle: eigene Abbildung

## 8.3 Spannungsmessung per ESP32

### 8.3.1 Code

Bei der Spannungsmessung wird kein eigentlicher Sensor verwendet stattdessen dient ein analoger Eingang des ESP32 selbst zur Spannungsmessung, wodurch der Mikrocontroller direkt als Messeinheit fungiert.

Zur Bestimmung des Akkustands wird die Spannung am analogen Eingang gemessen und über einen Spannungsteiler korrekt auf die tatsächliche Eingangsspannung zurückgerechnet. Zunächst werden mit define drei wichtige Konstanten festgelegt:

Der analoge Eingangspin (PIN\_TEST) ist auf GPIO 34 gesetzt. Die Referenzspannung (REF\_VOLTAGE) des ESP32 beträgt 3,3 Volt, und der ADC (Analog-Digital-Wandler) hat eine 12-Bit-Auflösung, was 4095 möglichen Stufen entspricht (PIN\_STEPS).

---

Zur Messung der hohen Spannungen wird ein Spannungsteiler mit zwei Widerständen verwendet: R1 (120 kOhm) und R2 (10,8 kOhm). Dieser reduziert die Eingangsspannung auf einen für den ESP32 messbaren Bereich. Die tatsächliche Spannung am Eingang des Spannungsteilers, also die reduzierte Spannung, wird später als `vout` gespeichert. Die berechnete Originalspannung vor dem Spannungsteiler wird in `vin` abgelegt. Der vom ADC gelieferte Rohwert wird in der Variable `rawValue` gespeichert.

```
// Spannung-Messung für Akkustand
#define PIN_TEST 34           // Analoger Eingangspin
#define REF_VOLTAGE 3.3      // Referenzspannung des ESP32
#define PIN_STEPS 4095.0     // ADC-Auflösung des ESP32 (12-bit)
const float R1 = 120000.0;   // Widerstand R1 (120 kOhm)
const float R2 = 10800.0;    // Widerstand R2 (11 kOhm)
float vout = 0.0;            // Gemessene Ausgangsspannung
float vin = 0.0;             // Berechnete Eingangsspannung
✓ int rawValue = 0;          // Rohwert vom ADC
//-----
```

Abbildung 157: Variablen für die Spannungsmessung  
Quelle: eigene Abbildung

### Messung und Berechnung der Spannungswerte:

- Der analoge Eingangspin `PIN_TEST` wird im Setup als Eingang konfiguriert, damit Spannungen gelesen werden können.
- `rawValue = analogRead(PIN_TEST);`  
Der Rohwert vom ADC wird am analogen Pin 34 eingelesen.
- `vout = (rawValue * REF_VOLTAGE) / PIN_STEPS;`  
Der Rohwert wird in eine Spannung (`vout`) umgerechnet, basierend auf der Referenzspannung (3,3 V) und der 12-Bit-Auflösung (4095 Schritte).
- `vin = vout / (R2 / (R1 + R2));`  
Über den Spannungsteilerfaktor wird die ursprüngliche Eingangsspannung (`vin`) berechnet – also die tatsächliche Akkuspannung vor dem Teiler.
- `if (vin < 0.09)`  
Falls die berechnete Spannung kleiner als 0,09 V ist (z. B. wegen Rauschen oder fehlerhafter Messung), wird `vin` auf 0 V gesetzt, um unrealistische Werte zu vermeiden.

```
//Im Setup Code:
pinMode(PIN_TEST, INPUT);

// Spannungsmessung in Loop() Code
rawValue = analogRead(PIN_TEST);
vout = (rawValue * REF_VOLTAGE) / PIN_STEPS;
vin = vout / (R2 / (R1 + R2));
if (vin < 0.09)
{
    vin = 0.0;
}
```

Abbildung 158: Messung der Spannung  
Quelle: eigene Abbildung

Danach wird die zuvor berechnete Eingangsspannung (vin) verwendet, um den Akkustand in Prozent zu berechnen und über die serielle Schnittstelle auszugeben. Zunächst wird vin um 2 V als Korrektur erhöht und mit einer Nachkommastelle formatiert angezeigt. Anschließend wird der Akkustand in Prozent berechnet, wobei angenommen wird, dass 9 V einem leeren Akku (0%) und 13 V einem vollen Akku (100%) entsprechen.

Der berechnete Prozentwert wird auf die nächsten 10% gerundet, um eine übersichtliche Anzeige zu ermöglichen. Schließlich wird der gerundete Akkustand als Prozentwert ohne Nachkommastellen über die serielle Schnittstelle ausgegeben.

```
Serial.println("U = " + String(vin + 2, 1) + " V");
float batteryPercentage = ((vin - 9) / 4) * 100;
float akku_round = round(batteryPercentage / 10) * 10;
Serial.println("Akkustand: " + String(akku_round, 0) + "%");
```

Abbildung 159: Berechnung der Spannung in Prozent  
Quelle: eigene Abbildung



---

Dann werden der berechnete Akkustand `akku_round` und der Gewichtswert (`weight`)<sup>4</sup> in Strings umgewandelt und anschließend in eine JSON-formatierte Nachricht verpackt. Die Nachricht enthält zwei Schlüssel: “battery” und “weight”, denen die entsprechenden Werte als Text zugewiesen werden.

Diese Nachricht wird dann über eine WebSocket-Verbindung mit `websocket.broadcastTXT()` an den verbundenen Client gesendet. Dadurch sieht man dann auf der Website den Akkustand der Batterie.

```
String batterystatus = String(akku_round, 0);
String weightstatus = String(weight, 0);
String message = "{\"battery\": \"" + batterystatus
+ "\", \"weight\": \"" + weightstatus + "\"}";
websocket.broadcastTXT(message.c_str());
```

Abbildung 160: Werte in JSON umwandeln und an Client senden  
Quelle: eigene Abbildung

### 8.3.2 Verbesserungen und Optimierungen

Die hier verwendete Methode zur Berechnung des Akkustands ist nur eine grobe Schätzung. Sie geht davon aus, dass der Akku zwischen 9 V (0%) und 13 V (100%) eine lineare Entladung zeigt. In der Realität ist die Entladekurve eines Akkus jedoch nicht linear: Die Spannung bleibt über einen langen Zeitraum relativ konstant und fällt erst am Ende der Entladung schnell ab.

Deshalb liefert eine einfache lineare Berechnung oft ungenaue oder irreführende Prozentwerte. Um realistischere und genauere Werte zu erhalten, sollte man stattdessen eine Mapping-Funktion oder eine Kennlinie per Tabelle oder mathematischer Funktion verwenden, die die tatsächliche Entladekurve des Akkutyps berücksichtigt.

## 8.4 Lichtsteuerung per Transistor

### 8.4.1 Code

Am Anfang wird mit `const int light_pin = 0;` festgelegt, dass das Licht über Pin 0 des Expander-Boards (MCP) gesteuert wird. An diesem Pin ist ein Transistor angeschlossen, der das Licht ein- und ausschaltet. Der Transistor dient dabei als elektronischer Schalter, über das Signal vom Expander-Pin wird der Transistor geschaltet, der dann das Licht steuert.

---

<sup>4</sup>Siehe Kapitel 8.2 für Details zur Gewichtsmessung



---

Die Zeile `int light_st = 0;` legt eine Variable an, die den aktuellen Zustand des Lichts speichert, also ob es gerade an oder aus ist.

```
const int light_pin = 0;  
int light_st = 0;
```

Abbildung 161: Pin und Variable für Lichtsteuerung  
Quelle: eigene Abbildung

Damit der Pin überhaupt ein Ausgang ist, wird im `Setup()`; mit `mcp.pinMode(light_pin, OUTPUT)`; festgelegt, dass dieser Pin als Ausgang (OUTPUT) arbeitet.

```
mcp.pinMode(light_pin, OUTPUT);
```

Abbildung 162: `Setup()`; - Code für Lichtsteuerung  
Quelle: eigene Abbildung

Danach wird geprüft, ob im vom Webserver erhaltenen JSON ein Eintrag mit dem Namen "light\_status" vorhanden ist. Wenn ja, wird der Wert daraus ausgelesen und in der Variable `light_status` gespeichert. Anschließend wird geprüft, ob `light_st` gleich 1 ist. Falls ja, wird die Funktion `lights_on()`; aufgerufen, und das Licht geht an. Wenn nicht, wird `lights_off()`; ausgeführt, um das Licht auszuschalten.

```
else if (jsonDoc.containsKey("light_status"))  
{  
    bool light_status = jsonDoc["light_status"];  
    light_st = light_status ? 1 : 0;  
  
    if (light_st == 1)  
    {  
        lights_on();  
    }  
    else  
    {  
        lights_off();  
    }  
}
```

Abbildung 163: `onWebSocketEvent()`; - Code für Lichtsteuerung  
Quelle: eigene Abbildung

---

Wenn `lights_on()` aufgerufen wird, passiert Folgendes:

- Im seriellen Monitor wird „Licht an“ ausgegeben.
- Dann wird über `mcp.digitalWrite(light_pin, HIGH)`; ein HIGH-Signal an den entsprechenden Pin des MCP-Expanders geschickt. Dadurch wird der Transistor aktiviert und das Licht geht an.

In `lights_off()` läuft es genau umgekehrt:

- Es wird „Licht aus“ im seriellen Monitor ausgegeben.
- Mit `mcp.digitalWrite(light_pin, LOW)`; wird ein LOW-Signal geschickt, der Transistor wird deaktiviert, und das Licht geht aus.

```
void lights_on()
{
    Serial.println("Licht an");
    mcp.digitalWrite(light_pin, HIGH);
}

void lights_off()
{
    Serial.println("Licht aus");
    mcp.digitalWrite(light_pin, LOW);
}
```

Abbildung 164: Funktionen zum Ein-/Ausalten des Lichtes

Quelle: eigene Abbildung

---

## 9 Entwicklungstools

### 9.1 Autodesk Fusion

Autodesk Fusion ist eine cloudbasierte 3D-CAD-, CAM- und CAE-Software, die für die Konstruktion, Simulation und Fertigung von Bauteilen verwendet wird. Die Software bietet eine Vielzahl von Funktionen, um den Entwicklungsprozess effizient zu gestalten, darunter parametrisches Modellieren, Freiformmodellierung und Baugruppenverwaltung. Autodesk Fusion ist unter <https://www.autodesk.de/products/fusion> verfügbar und kann sowohl als Desktop-Anwendung als auch über die Cloud genutzt werden.

Für unser Projekt wurde Autodesk Fusion verwendet, um komplexe Bauteile zu entwerfen und zu optimieren. Die Software ermöglicht eine benutzerfreundliche Bedienung und bietet leistungsstarke Werkzeuge zur Analyse und Simulation.

### 9.2 Autodesk Eagle

Autodesk Eagle ist eine leistungsstarke Software zur Erstellung von Leiterplatten-Layouts (PCB-Design). Sie wird vor allem für das Entwerfen und Platzieren elektronischer Schaltungen genutzt. Eagle bietet umfangreiche Funktionen wie einen Schematic-Editor, ein PCB-Layout-Tool und eine große Bauteilbibliothek. Die Software ist unter <https://www.autodesk.de/products/eagle> verfügbar und kann sowohl eigenständig als auch in Kombination mit Autodesk Fusion verwendet werden.

Für unser Projekt wurde Autodesk Eagle zur Entwicklung und Optimierung elektronischer Schaltungen genutzt. Durch die benutzerfreundliche Benutzeroberfläche und die Bibliotheken mit zahlreichen vorgefertigten Bauteilen konnte der Entwicklungsprozess effizient gestaltet werden.

### 9.3 VS-Code

Visual Studio Code (VS-Code) ist eine kostenlose IDE (integrated development environment) entwickelt von Microsoft. VS-Code funktioniert auch auf anderen Betriebssystemen wie zum Beispiel Windows, Linux oder macOS. VS-Code unterstützt einen Großteil der Programmiersprachen und kann durch Extensions mit vielen nützlichen Features und Sprachen immer wieder erweitert werden.<sup>5</sup>

---

<sup>5</sup><https://code.visualstudio.com/>

---

### 9.3.1 Setup

Um ein Projekt in VS-Code erstellen zu können, müssen einige Schritte befolgt werden. Zuerst muss die IDE von <https://code.visualstudio.com/> für das jeweilig passende Betriebssystem heruntergeladen und installiert werden. Um Mikrocontroller wie ESPs oder Arduinos in VS-Code programmieren zu können, wird die PlatformIO IDE Extension benötigt. Um die PlatformIO IDE Extension in VS-Code zu installieren, drückt man einfach auf das Extensions Symbol oder drückt die Tastenkombination Ctrl+Shift+X um das Extensions Menü zu öffnen. Danach gibt man in der Suchleiste "PlatformIO IDE" ein und wählt die Extension mit der Ameise als Icon. Dann drückt man auf "Install" und wartet, bis die Extension fertig heruntergeladen ist. Nach der Installation sollte das PlatformIO Icon (Ameisenkopf) auf der linken Seite unter dem Extension Menü erscheinen.

Um nun ein neues Projekt zu erstellen, klickt man auf das PlatformIO Icon und wählt "+ New Project".

Im Project Wizard wählt man nun den gewünschten Namen, das Board, das Framework als auch den Speicherort des Projektes. Bei unserem Projekt wählten wir das Board "Espressif ESP32 Dev Module" und als Framework "Arduino", da wir einen ESP32 zum Programmieren verwendeten.

### 9.3.2 Bibliotheken

Bibliotheken sind ein weiterer wichtiger Bestandteil für das Programmieren. Bibliotheken beinhalten bereits eine Sammlung von vorgefertigtem Code, der für bestimmte Aufgaben, wie zum Beispiel zum Auswerten eines Sensors, verwendet werden kann. Bibliotheken werden verwendet, um den Code zu minimieren und dadurch die Lesbarkeit sowie die Effizienz zu steigern. Um eine Bibliothek für PlatformIO in VS-Code zu installieren, muss das PIO Home Menü geöffnet werden. Darin befindet sich der Reiter „Libraries“. Wenn dieses geöffnet wird, erscheint eine Suchleiste, mit der die gewünschten Bibliotheken zum Projekt hinzugefügt werden können.

### 9.3.3 Verwendete Bibliotheken

#### ESPAsyncWebServer

Die ESPAsyncWebServer Bibliothek ermöglicht es, Webanwendungen effizient und mit hoher Performance auf ESP8266- und ESP32 Mikrocontroller zu hosten. Der asynchrone Betrieb verhindert Blockierungen und sorgt für eine flüssige Verarbeitung mehrere Anfragen gleichzeitig. Außerdem unterstützt die Bibliothek WebSockets, welche für die Echtzeitkommunikation zwischen Client und Server benötigt wurden. Die Bibliothek ist eine leistungsfähigere Alternative zur klassischen

---

WebServer-Bibliothek, da sie ressourcenschonender und nicht blockieren arbeitet.<sup>6</sup>

## **ArduinoJSON**

Die ArduinoJson Bibliothek ermöglicht die Verarbeitung von JSON-String in Objekte und umgekehrt. Sie ist speziell für Geräte mit begrenztem Speicher und Rechenleistung optimiert. Die Bibliothek wird benötigt, um die erhaltenen Steuerbefehle am ESP32 zu konvertieren und um sie anschließend weiterzuverarbeiten.<sup>7</sup>

## **HCSR04**

Die HCSR04.h-Bibliothek ermöglicht es mit einem ESP32, das HC-SR04 Ultraschallmodul zur Entfernungsmessung zu nutzen. Sie erlaubt die Verwendung mehrerer Sensoren gleichzeitig, um die aktuelle Distanz in cm zu erfassen.<sup>8</sup>

## **arduinoWebSockets**

Die WebSockets Bibliothek ermöglicht eine Kommunikation über das WebSocket Protokoll für Arduino-Boards, ESP8266 und ESP32. Die Bibliothek wird für die Echtzeit-Kommunikation zwischen dem Webserver und den ESP32 benötigt. Außerdem werden die Bilder der ESP32-CAM über einen WebSocket an den Webserver gesendet. Die Bibliothek ist eine großartige Ergänzung zur ESPAsyncWebServer Bibliothek.<sup>9</sup>

## **ESP32Servo**

Die ESP32Servo-Bibliothek ermöglicht es, Servomotoren mit einem ESP32 einfach zu steuern. Sie unterstützt viele Servotypen und funktioniert ähnlich wie die Arduino Servo-Bibliothek<sup>10</sup>. Zusätzlich bietet sie zwei Funktionen zur Anpassung der PWM-Timerbreite des ESP32 für genauere Steuerung.<sup>11</sup>

## **Adafruit\_MCP23x17**

Die Adafruit MCP23017 Arduino Library ermöglicht die Nutzung des MCP23017 I/O-Expanders mit Arduino und ESP32. Dadurch lassen sich 16 zusätzliche digitale Ein- und Ausgänge über I2C oder SPI steuern. Die Bibliothek bietet einfache Funktionen für Input, Output, Pull-ups und Interrupts. So können Projekte mit mehr Ein- und Ausgängen erweitert werden, ohne zusätzliche Pins am Mikrocontroller zu belegen.<sup>12</sup>

---

<sup>6</sup><https://github.com/lacamera/ESPAsyncWebServer>

<sup>7</sup><https://arduinojson.org/>

<sup>8</sup><https://docs.arduino.cc/libraries/hcsr04-ultrasonic-sensor/>

<sup>9</sup><https://github.com/Links2004/arduinoWebSockets>

<sup>10</sup><https://docs.arduino.cc/libraries/servo/>

<sup>11</sup><https://github.com/madhephaestus/ESP32Servos>

<sup>12</sup><https://github.com/adafruit/Adafruit-MCP23017-Arduino-Library>

---

## HX711\_ADC

Die HX711\_ADC-Bibliothek ermöglicht die Nutzung des HX711 ADC-Moduls mit Arduino und ESP32 zur präzisen Messung von Gewichtssensoren (Wägezellen). Sie bietet eine verbesserte Signalverarbeitung, Tare-Funktion und Kalibrierungsfunktionen. Die Bibliothek erleichtert das Auslesen der Sensordaten und sorgt für eine stabile Gewichtsmessung, ohne komplexe Signalverarbeitung manuell durchführen zu müssen.<sup>13</sup>

## 9.4 LaTeX

LaTeX ist ein Textsatzsystem, das besonders für die Erstellung von wissenschaftlichen Arbeiten, technischen Dokumentationen und anderen anspruchsvollen Texten verwendet wird. Es basiert auf dem Prinzip der Auszeichnungssprache, bei dem Inhalt und Formatierung getrennt voneinander behandelt werden. Dadurch lassen sich Dokumente sehr präzise und konsistent gestalten.

Wir haben für unser Projekt LaTeX benutzt, um die Dokumentation zu schreiben. Dabei haben wir das Programm TeXstudio<sup>14</sup> verwendet. Es hilft uns, den Text zu schreiben und gleich zu sehen, wie das fertige Dokument aussieht.

Damit alle immer die aktuelle Version unserer Datei haben und nichts verloren geht, werden unsere LaTeX-Dateien in unserem gemeinsamen GitHub-Repository gespeichert. So kann jeder aus dem Team die Datei bearbeiten und alle sehen sofort die neueste Version.

## 9.5 GitHub

GitHub ist eine webbasierte Plattform zur Versionsverwaltung und gemeinsam Softwareentwicklung. GitHub bietet Funktionen verschiedene Funktionen um den Entwicklungsprozess effizient zu gestalten. Die Plattform ist unter <https://github.com/> erreichbar und kann sowohl über die Web-Oberfläche als auch über lokale Tools wie GitHub Desktop benutzt werden.

Für eine einfache Verwaltung wurde für unser Projekt GitHub Desktop verwendet. GitHub Desktop ist eine benutzerfreundliche Anwendung, die eine grafische Oberfläche für die Verwaltung von Git-Repositories bietet. Sie erleichtert das Klonen, Committen, Pushen und Pullen von Projekten, ohne dass Befehle in der Kommandozeile erforderlich sind. Das Tool kann unter <https://desktop.github.com/> heruntergeladen werden.

Unser Projekt ist unter <https://github.com/Sparify/Carybot> zu finden.

---

<sup>13</sup>[https://github.com/olkal/HX711\\_ADC](https://github.com/olkal/HX711_ADC)

<sup>14</sup><https://www.texstudio.org/>

# 10 Abbildungsverzeichnis

## Abbildungsverzeichnis

|    |                                                                                                |    |
|----|------------------------------------------------------------------------------------------------|----|
| 1  | Porträt Daniel Schauer . . . . .                                                               | 9  |
| 2  | Porträt Simon Spari . . . . .                                                                  | 9  |
| 3  | Porträt Felix Hochegger . . . . .                                                              | 9  |
| 4  | Lastenroboter Projekt-Grobplan . . . . .                                                       | 10 |
| 5  | Lastenroboter Projekt-Plan . . . . .                                                           | 11 |
| 6  | Motoren . . . . .                                                                              | 13 |
| 7  | BLDC-Motor . . . . .                                                                           | 14 |
| 8  | Phasensteuerung . . . . .                                                                      | 14 |
| 9  | Motorteiber-ZS-X11H . . . . .                                                                  | 15 |
| 10 | Motorphasen Verbindung . . . . .                                                               | 16 |
| 11 | Pins für die PWM . . . . .                                                                     | 16 |
| 12 | Pin für die Bremse und den Richtungswechsel . . . . .                                          | 17 |
| 13 | Versorgungs-Pins . . . . .                                                                     | 17 |
| 14 | Hall-sensoren Eingänge . . . . .                                                               | 18 |
| 15 | Schaltungsaufbau mit einem Motor . . . . .                                                     | 19 |
| 16 | Hall-Sensoren Eingänge . . . . .                                                               | 20 |
| 17 | Hall-Sensoren-Motor . . . . .                                                                  | 20 |
| 18 | Motor Phasen . . . . .                                                                         | 20 |
| 19 | Treiber Phasen . . . . .                                                                       | 20 |
| 20 | Versorgung der Motoren und Treiberplatine . . . . .                                            | 20 |
| 21 | Ansteuerung der Treiberplatine . . . . .                                                       | 21 |
| 22 | Motoren-GPIO Konfiguration . . . . .                                                           | 23 |
| 23 | Adafruit_MCP23X17.h-Bibliothek . . . . .                                                       | 23 |
| 24 | MCP-Setup . . . . .                                                                            | 24 |
| 25 | Initialisierung Motoren . . . . .                                                              | 24 |
| 26 | Enumeration von Bewegungsrichtungen . . . . .                                                  | 25 |
| 27 | Strings zu Enum Variable umwandeln . . . . .                                                   | 26 |
| 28 | navigate() in loop() . . . . .                                                                 | 26 |
| 29 | Überprüfung auf Änderung und Speed in Integer umwandeln in navigate()-<br>Funktion . . . . .   | 27 |
| 30 | Motorbremse deaktivieren in navigate()-Funktion . . . . .                                      | 27 |
| 31 | switch-case Struktur in navigate()-Funktion . . . . .                                          | 28 |
| 32 | PWM mit analogWrite übertragen und letzte Werte speichern in navigate()-<br>Funktion . . . . . | 29 |

---

|    |                                                         |    |
|----|---------------------------------------------------------|----|
| 33 | ESPAsyncWebServer.h-Bibliothek . . . . .                | 31 |
| 34 | WebServer Netzwerkkonfiguration . . . . .               | 31 |
| 35 | Webserver Setup . . . . .                               | 32 |
| 36 | SPIFFS Initialisierung . . . . .                        | 33 |
| 37 | readfile()-Funktion . . . . .                           | 34 |
| 38 | Bibliotheken für die Kommunikation . . . . .            | 34 |
| 39 | Setup ESP32 . . . . .                                   | 35 |
| 40 | onWebSocketEvent()-Funktion . . . . .                   | 36 |
| 41 | JSON-Beispiel für Steuerung . . . . .                   | 37 |
| 42 | JSON-Beispiel für Kamerasteuerung . . . . .             | 37 |
| 43 | JSON-Beispiel für Lichtsteuerung . . . . .              | 37 |
| 44 | handleWebSocketMessage()-Funktion . . . . .             | 38 |
| 45 | Steuerkreuz auf der Website . . . . .                   | 39 |
| 46 | Steuerkreuz HTML-Code . . . . .                         | 39 |
| 47 | CSS-Klassen .dpad-container und .button . . . . .       | 40 |
| 48 | CSS-Richtungsklassen . . . . .                          | 41 |
| 49 | EventListener für ungewünschte Aktionen . . . . .       | 42 |
| 50 | EventListener für Eingabe . . . . .                     | 42 |
| 51 | send()-Funktion . . . . .                               | 43 |
| 52 | stop()-Funktion . . . . .                               | 44 |
| 53 | Geschwindigkeitssteuerung auf der Website . . . . .     | 44 |
| 54 | Geschwindigkeitssteuerung HTML-Code . . . . .           | 45 |
| 55 | CSS-Klassen für die Geschwindigkeitssteuerung . . . . . | 45 |
| 56 | speed-change()-Funktion . . . . .                       | 46 |
| 57 | Kamerasteuerung auf der Website . . . . .               | 46 |
| 58 | Kamerasteuerung HTML-Code . . . . .                     | 47 |
| 59 | CSS-Klassen für die Kamerasteuerung . . . . .           | 47 |
| 60 | camera-change()-Funktion . . . . .                      | 48 |
| 61 | Fernlicht-Icon auf der Website . . . . .                | 48 |
| 62 | HTML-Code für Fernlicht . . . . .                       | 49 |
| 63 | CSS-Klassen für das Fernlicht . . . . .                 | 49 |
| 64 | togglelight()-Funktion . . . . .                        | 50 |
| 65 | Menü-Icon auf der Website . . . . .                     | 51 |
| 66 | Sidebar mit Sensordaten . . . . .                       | 51 |
| 67 | Sensordaten HTML-Code . . . . .                         | 52 |
| 68 | CSS-Klassen für das Menü . . . . .                      | 53 |
| 69 | CSS-Klasse .daten . . . . .                             | 54 |
| 70 | JavaScript-Verarbeitung der Sensordaten . . . . .       | 54 |
| 71 | Statusbox auf der Website . . . . .                     | 55 |
| 72 | Statusbox: Verbunden . . . . .                          | 55 |



---

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 73  | Statusbox: Verbindung getrennt . . . . .                  | 55 |
| 74  | Statusbox HTML-Code . . . . .                             | 55 |
| 75  | CSS-Klassen für die Statusbox . . . . .                   | 56 |
| 76  | JavaScript Funktionen für den Verbindungsstatus . . . . . | 56 |
| 77  | Erstes Modell . . . . .                                   | 59 |
| 78  | Zweites Modell . . . . .                                  | 60 |
| 79  | Rahmen . . . . .                                          | 60 |
| 80  | Halterung 1 . . . . .                                     | 61 |
| 81  | Halterung 2 . . . . .                                     | 61 |
| 82  | Motorenhalterung . . . . .                                | 61 |
| 83  | Räder . . . . .                                           | 62 |
| 84  | Wägezellen . . . . .                                      | 62 |
| 85  | Wägezellenhalterung . . . . .                             | 63 |
| 86  | Fertiges Modell . . . . .                                 | 63 |
| 87  | Stahlprofile . . . . .                                    | 64 |
| 88  | Oberer Rahmen . . . . .                                   | 65 |
| 89  | Elektrodenschweißen . . . . .                             | 66 |
| 90  | Elektrodenschweißen . . . . .                             | 66 |
| 91  | Motorbefestigung . . . . .                                | 67 |
| 92  | Motorbefestigung . . . . .                                | 67 |
| 93  | WERKA PRO 10 . . . . .                                    | 68 |
| 94  | Verbindungselement . . . . .                              | 69 |
| 95  | Verbindungselement . . . . .                              | 69 |
| 96  | Verbindungselement . . . . .                              | 69 |
| 97  | Integration der Wägezelle . . . . .                       | 69 |
| 98  | Erster funktionierender Prototyp . . . . .                | 70 |
| 99  | Schaltplan Steuerplatine . . . . .                        | 74 |
| 100 | Schaltplan Spannungsversorgung . . . . .                  | 75 |
| 101 | Schraubklemmen . . . . .                                  | 75 |
| 102 | Schaltplan Masseanschlüsse . . . . .                      | 76 |
| 103 | Pinheader . . . . .                                       | 76 |
| 104 | 5V-Spannungsregelung . . . . .                            | 76 |
| 105 | LM317 . . . . .                                           | 77 |
| 106 | Spannungsregler . . . . .                                 | 78 |
| 107 | Schaltplan Spannungsversorgung . . . . .                  | 79 |
| 108 | Schaltplan ESP32 . . . . .                                | 79 |
| 109 | ESP32-Anschlüsse . . . . .                                | 80 |
| 110 | MCP23017 . . . . .                                        | 80 |
| 111 | expander-Anschlüsse . . . . .                             | 81 |
| 112 | Lichtsteuerung . . . . .                                  | 82 |

---

|     |                                                                       |     |
|-----|-----------------------------------------------------------------------|-----|
| 113 | Batteriemessung . . . . .                                             | 83  |
| 114 | Power-LED . . . . .                                                   | 83  |
| 115 | pullup-widerstände . . . . .                                          | 84  |
| 116 | schaltplan-wägezellen . . . . .                                       | 84  |
| 117 | i2c-anschlüsse . . . . .                                              | 85  |
| 118 | RX/TX . . . . .                                                       | 85  |
| 119 | PCB . . . . .                                                         | 86  |
| 120 | Steuerungsplatine . . . . .                                           | 87  |
| 121 | ESP32-CAM Modul . . . . .                                             | 88  |
| 122 | Videoübertragung auf der Website . . . . .                            | 90  |
| 123 | Kamera-GPIO Konfiguration . . . . .                                   | 91  |
| 124 | Kamera-Initialisierung . . . . .                                      | 92  |
| 125 | Halterung der Kamera . . . . .                                        | 93  |
| 126 | Videoübertragung HTML-Code . . . . .                                  | 93  |
| 127 | CSS-Formatierung für dynamicimage . . . . .                           | 94  |
| 128 | Videoübertragung im JavaScript-Code . . . . .                         | 94  |
| 129 | SG90-Servomotor . . . . .                                             | 95  |
| 130 | Verkabelung des Servomotors . . . . .                                 | 96  |
| 131 | ESP32Servo-Bibliothek . . . . .                                       | 96  |
| 132 | Variablen für Servo . . . . .                                         | 97  |
| 133 | Setup() ; -Code für Servo . . . . .                                   | 97  |
| 134 | camerapos . . . . .                                                   | 98  |
| 135 | Funktion zum Kamera drehen . . . . .                                  | 98  |
| 136 | Kameragehäuse . . . . .                                               | 99  |
| 137 | Aussparung für den Servomotor . . . . .                               | 99  |
| 138 | HC-SR04 Timing Diagram . . . . .                                      | 102 |
| 139 | Beispiel für Messung . . . . .                                        | 102 |
| 140 | HC-SR04 Pinbelegung . . . . .                                         | 103 |
| 141 | HC-SR04 Schaltungs Beispiel . . . . .                                 | 104 |
| 142 | HC-SR04.h Bibliothek . . . . .                                        | 105 |
| 143 | Erstellung von Objekten der Klasse UltraSonicDistanceSensor . . . . . | 105 |
| 144 | Messung der Entfernung in der loop()-Funktion . . . . .               | 105 |
| 145 | if-Abfrage für Mindestabstand und Geschwindigkeit . . . . .           | 106 |
| 146 | Halbbrücken-Wägezelle . . . . .                                       | 106 |
| 147 | HX711 24-bit A/D-Wandler . . . . .                                    | 107 |
| 148 | Prinzip einer Wägezelle . . . . .                                     | 107 |
| 149 | Wheatstone-Brückenschaltung . . . . .                                 | 108 |
| 150 | Beschaltung der Wägezellen . . . . .                                  | 109 |
| 151 | Verkabelung der Wägezellen mit HX711 . . . . .                        | 110 |
| 152 | Testaufbau der Wägezellen . . . . .                                   | 111 |

---

|     |                                                         |     |
|-----|---------------------------------------------------------|-----|
| 153 | Einbinden der HX711_ADC-Bibliothek . . . . .            | 111 |
| 154 | Setup(); - Code für den HX711 . . . . .                 | 112 |
| 155 | Timeout Abfrage in der Setup();-Funktion . . . . .      | 113 |
| 156 | Gewichtsmessung in der loop();-Funktion . . . . .       | 113 |
| 157 | Variablen für die Spannungsmessung . . . . .            | 114 |
| 158 | Messung der Spannung . . . . .                          | 115 |
| 159 | Berechnung der Spannung in Prozent . . . . .            | 115 |
| 160 | Werte in JSON umwandeln und an Client senden . . . . .  | 116 |
| 161 | Pin und Variable für Lichtsteuerung . . . . .           | 117 |
| 162 | Setup(); - Code für Lichtsteuerung . . . . .            | 117 |
| 163 | onWebSocketEvent(); - Code für Lichtsteuerung . . . . . | 117 |
| 164 | Funktionen zum Ein-/Ausalten des Lichtes . . . . .      | 118 |

# 11 Literaturverzeichnis

## Literaturverzeichnis

|    |                                                                                                                                                                               |     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | <a href="https://github.com/lacamera/ESPAsyncWebServer">https://github.com/lacamera/ESPAsyncWebServer</a> . . . . .                                                           | 31  |
| 2  | <a href="https://www.lorch.eu/de/produkte/manuelles-schweissen/elektroden-schweissen">https://www.lorch.eu/de/produkte/manuelles-schweissen/elektroden-schweissen</a> . . . . | 65  |
| 3  | <a href="https://www.mikrocontroller.net/articles/Basiswiderstand">https://www.mikrocontroller.net/articles/Basiswiderstand</a> . . . . .                                     | 82  |
| 4  | Siehe Kapitel 8.2 für Details zur Gewichtsmessung . . . . .                                                                                                                   | 116 |
| 5  | <a href="https://code.visualstudio.com/">https://code.visualstudio.com/</a> . . . . .                                                                                         | 119 |
| 6  | <a href="https://github.com/lacamera/ESPAsyncWebServer">https://github.com/lacamera/ESPAsyncWebServer</a> . . . . .                                                           | 121 |
| 7  | <a href="https://arduinojson.org/">https://arduinojson.org/</a> . . . . .                                                                                                     | 121 |
| 8  | <a href="https://docs.arduino.cc/libraries/hcsr04-ultrasonic-sensor/">https://docs.arduino.cc/libraries/hcsr04-ultrasonic-sensor/</a> . . . . .                               | 121 |
| 9  | <a href="https://github.com/Links2004/arduinoWebSockets">https://github.com/Links2004/arduinoWebSockets</a> . . . . .                                                         | 121 |
| 10 | <a href="https://docs.arduino.cc/libraries/servo/">https://docs.arduino.cc/libraries/servo/</a> . . . . .                                                                     | 121 |
| 11 | <a href="https://github.com/madhephaestus/ESP32Servos">https://github.com/madhephaestus/ESP32Servos</a> . . . . .                                                             | 121 |
| 12 | <a href="https://github.com/adafruit/Adafruit-MCP23017-Arduino-Library">https://github.com/adafruit/Adafruit-MCP23017-Arduino-Library</a> . . . . .                           | 121 |
| 13 | <a href="https://github.com/olka1/HX711_ADC">https://github.com/olka1/HX711_ADC</a> . . . . .                                                                                 | 122 |
| 14 | <a href="https://www.texstudio.org/">https://www.texstudio.org/</a> . . . . .                                                                                                 | 122 |